

SIMULAÇÃO DE ESTRATÉGIAS DE CONTROLE PARA ESTABILIZAÇÃO
DE MANIPULADORES SUBAQUÁTICOS COM MÚLTIPLOS GRAUS DE
LIBERDADE

Lucas da Silva Santos

Projeto Final apresentado ao Departamento de Engenharia de Controle e Automação do Centro Federal de Educação Tecnológica Celso Suckow da Fonseca *campus* Nova Iguaçu, como parte dos requisitos necessários à obtenção do título de Bacharel em Engenharia de Controle e Automação.

Orientadores: Luciano Santos Constantin
Raptopoulos
Josiel Alves Gouvêa

Nova Iguaçu
Julho de 2025

SIMULAÇÃO DE ESTRATÉGIAS DE CONTROLE PARA ESTABILIZAÇÃO
DE MANIPULADORES SUBAQUÁTICOS COM MÚLTIPLOS GRAUS DE
LIBERDADE

Lucas da Silva Santos

PROJETO FINAL SUBMETIDO AO CORPO DOCENTE DO CENTRO
FEDERAL DE EDUCAÇÃO TECNOLÓGICA CELSO SUCKOW DA FONSECA
COMO PARTE DOS REQUISITOS NECESSÁRIOS PARA A OBTENÇÃO DO
GRAU DE BACHAREL EM ENGENHARIA DE CONTROLE E AUTOMAÇÃO.

Examinado por:

Prof. Nome do Primeiro Examinador Sobrenome, D.Sc.

Prof. Nome do Segundo Examinador Sobrenome, Ph.D.

Prof. Nome do Terceiro Examinador Sobrenome, D.Sc.

Prof. Nome do Quarto Examinador Sobrenome,
Dr.-Ing.

Prof. Nome do Quinto Examinador Sobrenome, M.Sc.

NOVA IGUAÇU, RJ – BRASIL
JULHO DE 2025

da Silva Santos, Lucas

SIMULAÇÃO DE ESTRATÉGIAS DE CONTROLE
PARA ESTABILIZAÇÃO DE MANIPULADORES
SUBAQUÁTICOS COM MÚLTIPLOS GRAUS DE
LIBERDADE/Lucas da Silva Santos. – Nova Iguaçu:
CEFET/RJ *campus* Nova Iguaçu, 2025.

XVII, 165 p.: il.; 29, 7cm.

Orientadores: Luciano Santos Constantin Raptopoulos
Josiel Alves Gouvêa

Projeto Final – CEFET/RJ *campus* Nova
Iguaçu/Departamento de Engenharia de Controle e
Automação, 2025.

Referências Bibliográficas: p. 138 – 143.

1. Manipuladores Subaquáticos. 2. Modelagem
Simbólica. 3. Dinâmica Lagrangiana. I. Santos
Constantin Raptopoulos, Luciano *et al.* II. Centro Federal
de Educação Tecnológica Celso Suckow da Fonseca *campus*
Nova Iguaçu, Departamento de Engenharia de Controle e
Automação. III. Título.

*Ao meu saudoso pai, cuja
ausência fortaleceu minha
jornada, e às minhas amadas
mãe e irmã, exemplos de
coragem e resiliência em todos os
momentos da vida.*

Agradecimentos

Agradeço, primeiramente, a Deus, por me conceder forças e perseverança ao longo desta caminhada.

Aos meus orientadores, Prof. Dr. Luciano Santos Constantin Raptopoulos e Prof. Dr. Josiel Alves Gouvêa, pela orientação precisa, paciência e pelas valiosas contribuições técnicas ao desenvolvimento deste trabalho.

Aos colegas de curso, pelo apoio mútuo durante os desafios da graduação, especialmente nos momentos de maior dificuldade.

À minha família, pelo amor incondicional, incentivo e compreensão em todos os momentos.

A todos os professores do CEFET/RJ - campus Nova Iguaçu, por compartilharem seus conhecimentos ao longo desses anos de formação.

Aos amigos e demais pessoas que, direta ou indiretamente, contribuíram para a realização deste projeto, minha sincera gratidão.

Resumo do Projeto Final apresentado ao CEFET/RJ *campus* Nova Iguaçu como parte dos requisitos necessários para a obtenção do grau de Bacharel em Engenharia de Controle e Automação.

SIMULAÇÃO DE ESTRATÉGIAS DE CONTROLE PARA ESTABILIZAÇÃO DE MANIPULADORES SUBAQUÁTICOS COM MÚLTIPLOS GRAUS DE LIBERDADE

Lucas da Silva Santos

Julho/2025

Orientadores: Luciano Santos Constantin Raptopoulos
Josiel Alves Gouvêa

Departamento: Engenharia de Controle e Automação

Este trabalho apresenta o *Hephaestus*, um ambiente computacional de código aberto, desenvolvido em Python, para a modelagem dinâmica simbólica e a simulação de manipuladores robóticos seriais e de Sistemas Manipuladores-Veículos Subaquáticos (UVMSs). A plataforma deduz automaticamente, pela formulação de Euler-Lagrange, as equações de movimento do sistema e as compila em funções numéricas otimizadas, viabilizando a comparação direta de quatro estratégias de controle não-linear (CTC-PID, CT-SMC, Super-Twisting e LADRC) sob uma interface gráfica unificada. A validação é realizada em um UVMS de doze graus de liberdade em ambiente subaquático, em que todos os controladores apresentaram rastreamento preciso da trajetória, observando-se que o LADRC e o CT-SMC se destacaram pela eficiência energética e o Super-Twisting pelo menor erro de rastreamento. O *Hephaestus* é proposto como ferramenta didática que une, em um único ambiente livre, a modelagem matemática rigorosa e a simulação de alto desempenho.

Palavras-chave: Sistemas Manipuladores-Veículos Subaquáticos. Modelagem Simbólica Lagrangiana. Controle Não-Linear. Python. SymPy.

Abstract of Bachelor Report presented to CEFET/RJ *campus* Nova Iguaçu as a partial fulfillment of the requirements for the degree of Bachelor in Control and Automation Engineering.

SIMULATION OF CONTROL STRATEGIES FOR STABILIZATION OF UNDERWATER MANIPULATORS WITH MULTIPLE DEGREES OF FREEDOM

Lucas da Silva Santos

July/2025

Advisors: Luciano Santos Constantin Raptopoulos
Josiel Alves Gouvêa

Department: Control and Automation Engineering

This work presents *Hephaestus*, an open-source computational environment developed in Python for the symbolic dynamic modeling and simulation of serial robotic manipulators and Underwater Vehicle-Manipulator Systems (UVMSs). The platform automatically derives the equations of motion of the system through the Euler-Lagrange formulation and compiles them into optimized numerical functions, enabling the direct comparison of four nonlinear control strategies (CTC-PID, CT-SMC, Super-Twisting, and LADRC) under a unified graphical interface. Validation is carried out on a twelve-degree-of-freedom UVMS in an underwater environment, in which all controllers achieved accurate trajectory tracking, with LADRC and CT-SMC standing out for their energy efficiency and Super-Twisting for the lowest tracking error. *Hephaestus* is proposed as a didactic tool that combines, within a single free environment, rigorous mathematical modeling and high-performance simulation.

Keywords: Underwater Vehicle-Manipulator Systems. Lagrangian Symbolic Modeling. Nonlinear Control. Python. SymPy.

Lista de Figuras

1.1	Arquitetura geral do ambiente <i>Hephaestus</i> : fluxo de trabalho desde a especificação da cadeia cinemática (módulo Modelagem) até a simulação em malha fechada (módulo Simulação) e a comparação de resultados (módulo Análise). As dependências entre os módulos <code>engine.py</code> e <code>simulator.py</code> e os quatro controladores não-lineares são destacadas.	3
3.1	Fluxo de cinemática simbólica implementado em <code>step_1_kinematics</code> : da especificação de <code>joint_config</code> e <code>link_vectors_mask</code> à construção das transformações homogêneas acumuladas T_i , das posições dos centros de massa $\vec{p}_{\text{cm},i}$, das velocidades angulares $\vec{\omega}_i$ e do Jacobiano geométrico completo $J(q) \in \mathbb{R}^{6 \times N}$.	34
3.2	Fluxo de derivação simbólica da matriz de inércia $M(q)$, do vetor de gradiente de potencial $G(q)$ e dos coeficientes de Coriolis pelo método dos símbolos de Christoffel, implementado em <code>step_2_jacobian_M_G</code> e <code>step_3_coriolis_combined</code> .	37
3.3	Extensão hidrodinâmica implementada em <code>RobotMathHydro</code> : herança de <code>RobotMathEngine</code> com sobrescrita dos métodos de cinemática e dinâmica para incorporar o tensor de massa adicionada $M_A^{(g)}$ (Equação (3.19)) e o potencial aparente Peso–Empuxo (Equação (3.21)) segundo a formulação de Fossen [1].	44
3.4	Fluxo de compilação simbólico-numérica via <code>sp.lambdify()</code> com CSE ativo: as expressões SymPy de M , C , G e J passam pela Eliminação de Subexpressões Comuns antes de gerar código Python/NumPy compilado, reduzindo em 60–90% o número de operações primitivas e viabilizando a avaliação em tempo de simulação [2, 3].	48
3.5	Paralelismo em dois níveis no <i>Hephaestus</i> : (Nível 1) derivação paralela de $\partial M / \partial q_k$ e das linhas de C por <code>ProcessPoolExecutor</code> durante a geração <i>offline</i> do modelo; (Nível 2) avaliação paralela e persistente de <code>func_M</code> , <code>func_C</code> e <code>func_G</code> a cada passo de integração física durante a simulação <i>online</i> .	50

4.1	Fase de inicialização do simulador: resolução da IK estática por Levenberg-Marquardt com busca em linha (Seção 4.3), configuração do controlador e do executor paralelo, e verificação das condições de estabilidade numérica (Seção 4.9).	54
4.2	Passo de controle físico executado a cada $substep \Delta t_{phys}$: planejamento de trajetória, CLIK 6D, avaliação paralela de M , C , G , cálculo do torque de controle, aplicação das forças dissipativas e integração simplética de Euler (Equações (4.25)–(4.26)).	55
4.3	Fluxo de planejamento de trajetória no espaço cartesiano: o parâmetro cúbico $s(\tau)$ (Equação (4.1)) é aplicado ao segmento de reta (Equações (4.3)–(4.5)) ou ao arco circular (Equações (4.7)–(4.9)), gerando triplas $\{P_{ref}, \dot{P}_{ref}, \ddot{P}_{ref}\}$ a cada passo de integração para o CLIK [4, 5].	59
4.4	Modos de referência de orientação do efetuador disponíveis no <i>Hephaestus</i> : Livre (apenas controle posicional), Fixa, Tangente à Trajetória, Apontar para o Alvo, SLERP geodésico [6] e Normal à Superfície. Os modos 2–6 ativam o Jacobiano completo $J \in \mathbb{R}^{6 \times N}$ e o erro de orientação da Equação (4.19).	62
4.5	Fluxo do algoritmo CLIK (<i>Closed-Loop Inverse Kinematics</i>) implementado no <i>Hephaestus</i> : recebe as referências cartesianas $\{P_{ref}, \dot{P}_{ref}, \ddot{P}_{ref}\}$ e produz as referências de junta $\{q_d, \dot{q}_d, \ddot{q}_d\}$ por pseudo-inversão amortecida DLS (Equação (4.15)) com gestão de redundância por projeção no espaço nulo [7–9].	69
5.1	Estrutura do controlador Torque Computado com PID (CTC-PID): o sinal auxiliar v (Equação (5.3)) é pré-multiplicado por $M(q)$ e somado aos termos de cancelamento $C(q, \dot{q})$ e $G(q)$, resultando em uma dinâmica de erro linear de segunda/terceira ordem (Equação (5.4)). O integrador é sujeito a <i>anti-windup</i> por <i>clamping</i> [10].	87
5.2	Estrutura do Controle por Modo Deslizante com Torque Computado (CT-SMC): o sinal de chaveamento $-K \tanh(S/\phi)$ garante a condição de alcance $\dot{V}_i < 0$ para perturbações $\ d_i\ \leq K_i$ (Equação (5.9)). O filtro passa-baixas de $\ddot{q}_{d,f}$ previne amplificação de picos de aceleração do CLIK [11, 12].	91
5.3	Estrutura do algoritmo Super-Twisting (STA): a variável interna z_i (Equação (5.11)) integra o sinal descontínuo $-k_{2,i} \text{sign}(S_i)$, tornando $v_{sw,i}$ contínuo e eliminando o <i>chattering</i> . As condições de ganho $k_{2,i} > \delta_i$ e $k_{1,i} > 2\sqrt{k_{2,i}\delta_i}$ garantem convergência em tempo finito [13, 14].	95

5.4	Observador de Estado Estendido (ESO) de terceira ordem do LADRC: estima o estado ($z_1 \approx q_i$, $z_2 \approx \dot{q}_i$, $z_3 \approx f_i$) a partir da medição de posição q_i e da entrada de controle anterior. Os ganhos $\beta_k = \binom{3}{k} \omega_o^k$ alocam todos os polos do observador em $-\omega_o$ [15, 16].	100
5.5	Lei de controle do LADRC (Equações (5.22)–(5.23)): o distúrbio estimado $z_{3,i}^{\text{filt}}$ é subtraído da ação de controle PD $u_{0,i}$ e o resultado é dividido por $b_{0,i}$; o <i>feedforward</i> explícito de G_i e C_i reduz o distúrbio residual que o ESO precisa rastrear [15, 17, 18].	101
6.1	Visualização 3D do UVMS de 12 GDL durante a simulação.	117
6.2	Relatório comparativo dos controladores Torque Computado, ADRC, CT-SMC e Super-Twisting para o UVMS de 12 GDL.	119
6.3	Erro (rad) e torque (Nm) do controlador Torque Computado para o UVMS de 12 GDL.	120
6.4	Erro (rad) e torque (Nm) do controlador ADRC para o UVMS de 12 GDL.	121
6.5	Erro (rad) e torque (Nm) do controlador CT-SMC para o UVMS de 12 GDL.	121
6.6	Erro (rad) e torque (Nm) do controlador Super-Twisting para o UVMS de 12 GDL.	122

Lista de Tabelas

2.1	Mapeamento entre fundamentos teóricos e componentes do <i>Hephaestus</i> .	27
3.1	Complexidade computacional das etapas de modelagem e simulação. . .	51
5.1	Papel das matrizes dinâmicas em cada estratégia de controle.	103
5.2	Custo computacional adicional por passo de integração Δt_{phys}	104
5.3	Parâmetros de configuração dos controladores na interface <code>ctrl_</code> <code>params</code>	104
5.4	Comparação teórica das estratégias de controle implementadas. . . .	105
6.1	Parâmetros mecânicos do braço antropomórfico do UVMS (elos 7–12).	115
6.2	Parâmetros dos controladores para o UVMS 12-GDL subaquático. . .	117
6.3	Desempenho dos controladores no UVMS 12-GDL , modelo nominal. Médias sobre as 12 juntas.	118
7.1	<i>Roadmap</i> de <i>epics</i> propostos para a plataforma <i>Hephaestus</i>	132
C.1	Dependências Python do <i>Hephaestus</i>	163
C.2	Estrutura principal de arquivos do repositório <i>Hephaestus</i>	165

Sumário

Lista de Figuras	viii
Lista de Tabelas	xi
1 Introdução	1
1.1 Contextualização e Motivação	1
1.2 Justificativa	2
1.3 Objetivos	4
1.3.1 Objetivo Geral	4
1.3.2 Objetivos Específicos	4
1.4 Metodologia	6
1.4.1 Modelagem Matemática (A Camada <i>Engine</i>)	6
1.4.2 Transição Simbólico-Numérica e Otimização	6
1.4.3 Simulação e Controle (A Camada <i>Simulator</i>)	7
1.4.4 Validação Comparativa	7
2 Revisão Bibliográfica e Fundamentos Teóricos	8
2.1 Robótica de Manipuladores: Cinemática e Dinâmica	8
2.1.1 Cinemática de Corpos Rígidos	8
2.1.2 Dinâmica de Manipuladores: Formulação de Euler-Lagrange	10
2.1.3 Propriedades Estruturais da Dinâmica	12
2.2 Sistemas Manipuladores-Veículos Subaquáticos (UVMS)	12
2.2.1 Contexto e Motivação	12
2.2.2 Hidrodinâmica: Fundamentos e Formulação	13
2.2.3 Modelo Dinâmico Completo do UVMS	15
2.3 Cinemática Inversa Numérica	16
2.3.1 Formulação do Problema	16
2.3.2 Cinemática Inversa em Malha Fechada (CLIK)	16
2.3.3 Tratamento de Singularidades: Mínimos Quadrados Amortecidos (DLS)	17
2.3.4 Inicialização por Levenberg-Marquardt com Busca em Linha	18

2.3.5	Erros de Orientação e SLERP	18
2.4	Estratégias de Controle Não-Linear	19
2.4.1	Controle por Torque Computado (CTC)	19
2.4.2	Controle por Modo Deslizante (SMC)	20
2.4.3	Algoritmo Super-Twisting (STA)	20
2.4.4	Controle por Rejeição Ativa de Distúrbios (ADRC/LADRC)	21
2.5	Planejamento de Trajetórias	23
2.5.1	Polinômio Cúbico e Perfil de <i>Jerk</i> Finito	23
2.5.2	Trajetória Circular no Espaço Cartesiano	23
2.6	Computação Simbólica e Geração de Código Numérico	24
2.6.1	Álgebra Computacional Simbólica com SymPy	24
2.6.2	Eliminação de Subexpressões Comuns (CSE)	24
2.6.3	Paralelismo na Derivação Simbólica	25
2.6.4	Integração Numérica: Método de Euler Simplético	25
2.7	Ferramentas de Simulação para UVMS: Estado da Arte	26
2.8	Síntese da Revisão	26
3	Modelagem Matemática e Geração Simbólica	28
3.1	Visão Geral da Arquitetura	28
3.2	Representação da Cadeia Cinemática	29
3.2.1	Configuração de Juntas e Máscara de Vetores de Elo	29
3.2.2	Símbolos Dinâmicos e Parâmetros	30
3.3	Cinemática Simbólica Direta	30
3.3.1	Transformações Homogêneas Acumuladas	30
3.3.2	Posições dos Centros de Massa no Referencial Global	32
3.3.3	Velocidades Angulares Simbólicas	32
3.4	Derivação Simbólica da Matriz de Inércia e do Vetor de Gravidade	33
3.4.1	Método do Jacobiano da Inércia	33
3.4.2	Vetor de Gradiente de Potencial Gravitacional	35
3.4.3	Jacobiano Geométrico Completo	36
3.5	Cálculo Simbólico do Vetor de Coriolis	38
3.5.1	Formulação pelos Símbolos de Christoffel	38
3.5.2	Estratégia de Paralelização da Derivação	39
3.5.3	Modo Serial para Ambientes Executáveis	40
3.6	Extensão Hidrodinâmica: RobotMathHydro	40
3.6.1	Parâmetros Adicionais: Volumes e Massa Adicionada	41
3.6.2	Matriz de Inércia com Massa Adicionada	41
3.6.3	Vetor de Potencial Aparente: Peso menos Empuxo	42
3.6.4	Modelo Dinâmico Unificado	43

3.7	Preparação e Exportação do Modelo Simbólico	44
3.7.1	Substituição de Símbolos Dinâmicos	44
3.7.2	Exportação do Dicionário de Resultados	45
3.8	Compilação Simbólico-Numérica com <code>lambdify</code> e CSE	46
3.8.1	A Interface <code>lambdify</code>	46
3.8.2	Eliminação de Subexpressões Comuns	46
3.8.3	Compilação das Funções por Módulo	47
3.9	Paralelismo em Dois Níveis e Executor Persistente	48
3.9.1	Nível 1: Paralelismo na Derivação Simbólica	48
3.9.2	Nível 2: Paralelismo na Avaliação Numérica	48
3.10	Considerações sobre a Complexidade Computacional	49
3.11	Síntese do Capítulo	51
4	Simulação Numérica e Planejamento de Trajetórias	52
4.1	Arquitetura do Simulador	52
4.2	Planejamento de Trajetórias no Espaço Cartesiano	53
4.2.1	Polinômio Cúbico de Interpolação	56
4.2.2	Segmento de Reta	56
4.2.3	Arco Circular	57
4.2.4	Modos de Referência de Orientação	60
4.3	Inicialização da Postura: Levenberg-Marquardt com Busca em Linha	62
4.4	Cinemática Inversa em Malha Fechada (CLIK)	63
4.4.1	Formulação do CLIK Posicional (3D)	63
4.4.2	Limitação de Velocidade de Junta	65
4.4.3	Controle de Orientação no Espaço de Tarefa 6D	66
4.4.4	Gestão de Redundância via Espaço Nulo	67
4.5	Integrador Simplético de Euler com Substeps	68
4.5.1	Hierarquia de Passos de Tempo	68
4.5.2	Integrador Simplético de Euler	70
4.6	Estratégias de Controle Não-Linear	71
4.6.1	Controle por Torque Computado com PID (CTC-PID)	72
4.6.2	Controle por Modo Deslizante (CT-SMC)	73
4.6.3	Super-Twisting (STA)	74
4.6.4	Controle por Rejeição Ativa de Distúrbios Linear (LADRC) .	75
4.7	Modelos de Forças Dissipativas	78
4.7.1	Atrito nas Juntas	78
4.7.2	Arrasto Hidrodinâmico	79
4.8	Paralelismo na Avaliação Numérica	80
4.9	Verificação de Estabilidade Numérica	81

4.10	Síntese do Capítulo	82
5	Controladores Implementados e Integração com o Modelo	83
5.1	Estrutura Unificada de Controle em Malha Fechada	83
5.2	Controle por Torque Computado com PID	84
5.2.1	Fundamentação Teórica e Cancelamento de Não-Linearidade	84
5.2.2	Integração com o Modelo Simbólico-Numérico	86
5.2.3	Análise de Robustez a Incertezas Paramétricas	86
5.3	Controle por Modo Deslizante com Torque Computado (CT-SMC)	87
5.3.1	Superfície Deslizante e Interpretação Geométrica	87
5.3.2	Lei de Controle CT-SMC e Condição de Alcance	88
5.3.3	Integração com o Modelo e Sensibilidade Paramétrica	89
5.4	Algoritmo Super-Twisting (STA)	90
5.4.1	Motivação: Chattering e Modos Deslizantes de Ordem Superior	90
5.4.2	Análise de Estabilidade via Função de Lyapunov Estrita	92
5.4.3	Continuidade do Sinal de Controle e Eliminação do Chattering	93
5.4.4	Implementação Discreta e Estabilidade Numérica	94
5.5	Controle por Rejeição Ativa de Distúrbios Linear (LADRC)	94
5.5.1	Paradigma de Rejeição Ativa e Motivação	94
5.5.2	Formulação do Modelo de Planta de Canal Integrador	96
5.5.3	Observador de Estado Estendido (ESO) Linear	97
5.5.4	Lei de Controle e Feedforward Explícito	99
5.5.5	Descentralização e Acoplamento Dinâmico Residual	101
5.6	Integração dos Controladores com o Modelo Simbólico	102
5.6.1	Cadeia de Dependências: do Simbólico ao Numérico	102
5.6.2	Uso Diferenciado do Modelo por Cada Estratégia	103
5.6.3	Avaliação Numérica e Custo Computacional por Controlador	103
5.7	Interface Gráfica e Configuração dos Controladores	104
5.7.1	Arquitetura de Configuração via <code>ctrl_params</code>	104
5.7.2	Ferramentas de Ajuste e Análise Comparativa	105
5.8	Análise Comparativa das Estratégias de Controle	105
5.8.1	Propriedades Teóricas Comparadas	105
5.8.2	Orientações para Seleção e Ajuste de Ganhos	106
5.8.3	Aplicabilidade por Cenário	108
5.9	Forças Dissipativas como Perturbações Estruturadas	108
5.10	Validação da Integração por Análise de Conservação de Energia	109
5.11	Síntese do Capítulo	110

6	Experimentos, Resultados e Discussão	112
6.1	Metodologia de Validação	112
6.1.1	CrITÉrios de AvaliaÇ�o de Desempenho	112
6.1.2	Plataforma e Par�metros de IntegraÇ�o	114
6.2	UVMS 12-GDL em Ambiente Subaqu�tico	114
6.2.1	ConfiguraÇ�o do UVMS	114
6.2.2	Traj�t�ria de Refer�ncia e Par�metros Hidrodin�micos	116
6.2.3	ConfiguraÇ�o dos Controladores	117
6.2.4	Resultados Comparativos	117
6.3	An�lise Comparativa das Estrat�gias de Controle	120
6.3.1	S�ntese Quantitativa	120
6.3.2	Discuss�o: Mecanismos de Rejei�o de Perturba��es	122
6.3.3	Sele��o do Controlador para UVMS	123
6.4	Valida��o do Desempenho Computacional	124
6.4.1	Tempo de Gera��o do Modelo Simb�lico	124
6.4.2	Efici�ncia da Compila��o com CSE	124
6.4.3	Executor Paralelo e Tempo de Simula��o	124
6.4.4	Integrador Simpl�tico	124
6.5	S�ntese do Cap�tulo	125
7	Conclus�es e Trabalhos Futuros	126
7.1	S�ntese das Contribui��es	126
7.2	Conclus�es sobre os Resultados Obtidos	128
7.2.1	Sobre a Modelagem Simb�lica e a Extens�o Hidrodin�mica	128
7.2.2	Sobre a Efici�ncia Computacional	128
7.2.3	Sobre o Integrador Simpl�tico e a Fidelidade Num�rica	129
7.2.4	Sobre as Estrat�gias de Controle	129
7.3	Limita��es do Trabalho	130
7.4	Trabalhos Futuros	131
7.4.1	Roadmap da Plataforma: Epics Propostos	131
7.4.2	Trabalhos Futuros Relativos ao UVMS	132
7.4.3	Extens�es da Modelagem Matem�tica	133
7.4.4	Extens�es das Estrat�gias de Controle	134
7.4.5	Integra��es de Software e Hardware	136
7.5	Considera��es Finais	137
	Refer�ncias Bibliogr�ficas	138
A	C�digo-fonte da <i>Engine</i> Matem�tica (<code>engine.py</code>)	144

B	Código-fonte dos Controladores e do Simulador (<code>simulator.py</code>)	153
C	Requisitos de Software, Instalação e Execução do <i>Hephaestus</i>	162

Capítulo 1

Introdução

1.1 Contextualização e Motivação

O crescente interesse na exploração subaquática tem realçado a relevância dos manipuladores integrados a veículos autônomos, formando os chamados Sistemas Manipuladores-Veículos Subaquáticos (UVMSs). Segundo Antonelli [19], esses sistemas permitem a execução autônoma de tarefas complexas em ambientes inóspitos, superando os altos custos, riscos e limitações associados à operação humana direta.

Além de seu valor estratégico em operações *offshore* e científicas, os UVMSs oferecem uma oportunidade única do ponto de vista da engenharia, o qual é a sua redundância cinemática que pode ser explorada para otimização energética e aumento de manipulabilidade.

No entanto, o estudo e o desenvolvimento de controladores para esses sistemas enfrentam barreiras substanciais. A primeira é a natureza física do ambiente: conforme Aldhaeri et al. [20], a imprevisibilidade do meio marinho e as forças hidrodinâmicas acopladas dificultam a plena autonomia.

A segunda barreira, frequentemente subestimada, é o desafio computacional e educacional da modelagem. A dinâmica de um UVMS é regida por equações diferenciais não-lineares de alta complexidade. Tradicionalmente, o ensino e a pesquisa nessa área dependem de softwares proprietários (como MATLAB/Simulink). Embora robustas, essas ferramentas atuam muitas vezes como “caixas-pretas”, ocultando a formulação matemática explícita do aluno e impondo um gargalo computacional severo: a transição entre a modelagem simbólica (necessária para derivar as equações exatas) e a simulação numérica em tempo real é frequentemente lenta ou exige ferramentas de compilação complexas.

Dada a elevada complexidade necessária para modelar e simular um manipulador subaquático com múltiplos graus-de-liberdade, torna-se imperioso buscar abordagens acessíveis e computacionalmente eficientes.

1.2 Justificativa

A motivação para o desenvolvimento deste trabalho surge da necessidade de democratizar o acesso a ferramentas de simulação de alta fidelidade e superar as limitações de integração observadas em ambientes clássicos de engenharia.

Historicamente, a dinâmica de sistemas multicorpos é ensinada de forma desconexa da implementação. Quando se busca precisão via toolboxes comerciais, esbarra-se no custo de licenças e na rigidez da arquitetura de software, o que dificulta a experimentação de novos algoritmos por estudantes e pesquisadores independentes. Esse gargalo de software é particularmente crítico na robótica marinha. Como afirmam Yuh e West [21], a adição de manipuladores transforma o veículo em um sistema multicorpos de modelagem altamente complexa, tornando o uso de simulações precisas e integradas uma etapa obrigatória para lidar com os altos custos e riscos dos testes reais..

Neste contexto, a migração para o ecossistema Python apresenta-se como a solução técnica ideal, permitindo a integração de bibliotecas de computação simbólica (SymPy) com o ecossistema científico e numérico para um fluxo de trabalho contínuo [2]. Para contornar os gargalos de desempenho inerentes à manipulação simbólica pura [2], a abordagem proposta neste trabalho utiliza as capacidades de geração de código do SymPy aliadas à Eliminação de Subexpressões Comuns (*Common Subexpression Elimination*, CSE) [2]. Assim, através da compilação *Just-in-Time* via `lambdify`, a modelagem lagrangiana do sistema é convertida diretamente em código de máquina vetorizado (NumPy) e otimizado para simulações de alta performance.

A concretização da “plataforma didática” passa ainda pela existência de uma interface gráfica (*Graphical User Interface*, GUI) desenvolvida com a biblioteca CustomTkinter, que torna o ambiente acessível sem a necessidade de programação direta. Essa interface estrutura-se em três módulos funcionais: **Modelagem** (configuração da cadeia cinemática e geração simbólica do modelo dinâmico), **Simulação** (definição de trajetórias, controladores e parâmetros físicos) e **Análise** (comparação entre múltiplas sessões de controle).

Assim, este trabalho justifica-se pela criação do ambiente *Hephaestus*: uma ferramenta que não apenas modela os efeitos hidrodinâmicos (massa adicionada, arrasto, empuxo) com rigor matemático, mas também serve como plataforma didática e *open-source* para a validação de estratégias de controle avançado, unindo a teoria de Dinâmica e Controle à moderna Engenharia de Software. Sintetiza-se, na Figura 1.1, a arquitetura geral do *Hephaestus*, com destaque para o fluxo de trabalho que vai da especificação simbólica da cadeia cinemática até a comparação das estratégias de controle.

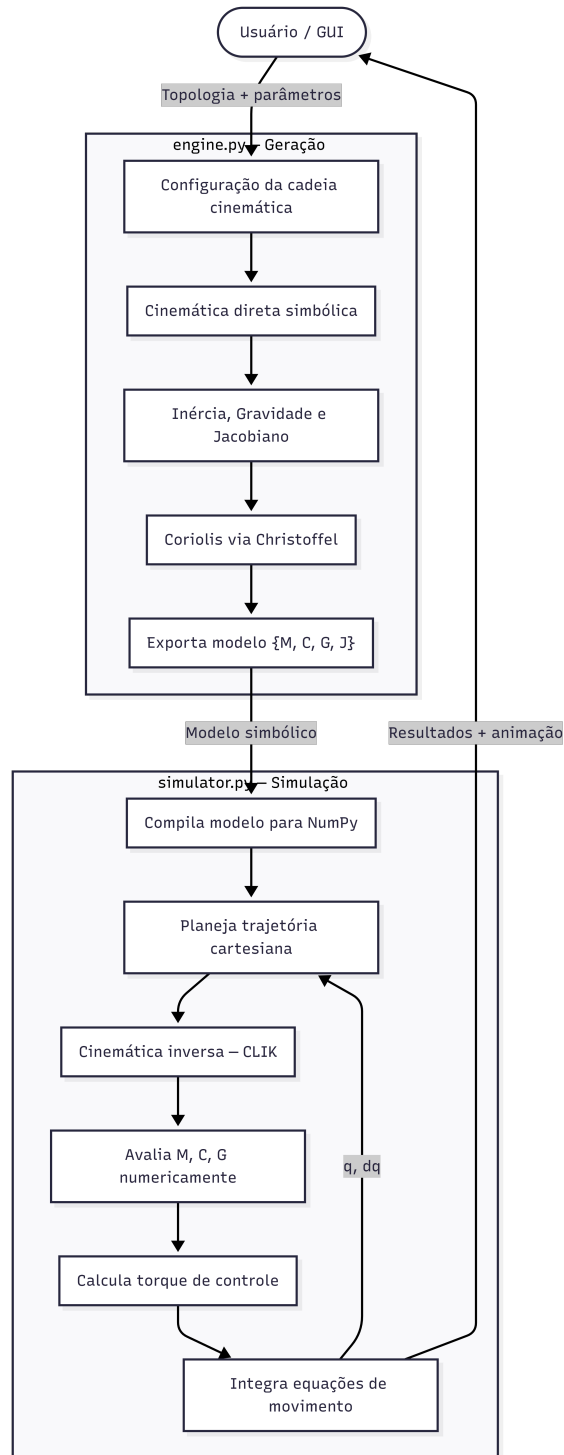


Figura 1.1: Arquitetura geral do ambiente *Hephaestus*: fluxo de trabalho desde a especificação da cadeia cinemática (módulo Modelagem) até a simulação em malha fechada (módulo Simulação) e a comparação de resultados (módulo Análise). As dependências entre os módulos `engine.py` e `simulator.py` e os quatro controladores não-lineares são destacadas.

1.3 Objetivos

1.3.1 Objetivo Geral

Desenvolver um ambiente computacional integrado em Python para a modelagem dinâmica simbólica e simulação de manipuladores robóticos e sistemas UVMS. O projeto visa implementar uma arquitetura que automatize a derivação das equações de movimento via formulação Lagrangiana e execute simulações de controle em malha fechada com alto desempenho, considerando explicitamente os efeitos hidrodinâmicos, o controle no *espaço de tarefa 6D* (posição + orientação) e a comparação entre múltiplas estratégias de controle não-linear.

O que é o espaço de tarefa 6D. O termo “6D” refere-se à descrição completa da *pose* do efetuador no espaço cartesiano, dada por seis coordenadas independentes: três para a **posição** e três para a **orientação**. As três coordenadas de posição (x, y, z) localizam um ponto de referência do efetuador no espaço, enquanto as três coordenadas de orientação, tipicamente representadas por ângulos de *roll*, *pitch* e *yaw* ou, equivalentemente, pelo eixo-ângulo de uma rotação em $SO(3)$, descrevem a sua atitude angular. Tarefas de robótica que envolvem alinhar uma ferramenta a uma superfície, soldar ao longo de uma trajetória ou inserir uma peça em um furo exigem o controle simultâneo dessas seis coordenadas, e não apenas das três posicionais. Por essa razão, o *Hephaestus* estende o algoritmo de cinemática inversa em malha fechada (CLIK) para operar com o Jacobiano completo $J \in \mathbb{R}^{6 \times N}$, Capítulo 2, que mapeia as N velocidades de junta para as seis componentes da *twist* cartesiana (três lineares + três angulares).

1.3.2 Objetivos Específicos

A fim de atingir o objetivo geral, foram definidos os seguintes objetivos específicos:

- **Revisão Bibliográfica e Metodológica:** Analisar as abordagens clássicas de modelagem (Craig, Spong) e as especificidades da hidrodinâmica de UVMS (Fossen, Antonelli), identificando as limitações das ferramentas de simulação atuais.
- **Desenvolvimento da *Engine Matemática*:** Implementar algoritmos em Python capazes de gerar simbolicamente as matrizes de Inércia (M), o vetor de Coriolis e Centrípeto (C) e o vetor de Gravidade/Empuxo (G) para manipuladores genéricos de N graus de liberdade, tanto em ambiente terrestre quanto subaquático. No cenário subaquático, a massa adicionada é modelada

como um tensor diagonal 6×6 por elo (três componentes translacionais e três rotacionais), rotacionado para o referencial global, em conformidade com a formulação de Fossen [1].

- **Otimização Computacional:** Desenvolver um método de compilação das equações simbólicas para funções numéricas otimizadas por meio do `lambdify` com Eliminação de Subexpressões Comuns (CSE), superando os gargalos de desempenho típicos de simulações simbólicas puras. Implementar dois níveis de paralelismo: (i) na *fase de derivação* do modelo, paralelizando o cálculo de $\partial M / \partial q_k$ e das linhas do vetor C entre múltiplos processos; e (ii) na *fase de simulação*, por meio de um executor de processos persistente (`ProcessPoolExecutor`) que avalia M , C e G numericamente em paralelo a cada passo de integração, eliminando o custo de *respawn* entre execuções.
- **Implementação do Simulador Numérico:** Construir um ambiente de simulação capaz de resolver a dinâmica direta e inversa, incluindo: planejamento de trajetórias (segmentos de reta e arcos circulares com polinômio cúbico de interpolação); modelos de atrito nas juntas (Coulomb + viscoso, suavizados por tanh para evitar descontinuidades); e modelos de arrasto hidrodinâmico no espaço de juntas (linear e quadrático). Incluir visualização gráfica animada dos resultados.
- **Controle de Orientação no Espaço de Tarefa 6D:** Estender o algoritmo de Cinemática Inversa em Malha Fechada (CLIK) para o espaço de tarefa completo de seis dimensões, em que as três primeiras correspondem à posição cartesiana (x, y, z) do efetuador e as três últimas à sua orientação angular (*roll*, *pitch*, *yaw*), utilizando o Jacobiano completo $J \in \mathbb{R}^{6 \times N}$. Implementar múltiplos modos de referência de orientação: *Livre*, *Fixa*, *Tangente à Trajetória*, *Apontar para o Alvo*, interpolação SLERP (*Spherical Linear Interpolation*) [6] e *Normal à Superfície*.
- **Projeto e Validação de Controle:** Implementar e comparar quatro estratégias de controle não-linear em malha fechada:
 1. **Controle por Torque Computado com PID (CTC-PID):** linearização por realimentação com termo integral [5], acrescido de uma compensação *anti-windup* para evitar a instabilidade frente à saturação dos atuadores [22]
 2. **Controle por Modo Deslizante (CT-SMC):** superfície deslizante [11] com camada limite suavizante (tanh) [12];

3. **Super-Twisting SMC** (STA): algoritmo de modo deslizante de 2ª ordem com sinal de controle contínuo e convergência em tempo finito [13, 14];
4. **Controle por Rejeição Ativa de Distúrbios** (LADRC): observador de estado estendido (ESO) descentralizado com estimação e cancelamento em malha fechada dos distúrbios residuais [17].

Todos os controladores são alimentados pelo algoritmo de Cinemática Inversa em Malha Fechada (CLIK). O tratamento de singularidades é realizado pelo método de Mínimos Quadrados Amortecidos (DLS) , formulado originalmente como Inversa Robusta a Singularidades (SR-Inverse) [8] ,, e a gestão de redundância é estruturada via projeção no Espaço Nulo [9].

1.4 Metodologia

A metodologia adotada estrutura-se em uma abordagem incremental de desenvolvimento de software e validação matemática:

1.4.1 Modelagem Matemática (A Camada *Engine*)

Utilizando a biblioteca SymPy , em particular o módulo `dynamicsymbols` da mecânica analítica, que fornece símbolos dependentes do tempo com diferenciação automática , será implementada a formulação de Euler-Lagrange. Diferente de abordagens numéricas de caixa-preta, o software deduzirá as equações literais. Para o cenário subaquático, serão incorporados os termos de energia potencial aparente (Peso - Empuxo) e energia cinética modificada (Massa Adicionada como tensor 6D por elo), conforme Fossen [1]. O vetor resultante $C(q, \dot{q})$ unifica os efeitos de Coriolis e centrípeto pela formulação dos símbolos de Christoffel de primeira espécie [23].

1.4.2 Transição Simbólico-Numérica e Otimização

Esta etapa foca na eficiência computacional e na superação do gargalo de desempenho típico de álgebras simbólicas. As equações dinâmicas geradas serão compiladas via `lambdify` com CSE (*Common Subexpression Elimination*) ativo, que identifica e fatoriza subexpressões repetidas na expressão simbólica antes da geração do código NumPy, reduzindo o tamanho do *bytecode* compilado e acelerando a avaliação numérica [2].

Para lidar com a complexidade computacional do termo de Coriolis em sistemas de múltiplos graus de liberdade, será implementado processamento paralelo em dois níveis distintos. No primeiro nível, a derivação simbólica de $\partial M / \partial q_k$

e o cálculo das linhas do vetor C são distribuídos entre múltiplos processos via `ProcessPoolExecutor`, visando otimizar o tempo de geração do modelo conforme as métricas de eficiência discutidas por Hollerbach [24]. No segundo nível, durante a simulação, um executor de processos persistente, pré-inicializado após a compilação e reutilizado em todos os ciclos subsequentes, avalia $M(q)$, $C(q, \dot{q})$ e $G(q)$ em paralelo a cada passo de integração, eliminando o custo de instanciação repetida de *workers*.

1.4.3 Simulação e Controle (A Camada *Simulator*)

Será desenvolvido um integrador numérico Simplético de Euler operando em malha fechada. A escolha pelo método Simplético (atualização de q antes de $\dot{q}$) em detrimento do Euler Explícito padrão é motivada pela melhor preservação da energia do sistema ao longo da trajetória. O passo de integração física pode ser subdividido (*substeps*) de forma independente do passo de visualização, permitindo alta fidelidade numérica sem custo proporcional de renderização.

O controle dinâmico será realizado por uma das quatro estratégias implementadas (CTC-PID, CT-SMC, STA ou LADRC), todas alimentadas por um algoritmo de Cinemática Inversa em Malha Fechada (CLIK). A fase de inicialização da postura utiliza um algoritmo de Levenberg-Marquardt com busca em linha adaptativa, que ajusta dinamicamente o parâmetro de amortecimento λ para garantir convergência robusta antes do início da trajetória. Durante a simulação, o CLIK opera no espaço de tarefa 6D (posição + orientação), incorporando: referência de velocidade e aceleração por *feedforward*, correção de deriva do Jacobiano ($\dot{J}\dot{q}$), tratamento de singularidades via DLS [8] e gestão de redundância via projeção no Espaço Nulo [9].

Modelos de atrito nas juntas (Coulomb + viscoso, suavizados por tanh) e de arrasto hidrodinâmico no espaço de juntas (linear e quadrático) são aplicados na equação de integração como forças passivas, separadamente do sinal de controle.

1.4.4 Validação Comparativa

O software desenvolvido (*Hephaestus*) será submetido a testes de trajetória em dois cenários distintos: um manipulador serial em ambiente terrestre e um UVMS em ambiente submerso. A validação do cenário submerso seguirá os critérios de modelagem hidrodinâmica estabelecidos por Fossen e Antonelli [1, 19], analisando comparativamente os torques requeridos, o erro de rastreamento de posição e orientação, e o comportamento frente a perturbações externas para confirmar a consistência física dos termos de massa adicionada e empuxo implementados. A comparação entre as quatro estratégias de controle será realizada na aba de **Análise** da interface gráfica, que permite sobrepor múltiplas sessões de simulação nos mesmos gráficos.

Capítulo 2

Revisão Bibliográfica e Fundamentos Teóricos

Este capítulo apresenta a fundamentação teórica e a revisão da literatura sobre os tópicos centrais que sustentam o desenvolvimento do ambiente *Hephaestus*. As áreas abordadas compreendem a modelagem cinemática e dinâmica de manipuladores robóticos, os fundamentos específicos dos Sistemas Manipuladores-Veículos Subaquáticos (UVMS), os algoritmos de cinemática inversa numérica, as estratégias de controle não-linear implementadas e os paradigmas de computação simbólica e numérica explorados. A revisão prioriza referências seminais e trabalhos recentes que fundamentam diretamente as escolhas de projeto e implementação.

2.1 Robótica de Manipuladores: Cinemática e Dinâmica

2.1.1 Cinemática de Corpos Rígidos

O estudo sistemático da posição e orientação de corpos rígidos no espaço constitui a base da robótica de manipuladores. A representação por matrizes de transformação homogênea 4×4 , que codificam simultaneamente rotação e translação, foi popularizada por Denavit e Hartenberg [25], cuja convenção parametriza o posicionamento de elos sequenciais por apenas quatro parâmetros por junta: a_i , d_i , α_i e θ_i . Embora a convenção DH permaneça amplamente utilizada no ensino, a representação por matrizes de rotação diretas (eixo-ângulo) adotada neste trabalho oferece maior transparência algébrica e facilita a integração com ferramentas de computação simbólica [5, 23].

A cinemática direta (FK) mapeia as variáveis de junta $q \in \mathbb{R}^N$ para a posição e orientação do efetuador final no espaço cartesiano. Para uma cadeia serial aberta

de N graus de liberdade, a transformação homogênea acumulada T_N é obtida pelo produto:

$$T_N(q) = T_1(q_1) \cdot T_2(q_2) \cdots T_N(q_N), \quad (2.1)$$

onde:

- $T_N(q) \in \mathbb{R}^{4 \times 4}$ é a transformação homogênea acumulada do referencial da base até o efetuador final;
- $T_i(q_i) \in \mathbb{R}^{4 \times 4}$ é a transformação homogênea associada à junta i e ao elo i ;
- q_i é a variável generalizada (ângulo ou deslocamento) da junta i ;
- N é o número de graus de liberdade da cadeia [23].

A cinemática diferencial relaciona as velocidades no espaço de juntas às velocidades no espaço cartesiano por meio do Jacobiano geométrico $J(q) \in \mathbb{R}^{6 \times N}$:

$$\dot{x} = J(q)\dot{q}, \quad (2.2)$$

onde:

- $\dot{x} = [\dot{p}^\top, \omega^\top]^\top \in \mathbb{R}^6$ é o vetor de *twist* do efetuador, reunindo a velocidade linear $\dot{p} \in \mathbb{R}^3$ e a velocidade angular $\omega \in \mathbb{R}^3$;
- $J(q) \in \mathbb{R}^{6 \times N}$ é o Jacobiano geométrico, decomposto em linhas translacionais J_v e linhas rotacionais J_ω ;
- $\dot{q} \in \mathbb{R}^N$ é o vetor de velocidades das juntas;
- N é o número de graus de liberdade da cadeia.

A decomposição de J em J_v e J_ω é fundamental para o projeto dos controladores em espaço de tarefa [4]: as três primeiras linhas, $J_v \in \mathbb{R}^{3 \times N}$, mapeiam $\dot{q}$ na velocidade linear $\dot{p}$, enquanto as três últimas, $J_\omega \in \mathbb{R}^{3 \times N}$, mapeiam $\dot{q}$ na velocidade angular ω . Essa estrutura é o que define o chamado *espaço de tarefa 6D*, em que três coordenadas descrevem a **posição** cartesiana e três descrevem a **orientação** angular do efetuador, totalizando seis graus de liberdade do efetuador no espaço.

As singularidades cinemáticas, configurações em que J perde posto, representam pontos de degeneração do mapeamento diferencial, causando velocidades de junta ilimitadas para movimentos cartesianos finitos. Sua análise e tratamento, seja por critério do determinante de $JJ^\top$, por decomposição em valores singulares (SVD) [26] ou por regularização do tipo Mínimos Quadrados Amortecidos (DLS), é abordada em detalhe por Nakamura e Hanafusa [8] e Wampler [27].

2.1.2 Dinâmica de Manipuladores: Formulação de Euler-Lagrange

A modelagem dinâmica de manipuladores pode ser abordada por dois formalismos clássicos: Newton-Euler e Euler-Lagrange. A formulação de Newton-Euler é recursiva e numericamente eficiente, sua versão recursiva, proposta por Luh, Walker e Paul, tem complexidade $O(N)$ no número de elos [28]. Já a formulação de Euler-Lagrange, adotada neste trabalho, deriva equações explícitas e simbólicas, tornando-a especialmente adequada para plataformas didáticas e para a síntese de controladores baseados em modelo.

O Lagrangiano $\mathcal{L} = T - V$, diferença entre a energia cinética T e a potencial V do sistema, conduz às equações de movimento de Euler-Lagrange:

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i} \right) - \frac{\partial \mathcal{L}}{\partial q_i} = \tau_i, \quad i = 1, \dots, N. \quad (2.3)$$

onde:

- $\mathcal{L}(q, \dot{q}) = T - V$ é o Lagrangiano do sistema;
- $T(q, \dot{q})$ é a energia cinética total;
- $V(q)$ é a energia potencial total;
- q_i e $\dot{q}_i$ são a coordenada e a velocidade generalizadas da junta i ;
- τ_i é o torque generalizado aplicado na junta i ;
- N é o número de graus de liberdade.

O desenvolvimento da Equação (2.3) para uma cadeia serial resulta na forma estruturada da equação de movimento:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau, \quad (2.4)$$

onde:

- $M(q) \in \mathbb{R}^{N \times N}$ é a matriz de inércia generalizada, simétrica e definida positiva;
- $C(q, \dot{q}) \in \mathbb{R}^{N \times N}$ é a matriz de Coriolis e termos centrípetos;
- $G(q) \in \mathbb{R}^N$ é o vetor de gradiente de energia potencial gravitacional;
- $q, \dot{q}, \ddot{q} \in \mathbb{R}^N$ são, respectivamente, os vetores de posições, velocidades e acelerações generalizadas;

- $\tau \in \mathbb{R}^N$ é o vetor de torques generalizados aplicados pelos atuadores [4, 5, 23].

Os elementos C_{ij} da matriz de Coriolis são calculados pelos símbolos de Christoffel de segunda espécie [23]:

$$C_{ij}(q, \dot{q}) = \sum_{k=1}^N c_{ijk}(q) \dot{q}_k, \quad c_{ijk} = \frac{1}{2} \left(\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{jk}}{\partial q_i} \right). \quad (2.5)$$

onde:

- $C_{ij}(q, \dot{q})$ é o elemento (i, j) da matriz de Coriolis;
- $c_{ijk}(q)$ são os símbolos de Christoffel de segunda espécie;
- M_{ij}, M_{ik}, M_{jk} são elementos da matriz de inércia $M(q)$;
- $\partial M_{ab} / \partial q_c$ denota a derivada parcial do elemento M_{ab} em relação à coordenada q_c ;
- $\dot{q}_k$ é a velocidade generalizada da junta k ;
- N é o número de graus de liberdade.

A matriz de inércia pode ser expressa em termos dos Jacobianos de velocidade linear J_{v_i} e angular J_{ω_i} do centro de massa do elo i :

$$M(q) = \sum_{i=1}^N \left[m_i J_{v_i}^\top J_{v_i} + J_{\omega_i}^\top I_i^{(g)} J_{\omega_i} \right], \quad (2.6)$$

onde:

- $M(q) \in \mathbb{R}^{N \times N}$ é a matriz de inércia generalizada;
- m_i é a massa do elo i ;
- $J_{v_i} \in \mathbb{R}^{3 \times N}$ e $J_{\omega_i} \in \mathbb{R}^{3 \times N}$ são os Jacobianos linear e angular do CM do elo i ;
- $I_i^{(g)} = R_i I_i^{(l)} R_i^\top \in \mathbb{R}^{3 \times 3}$ é o tensor de inércia do elo i expresso no referencial global, obtido pela transformação do tensor local $I_i^{(l)}$ pela matriz de rotação acumulada R_i [4];
- N é o número de elos.

Esta formulação, denominada *método do Jacobiano da inércia*, é a base da implementação na classe **RobotMathEngine**.

A eficiência computacional do cálculo de $C(q, \dot{q})$ é crítica em sistemas com muitos graus de liberdade. Hollerbach [24] demonstrou que a formulação recursiva

Newton-Euler reduz a complexidade de $O(N^4)$ (Lagrange ingênuo) para $O(N)$. Em abordagens simbólicas como a deste trabalho, a paralelização da computação de $\partial M/\partial q_k$ mitiga esse custo.

2.1.3 Propriedades Estruturais da Dinâmica

A estrutura da Equação (2.4) exhibe propriedades que são centrais para a análise de estabilidade e o projeto de controladores [11, 23]:

1. **Simetria e positividade de $M(q)$:** $M(q) = M(q)^\top > 0$ para todo q .
2. **Antissimetria de $\dot{M} - 2C$:** A matriz $N(q, \dot{q}) = \dot{M}(q) - 2C(q, \dot{q})$ é antissimétrica, i.e., $x^\top N x = 0$ para todo x . Esta propriedade é a base da prova de estabilidade de Lyapunov para o controle por torque computado.
3. **Linearidade nos parâmetros:** A equação de movimento pode ser reescrita como $M\ddot{q} + C\dot{q} + G = Y(q, \dot{q}, \ddot{q})\theta$, onde Y é a *matriz de regressores* e θ é o vetor de parâmetros inerciais. Isso sustenta os controladores adaptativos [29].

2.2 Sistemas Manipuladores-Veículos Subaquáticos (UVMS)

2.2.1 Contexto e Motivação

Os robôs subaquáticos representam uma das fronteiras mais ativas da engenharia mecatrônica e oceânica. Desde os primeiros estudos sistemáticos de Yuh [30], a modelagem de Veículos Subaquáticos Não Tripulados (UUVs) evoluiu para incorporar a análise de sistemas acoplados veículo-manipulador, os UVMSs. A revisão abrangente de Yuh e West [21] cataloga as aplicações práticas, inspeção de dutos, manutenção de plataformas *offshore*, arqueologia submarina e pesquisa oceanográfica, que motivam a crescente demanda por autonomia e precisão.

Trabalhos mais recentes, como a revisão de Sivčev et al. [31], confirmam que a principal barreira para a automação plena de UVMSs reside na complexidade da interação dinâmica veículo-braço e nas incertezas do meio aquático. Aldhaheri et al. [20] fornecem uma revisão atualizada dos avanços em controle e navegação, reforçando a necessidade de simuladores de alta fidelidade para o desenvolvimento e validação de estratégias de controle antes do ensaio físico.

2.2.2 Hidrodinâmica: Fundamentos e Formulação

A modelagem hidrodinâmica de corpos submersos tem como referência fundamental o livro de Thor I. Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control* [1]. Fossen sistematiza a modelagem baseada na formulação SNAME, onde as forças e momentos hidrodinâmicos atuantes sobre um corpo rígido submerso são decompostos em três categorias principais:

Massa Adicionada (*Added Mass*). Ao acelerar um corpo submerso, é necessário acelerar também o fluido ao seu redor. Esse efeito é modelado como uma inércia adicional, a massa adicionada, capturada pelo tensor $M_A \in \mathbb{R}^{6 \times 6}$ [1, 32]:

$$M_A = -\frac{\partial}{\partial \dot{\nu}} [\text{forças e momentos hidrodinâmicos}], \quad (2.7)$$

onde:

- $M_A \in \mathbb{R}^{6 \times 6}$ é o tensor de massa adicionada do corpo submerso;
- $\nu = [u, v, w, p, q, r]^T \in \mathbb{R}^6$ é o vetor de velocidades no referencial do corpo (*body-fixed*), com u, v, w velocidades lineares (*surge, sway, heave*) e p, q, r velocidades angulares (*roll, pitch, yaw*);
- $\dot{\nu}$ é a derivada temporal de ν (acelerações lineares e angulares);
- o sinal negativo segue a convenção de Fossen, em que M_A representa a inércia virtual sentida pelo corpo devido ao fluido acelerado.

Para corpos com planos de simetria, M_A torna-se diagonal ou quase-diagonal [1]. No *Hephaestus*, M_A é modelado como um tensor diagonal 6×6 por elo, com três componentes translacionais ($m_{a,u}, m_{a,v}, m_{a,w}$) e três rotacionais ($m_{a,p}, m_{a,q}, m_{a,r}$), rotacionado para o referencial global conforme a formulação de Fossen [1]:

$$M_A^{(g)} = R_i M_{A,\text{lin}}^{(l)} R_i^T, \quad I_A^{(g)} = R_i M_{A,\text{rot}}^{(l)} R_i^T. \quad (2.8)$$

onde:

- $M_A^{(g)} \in \mathbb{R}^{3 \times 3}$ é a matriz de massa adicionada translacional do elo i no referencial global;
- $I_A^{(g)} \in \mathbb{R}^{3 \times 3}$ é a matriz de massa adicionada rotacional do elo i no referencial global;
- $M_{A,\text{lin}}^{(l)}, M_{A,\text{rot}}^{(l)} \in \mathbb{R}^{3 \times 3}$ são as matrizes diagonais de massa adicionada translacional e rotacional no referencial local;

- $R_i \in \mathbb{R}^{3 \times 3}$ é a matriz de rotação acumulada até o elo i .

A contribuição da massa adicionada à matriz de inércia generalizada do UVMS é, portanto:

$$M_{\text{Hydro}}(q) = \sum_{i=1}^N \left[J_{v_i}^\top (m_i I + M_{A,i}^{(g)}) J_{v_i} + J_{\omega_i}^\top (I_i^{(g)} + I_{A,i}^{(g)}) J_{\omega_i} \right]. \quad (2.9)$$

onde:

- $M_{\text{Hydro}}(q) \in \mathbb{R}^{N \times N}$ é a matriz de inércia generalizada do UVMS;
- $m_i I$ é a inércia rígida translacional do elo i , com $I = I_3$ (identidade 3×3);
- $M_{A,i}^{(g)} \in \mathbb{R}^{3 \times 3}$ é a massa adicionada translacional do elo i no referencial global;
- $I_i^{(g)} \in \mathbb{R}^{3 \times 3}$ é o tensor de inércia rígida do elo i no referencial global;
- $I_{A,i}^{(g)} \in \mathbb{R}^{3 \times 3}$ é a massa adicionada rotacional do elo i no referencial global;
- $J_{v_i}, J_{\omega_i} \in \mathbb{R}^{3 \times N}$ são os Jacobianos linear e angular do elo i .

Esta formulação, diretamente implementada no método `step_2_jacobian_M_G` da classe `RobotMathHydro`, garante que a contribuição de inércia aparente seja corretamente projetada no espaço de juntas via Jacobianos [19].

Arrasto Hidrodinâmico (*Drag*). O arrasto sobre um corpo submerso em movimento possui componentes viscosa (linear em ν) e de pressão (quadrática em ν). A formulação simplificada, mas amplamente adotada em simuladores de UVMS [1, 19], escreve o vetor de forças de arrasto no espaço de juntas como:

$$\tau_{\text{drag}} = D_{\text{lin}} \dot{q} + D_{\text{quad}} |\dot{q}| \dot{q}, \quad (2.10)$$

onde:

- $\tau_{\text{drag}} \in \mathbb{R}^N$ é o vetor de torques de arrasto no espaço de juntas;
- $D_{\text{lin}} \in \mathbb{R}^N$ é o vetor de coeficientes de arrasto linear (regime laminar);
- $D_{\text{quad}} \in \mathbb{R}^N$ é o vetor de coeficientes de arrasto quadrático (regime turbulento);
- $\dot{q} \in \mathbb{R}^N$ é o vetor de velocidades das juntas;
- $|\dot{q}|$ denota o vetor componente a componente do valor absoluto de $\dot{q}$;
- os produtos $D_{\text{lin}} \dot{q}$ e $D_{\text{quad}} |\dot{q}| \dot{q}$ são realizados elemento por elemento.

Essa formulação é modelada como força passiva no integrador Simplético, conforme discutido na Seção 1.4.3.

Força de Empuxo e Gradiente Estático (*Buoyancy*). O princípio de Arquimedes estabelece que um corpo submerso experimenta uma força de empuxo $B = \rho g V_{\text{sub}}$, onde ρ é a densidade do fluido, g é a aceleração gravitacional e V_{sub} é o volume deslocado [32]. Para um elo submerso, o potencial aparente é:

$$V_i^{\text{app}} = (m_i - \rho \text{vol}_i) g z_{\text{cm},i}, \quad (2.11)$$

onde:

- V_i^{app} é a energia potencial aparente (peso menos empuxo) do elo i ;
- m_i é a massa do elo i ;
- ρvol_i é a massa de fluido deslocada pelo elo (princípio de Arquimedes);
- ρ é a densidade do fluido, vol_i é o volume submerso do elo;
- g é a aceleração da gravidade;
- $z_{\text{cm},i}$ é a coordenada vertical do centro de massa do elo i .

O vetor $G(q)$ resultante combina, portanto, o gradiente de Peso menos Empuxo, podendo ser nulo ou muito pequeno quando o elo é neutralmente flutuante ($m_i \approx \rho \text{vol}_i$), o que reduz os torques estáticos necessários e é o cenário típico de UVMSs projetados para neutralidade [1, 19].

2.2.3 Modelo Dinâmico Completo do UVMS

Antonelli [19] apresenta o modelo dinâmico unificado de um UVMS como:

$$M_{\text{Hydro}}(q)\ddot{q} + C_{\text{Hydro}}(q, \dot{q})\dot{q} + D(q, \dot{q})\dot{q} + G_{\text{Hydro}}(q) = \tau + J^{\top} f_{\text{ext}}, \quad (2.12)$$

onde:

- $M_{\text{Hydro}}(q) \in \mathbb{R}^{N \times N}$ é a matriz de inércia generalizada com massa adicionada;
- $C_{\text{Hydro}}(q, \dot{q}) \in \mathbb{R}^{N \times N}$ é a matriz de Coriolis e termos centrípetos hidrodinâmica;
- $D(q, \dot{q}) \in \mathbb{R}^{N \times N}$ reúne os efeitos de arrasto linear e quadrático;
- $G_{\text{Hydro}}(q) \in \mathbb{R}^N$ é o vetor de torques aparentes (peso menos empuxo);
- $q, \dot{q}, \ddot{q} \in \mathbb{R}^N$ são posições, velocidades e acelerações generalizadas;
- $\tau \in \mathbb{R}^N$ é o vetor de torques aplicados pelos atuadores;

- $f_{\text{ext}} \in \mathbb{R}^6$ são as forças e momentos externos aplicados ao efetuador (e.g., interação com o ambiente);
- $J^\top \in \mathbb{R}^{N \times 6}$ é a transposta do Jacobiano geométrico, que projeta as forças cartesianas no espaço de juntas.

Esta estrutura é idêntica à da Equação (2.4), com os termos M , C e G modificados pela contribuição hidrodinâmica, o que permite que a mesma *engine* seja reutilizada em ambos os modos (ar e água) com a simples extensão implementada em RobotMathHydro.

2.3 Cinemática Inversa Numérica

2.3.1 Formulação do Problema

A cinemática inversa (IK), determinação das variáveis de junta q que posicionam o efetuador em uma configuração cartesiana desejada x_d , é um problema central em robótica. Para cadeias seriais não-redundantes ($N = 6$), soluções analíticas fechadas são, em geral, possíveis [5]. Para sistemas redundantes ($N > 6$) ou com geometrias genéricas, as abordagens numéricas iterativas são necessárias [4].

2.3.2 Cinemática Inversa em Malha Fechada (CLIK)

O algoritmo de Cinemática Inversa em Malha Fechada (*Closed-Loop Inverse Kinematics*, CLIK), proposto por Samson et al. [33] e refinado por Chiaverini et al. [7], integra numericamente a equação diferencial de erro cartesiano para produzir uma referência de velocidade de junta que converge para a configuração desejada. A lei de atualização, na forma discreta com passo dt , é:

$$\dot{q}_{k+1} = J^\dagger(q_k) [\dot{x}_d + K_P e_k] + (I - J^\dagger J) \dot{q}_0, \quad (2.13)$$

onde:

- $\dot{q}_{k+1} \in \mathbb{R}^N$ é o vetor de velocidades de junta no próximo instante discreto;
- $J^\dagger(q_k) \in \mathbb{R}^{N \times m}$ é a pseudo-inversa (ou pseudo-inversa amortecida) do Jacobiano avaliado em q_k , com $m \in \{3, 6\}$ a dimensão da tarefa;
- $\dot{x}_d \in \mathbb{R}^m$ é a velocidade cartesiana de referência (*feedforward*);
- $e_k = x_d - x_k$ é o erro cartesiano (posição/orientação) no instante k ;
- K_P é o ganho proporcional de realimentação;

- $(I - J^\dagger J) \in \mathbb{R}^{N \times N}$ é o operador de projeção no espaço nulo de J ;
- $\dot{q}_0 \in \mathbb{R}^N$ é a velocidade auxiliar usada para objetivos secundários (e.g., afastar-se de limites de junta) sem afetar a tarefa primária [9].

O CLIK implementado no *Hephaestus* opera no espaço de tarefa completo de seis dimensões (3 da posição e 3 da orientação angular do efetuador, totalizando *seis graus de liberdade da tarefa*), utilizando o Jacobiano completo $J \in \mathbb{R}^{6 \times N}$, que relaciona, conforme a Equação (2.2), as N velocidades de junta com as três velocidades lineares e três velocidades angulares do efetuador, e incorpora:

- **Feedforward de velocidade e aceleração:** contribuição de $\dot{x}_d$ e $\ddot{x}_d$ para melhorar o rastreamento;
- **Correção de deriva do Jacobiano:** subtração do termo $\dot{J}\dot{q}$ para compensar a aceleração já contabilizada na derivada temporal do Jacobiano [4];
- **Gestão de redundância via Espaço Nulo:** projeção da lei de controle $K_n(q_{\text{home}} - q)$ no *kernel* de J para atrair o robô à postura preferida [9, 34].

2.3.3 Tratamento de Singularidades: Mínimos Quadrados Amortecidos (DLS)

Próximo às singularidades, os valores singulares de J se aproximam de zero, tornando a pseudo-inversa $J^+ = J^\top (JJ^\top)^{-1}$ numericamente instável. Nakamura e Hanafusa [8] propuseram a pseudo-inversa amortecida (DLS, *Damped Least Squares*):

$$J_\lambda^\dagger = J^\top (JJ^\top + \lambda^2 I)^{-1}, \quad (2.14)$$

onde:

- $J_\lambda^\dagger \in \mathbb{R}^{N \times m}$ é a pseudo-inversa amortecida (DLS) do Jacobiano;
- $J \in \mathbb{R}^{m \times N}$ é o Jacobiano (com $m \in \{3, 6\}$);
- $J^\top$ é a transposta do Jacobiano;
- $\lambda > 0$ é o fator de amortecimento, controla o compromisso entre precisão e estabilidade próximo de singularidades;
- $I \in \mathbb{R}^{m \times m}$ é a matriz identidade da mesma dimensão da tarefa.

Wampler [27] e Chiaverini et al. [7] analisaram as propriedades de estabilidade e a escolha ótima de λ em função dos valores singulares mínimos de J . A abordagem de DLS com λ variável (*variable DLS*), descrita em Chiaverini et al. [7], é especialmente robusta e consiste em aumentar λ continuamente conforme o valor singular mínimo decresce.

2.3.4 Inicialização por Levenberg-Marquardt com Busca em Linha

Para a inicialização da postura antes do início da trajetória, o *Hephaestus* utiliza uma variante do algoritmo de Levenberg-Marquardt [35, 36], que ajusta adaptativamente o parâmetro λ entre iterações com base no sucesso da redução do erro, combinado com uma busca em linha por *backtracking* (regra de Armijo [37]) para garantir descida suficiente a cada passo.

2.3.5 Erros de Orientação e SLERP

No espaço de tarefa 6D, as três últimas componentes correspondem à orientação angular do efetuador, codificada em $SO(3)$. Diferentemente da posição, em que o erro é simplesmente a diferença vetorial $e_p = P_d - P_c$, o erro de orientação não pode ser obtido pela subtração direta de matrizes de rotação, pois $SO(3)$ não é um espaço vetorial. Por isso, o erro de orientação é computado pela formulação de Anel [9], via extração do vetor axial da parte antissimétrica do produto $R_d R_c^\top - R_c R_d^\top$:

$$e_R = \frac{1}{2} \text{vee} \left(R_d R_c^\top - R_c R_d^\top \right), \quad (2.15)$$

onde:

- $e_R \in \mathbb{R}^3$ é o vetor de erro de orientação (formulação axial);
- $R_d \in SO(3)$ é a matriz de rotação desejada do efetuador;
- $R_c \in SO(3)$ é a matriz de rotação corrente do efetuador;
- $\text{vee}(\cdot) : \mathfrak{so}(3) \rightarrow \mathbb{R}^3$ é o operador que extrai o vetor axial associado à parte antissimétrica de uma matriz 3×3 .

Esta formulação é proporcional ao eixo-ângulo para erros pequenos e evita singularidades de representação por ângulos de Euler [4].

Para a interpolação suave entre duas orientações, é utilizada a *Spherical Linear Interpolation* (SLERP), proposta por Shoemake [6] no contexto de animação computacional. Dados dois quaternions (ou matrizes de rotação) R_0 e R_f , a SLERP define:

$$R(s) = R_0 \left(R_0^\top R_f \right)^s, \quad s \in [0, 1], \quad (2.16)$$

onde:

- $R(s) \in SO(3)$ é a matriz de rotação interpolada entre R_0 e R_f ;
- $R_0 \in SO(3)$ é a rotação inicial;

- $R_f \in SO(3)$ é a rotação final desejada;
- $s \in [0, 1]$ é o parâmetro de interpolação (0 corresponde a R_0 e 1 a R_f);
- a operação $(\cdot)^s$ é a potência fracionária no grupo $SO(3)$, equivalente a $\exp(s \log(R_0^\top R_f))$;
- garante interpolação geodésica (caminho mais curto) sobre $SO(3)$ [38].

2.4 Estratégias de Controle Não-Linear

2.4.1 Controle por Torque Computado (CTC)

O Controle por Torque Computado (*Computed Torque Control*, CTC), derivado da linearização por realimentação (*feedback linearization*) de Spong et al. [39], é a estratégia fundamental de controle baseado em modelo para robôs. A lei de controle cancela a não-linearidade da dinâmica e impõe uma dinâmica linear de malha fechada:

$$\tau = M(q) \left[\ddot{q}_d + K_D \dot{e} + K_P e + K_I \int e dt \right] + C(q, \dot{q}) \dot{q} + G(q), \quad (2.17)$$

onde:

- $\tau \in \mathbb{R}^N$ é o vetor de torque de controle aplicado;
- $M(q), C(q, \dot{q}), G(q)$ são as matrizes/vetor da dinâmica do robô (Equação (2.4));
- $\ddot{q}_d \in \mathbb{R}^N$ é a aceleração de referência (*feedforward*);
- $e = q_d - q \in \mathbb{R}^N$ é o erro de junta entre referência e estado corrente;
- $\dot{e}$ é a derivada temporal do erro;
- $K_P, K_D, K_I \in \mathbb{R}^{N \times N}$ são as matrizes de ganho proporcional, derivativo e integral, tipicamente diagonais.

Com modelo exato, a dinâmica de erro satisfaz $\ddot{e} + K_D \dot{e} + K_P e = 0$, que é um sistema linear de segunda ordem estável para $K_P, K_D > 0$ [5].

Na prática, erros de modelo introduzem distúrbios residuais. O termo integral (CTC-PID) melhora a rejeição a distúrbios constantes, mas requer *anti-windup* para evitar saturação do integrador [10]. A implementação no *Hephaestus* inclui *anti-windup* por clamping com limites ajustáveis.

2.4.2 Controle por Modo Deslizante (SMC)

O Controle por Modo Deslizante (*Sliding Mode Control*, SMC) é uma técnica de controle robusto que garante convergência à superfície deslizante $S = \dot{e} + \lambda e = 0$ mesmo na presença de incertezas e perturbações limitadas. A referência canônica é o livro de Slotine e Li [11], que apresenta a análise de estabilidade via função de Lyapunov $V = \frac{1}{2}S^\top S$.

A lei de controle implementada no *Hephaestus* segue a formulação por torque computado com superfície deslizante [11]:

$$\tau = M(q) [\ddot{q}_d - \lambda \dot{e} - K \tanh(S/\phi)] + C(q, \dot{q})\dot{q} + G(q), \quad (2.18)$$

onde:

- $\tau \in \mathbb{R}^N$ é o vetor de torque de controle SMC;
- $M(q), C(q, \dot{q}), G(q)$ são os termos da dinâmica nominal do robô;
- $\ddot{q}_d$ é a aceleração de referência;
- $\dot{e}$ é a derivada do erro de junta;
- λ é o parâmetro de pendente da superfície deslizante $S = \dot{e} + \lambda e$;
- K é a matriz de ganhos de robustez (define a magnitude do torque de chaveamento);
- $\phi > 0$ é a espessura da camada limite que substitui a função $\text{sign}(S)$ pela $\tanh(S/\phi)$, eliminando o *chattering* (oscilações de alta frequência) ao custo de uma robustez finita dentro da camada [11, 40];
- $\tanh(\cdot)$ é aplicada componente a componente.

A análise de robustez com camada limite foi formalizada por Burton e Zinober [12].

2.4.3 Algoritmo Super-Twisting (STA)

O algoritmo Super-Twisting (*Super-Twisting Algorithm*, STA), proposto por Levant [13], é um modo deslizante de segunda ordem que elimina o *chattering* intrinsecamente, produzindo um sinal de controle contínuo com convergência em tempo finito:

$$v_{\text{sw}} = -k_1 |S|^{1/2} \text{sign}(S) + z, \quad (2.19)$$

$$\dot{z} = -k_2 \text{sign}(S), \quad (2.20)$$

onde:

- v_{sw} é o sinal de chaveamento contínuo do STA;
- S é a variável de superfície deslizante (definida a partir do erro de junta);
- $|S|^{1/2}$ é a raiz quadrada do módulo de S , aplicada componente a componente;
- $\text{sign}(S)$ é a função sinal aplicada componente a componente;
- $k_1, k_2 > 0$ são os ganhos do STA, ajustados conforme a magnitude do distúrbio;
- z é uma variável interna que integra o sinal descontínuo, produzindo v_{sw} contínuo;
- $\dot{z}$ é a derivada temporal de z .

Moreno e Osorio [14] forneceram uma função de Lyapunov estrita para o STA, estabelecendo condições de ganho suficientes para rejeição de perturbações com derivada limitada: $k_2 > \delta$, $k_1 > 2\sqrt{k_2\delta}$, onde $|\dot{d}| \leq \delta$. A convergência em tempo finito diferencia o STA do SMC de primeira ordem, cuja convergência é assintótica [41].

2.4.4 Controle por Rejeição Ativa de Distúrbios (ADRC/LADRC)

O Controle por Rejeição Ativa de Distúrbios (*Active Disturbance Rejection Control*, ADRC), proposto por Han [42] e popularizado em versão linear (LADRC) por Gao [15], é uma abordagem paradigmática de controle que interpreta todas as incertezas do modelo, dinâmicas não-modeladas e perturbações externas como um único “distúrbio total” $f(q, \dot{q}, d, t)$, estimado em tempo real por um Observador de Estado Estendido (ESO) e cancelado pela lei de controle.

A dinâmica de segunda ordem de uma junta é reescrita como:

$$\ddot{q}_i = b_0\tau_i + f_i, \quad f_i \triangleq \text{distúrbio total}, \quad (2.21)$$

onde:

- $\ddot{q}_i$ é a aceleração da junta i ;
- τ_i é o torque aplicado à junta i ;
- b_0 é um ganho nominal conhecido (aqui, $b_0 \approx 1/M_{ii}$, sendo M_{ii} o elemento diagonal da matriz de inércia);
- f_i é o distúrbio total, que engloba acoplamentos dinâmicos com outras juntas, erros de modelo e perturbações externas.

O ESO de terceira ordem (linear) observa o estado aumentado $[q_i, \dot{q}_i, f_i]$:

$$\dot{z}_1 = z_2 - \beta_1(z_1 - q_i), \quad (2.22)$$

$$\dot{z}_2 = z_3 - \beta_2(z_1 - q_i) + b_0\tau_i, \quad (2.23)$$

$$\dot{z}_3 = -\beta_3(z_1 - q_i), \quad (2.24)$$

onde:

- z_1, z_2, z_3 são as estimativas de $q_i, \dot{q}_i$ e f_i produzidas pelo ESO;
- q_i é a posição medida da junta;
- τ_i é o torque aplicado;
- b_0 é o ganho nominal de canal;
- $\beta_1 = 3\omega_o, \beta_2 = 3\omega_o^2, \beta_3 = \omega_o^3$ são os ganhos do observador, parametrizados pela frequência de observador $\omega_o > 0$ [15].

A lei de controle cancela o distúrbio estimado z_3 :

$$\tau_i = \frac{u_0 - z_3}{b_0}, \quad u_0 = \ddot{q}_{d,i} + k_p(q_{d,i} - q_i) + k_d(\dot{q}_{d,i} - \dot{q}_i). \quad (2.25)$$

onde:

- τ_i é o torque de controle ADRC para a junta i ;
- u_0 é o sinal de controle nominal (PD com *feedforward* de aceleração);
- z_3 é a estimativa do distúrbio total f_i produzida pelo ESO;
- b_0 é o ganho nominal de canal;
- $\ddot{q}_{d,i}, \dot{q}_{d,i}, q_{d,i}$ são as referências de aceleração, velocidade e posição da junta i ;
- $k_p, k_d > 0$ são os ganhos proporcional e derivativo, respectivamente.

A análise de estabilidade do LADRC foi formalizada por Zheng e Gao [16]. A formulação descentralizada, um ESO por junta, ignorando o acoplamento dinâmico, é especialmente atraente para robótica: Huang e Xue [18] e trabalhos subsequentes demonstraram sua eficácia em manipuladores, onde os termos de Coriolis e gravidade são tratados como distúrbios a serem estimados pelo ESO. O artigo de revisão de Han [17] sintetiza a motivação filosófica da abordagem ADRC como extensão natural do controlador PID.

No *Hephaestus*, a implementação inclui feedforward explícito de $G(q)$ e $C(q, \dot{q})$, reduzindo o distúrbio residual que o ESO deve estimar para apenas erros de modelo e perturbações externas.

2.5 Planejamento de Trajetórias

O planejamento de trajetórias em robótica define os perfis de posição, velocidade e aceleração que o efetuador deve seguir ao longo do tempo. Siciliano et al. [4] distinguem entre planejamento no espaço de juntas e no espaço cartesiano.

2.5.1 Polinômio Cúbico e Perfil de *Jerk* Finito

Para interpolação de um ponto P_i a um ponto P_f em um tempo T , o polinômio cúbico (grau 3) garante continuidade de posição e velocidade nos extremos com velocidades inicial e final nulas [4, 5]:

$$s(t) = 3 \left(\frac{t}{T} \right)^2 - 2 \left(\frac{t}{T} \right)^3, \quad (2.26)$$

onde:

- $s(t) \in [0, 1]$ é a variável de progresso normalizada da trajetória, com $s(0) = 0$ e $s(T) = 1$;
- t é o tempo decorrido desde o início do trecho, com $0 \leq t \leq T$;
- T é a duração total do trecho de trajetória.

A primeira derivada satisfaz $\dot{s}(0) = \dot{s}(T) = 0$ e a aceleração máxima é $\ddot{s}_{\max} = 6/T^2$ no instante $T/2$. O polinômio de grau 5, embora garanta continuidade de aceleração nos extremos (menor *jerk*), não foi adotado aqui por apresentar valores de pico de aceleração ligeiramente superiores para o mesmo tempo de trajetória [4].

2.5.2 Trajetória Circular no Espaço Cartesiano

A trajetória circular é parametrizada por um centro C , dois vetores ortonormais e_1, e_2 no plano do arco, raio R e ângulo total $\theta_f - \theta_0$:

$$P(t) = C + R [\cos(\theta(t)) e_1 + \sin(\theta(t)) e_2], \quad (2.27)$$

onde:

- $P(t) \in \mathbb{R}^3$ é a posição cartesiana do efetuador em função do tempo;
- $C \in \mathbb{R}^3$ é o centro da circunferência;
- R é o raio do arco;
- $e_1, e_2 \in \mathbb{R}^3$ são vetores unitários ortonormais que definem o plano do arco;

- $\theta(t) = \theta_0 + (\theta_f - \theta_0) s(t)$ é o ângulo varrido em função do tempo, sendo θ_0 e θ_f os ângulos inicial e final do arco;
- $s(t)$ é o polinômio cúbico de Eq. (2.26), que produz partida e chegada suaves.

As velocidade e aceleração cartesianas são obtidas pela diferenciação analítica de $P(t)$ [4].

2.6 Computação Simbólica e Geração de Código Numérico

2.6.1 Álgebra Computacional Simbólica com SymPy

O projeto SymPy [2] é uma biblioteca Python de Álgebra Computacional Simbólica (*Computer Algebra System*, CAS) inteiramente implementada em Python puro, o que a torna portátil e integrável sem dependências externas de compilação. Meurer et al. [2] descrevem a arquitetura modular do SymPy, destacando o módulo `sympy.physics.mechanics`, que fornece o tipo `dynamicsymbols`, símbolos dependentes do tempo com diferenciação automática, e as funções do método de Lagrange que automatizam a derivação das equações de movimento a partir da energia cinética e potencial.

A manipulação simbólica de expressões de grande porte, como as matrizes $M(q)$ de sistemas com 6 ou mais graus de liberdade, pode produzir expressões com dezenas de milhares de subexpressões primitivas. A gestão eficiente dessas expressões é crítica para viabilizar a simulação numérica subsequente [43].

2.6.2 Eliminação de Subexpressões Comuns (CSE)

A Eliminação de Subexpressões Comuns (*Common Subexpression Elimination*, CSE) é uma otimização clássica de compiladores [3] que identifica subexpressões repetidas em uma expressão aritmética e as substitui por variáveis temporárias, reduzindo o número total de operações elementares. No contexto da simulação robótica, Moses [44] demonstrou que a simplificação algébrica de expressões cinemáticas pode reduzir em mais de uma ordem de grandeza o número de operações necessárias.

A função `sp.lambdify()` do SymPy com a *flag* `cse=True` aplica automaticamente a CSE antes de gerar o código Python/NumPy, detectando e fatorando subexpressões comuns nas matrizes M , C e G antes da geração do código numérico [2]. Isso reduz significativamente o tamanho do *bytecode* compilado, crítico para evitar `MemoryError` em sistemas de grande porte, e acelera a avaliação numérica por reutilização de subexpressões em *cache*.

2.6.3 Paralelismo na Derivação Simbólica

A derivação simbólica do vetor de Coriolis, que requer o cálculo de $\partial M/\partial q_k$ para $k = 1, \dots, N$ e de todas as combinações de símbolos de Christoffel (Equação (2.5)), possui complexidade $O(N^3)$ em termos de operações simbólicas, tornando-se o principal gargalo de tempo para sistemas com muitos graus de liberdade.

O módulo `concurrent.futures.ProcessPoolExecutor` da biblioteca padrão do Python permite a distribuição dessas tarefas entre múltiplos processos, contornando o *Global Interpreter Lock* (GIL) do CPython [45]. A abordagem de *pool* de processos com passagem de dados simbólicos via `pickle` foi validada por Leu et al. [43] para sistemas de dinâmica multivariável de grande porte.

2.6.4 Integração Numérica: Método de Euler Simplético

A integração numérica das equações de movimento (2.4) é realizada pelo método de Euler Simplético (também chamado de Euler Simplético de primeira ordem ou método de Euler semi-implícito). Em contraste com o Euler Explícito padrão (atualização de q e $\dot{q}$ simultaneamente com valores do passo anterior), o Euler Simplético atualiza primeiro a velocidade e depois a posição:

$$\dot{q}^{n+1} = \dot{q}^n + \ddot{q}^n \Delta t, \quad (2.28)$$

$$q^{n+1} = q^n + \dot{q}^{n+1} \Delta t. \quad (2.29)$$

onde:

- $q^n, \dot{q}^n, \ddot{q}^n$ são, respectivamente, a posição, velocidade e aceleração generalizadas avaliadas no instante t_n ;
- $q^{n+1}, \dot{q}^{n+1}$ são a posição e velocidade no instante $t_{n+1} = t_n + \Delta t$;
- Δt é o passo de integração;
- a aceleração $\ddot{q}^n$ é obtida resolvendo a equação dinâmica (2.4) no estado $(q^n, \dot{q}^n)$, ou seja, $\ddot{q}^n = M^{-1}(q^n) [\tau^n - C(q^n, \dot{q}^n) \dot{q}^n - G(q^n)]$;
- observe que a posição é atualizada com a velocidade já corrigida $\dot{q}^{n+1}$ (semi-implícito), o que confere a propriedade simplética ao integrador.

Esta sequência é um método simplético de primeira ordem, pertence à família de integradores que preservam a estrutura Hamiltoniana do sistema [46, 47]. Sua principal vantagem sobre o Euler Explícito é a preservação de energia a longo prazo: enquanto o Euler Explícito apresenta acúmulo secular de energia (instabilidade numérica suave), o Euler Simplético preserva uma “energia modificada” próxima da

real [46]. Para simulações de controle em malha fechada com duração moderada (tipicamente < 30 s), essa diferença é crítica para a fidelidade dos resultados.

2.7 Ferramentas de Simulação para UVMS: Estado da Arte

O desenvolvimento de simuladores para robótica subaquática é um campo ativo. O ambiente UWSim, descrito por Prats et al. [48], foi um dos primeiros a oferecer simulação de UVMS com integração ao ROS (*Robot Operating System*) e renderização realista por OpenSceneGraph. UWSim suporta múltiplos veículos e manipuladores, mas sua arquitetura de simulação dinâmica é baseada em Newton-Euler recursivo sem suporte nativo à modelagem simbólica.

O simulador DAVE, apresentado por Zhang et al. [49], estende o Gazebo com modelos hidrodinâmicos para veículos e manipuladores subaquáticos, integrando ao ROS2. Apesar da fidelidade visual e da integração com *hardware* real, a caixa-preta do motor de física do Gazebo (ODE ou Bullet) não expõe as equações analíticas, dificultando estudos de modelagem comparativa.

No domínio educacional, trabalhos como o de Corke et al. [50] propõem ambientes simplificados em Python para ensino de controle de manipuladores. Contudo, esses ambientes não contemplam a modelagem hidrodinâmica completa de UVMS nem a transição simbólico-numérica otimizada.

O *Hephaestus* diferencia-se dessas ferramentas por: (i) derivar analiticamente e exibir as equações lagrangianas explícitas; (ii) suportar modelos de massa adicionada 6D por elo; (iii) implementar quatro estratégias de controle não-linear comparáveis em uma única interface; e (iv) operar como aplicação *standalone* sem dependência de ROS ou Gazebo, distribuída via PyInstaller.

2.8 Síntese da Revisão

Sintetizam-se, na Tabela 2.1, as referências seminais e os módulos do *Hephaestus* correspondentes, com destaque para a cobertura da literatura em cada componente técnico.

A revisão apresentada neste capítulo evidencia que o *Hephaestus* não apenas integra o estado da arte em cada uma das áreas abordadas, mas o faz por meio de uma cadeia de software transparente e documentada, onde cada escolha de algoritmo pode ser rastreada à literatura científica correspondente. Os capítulos seguintes detalham a arquitetura de implementação e os resultados experimentais obtidos com a plataforma.

Tabela 2.1: Mapeamento entre fundamentos teóricos e componentes do *Hephaestus*.

Fundamento	Referências Principais	Componente
Cinemática FK / Jacobi-ano	[4, 5, 23]	step_1_kinematics
Dinâmica Lagrangiana (M, C, G)	[23, 24]	step_2, step_3
Modelagem Hidrodinâmica	[1, 19]	RobotMathHydro
CLIK + DLS + Espaço Nulo	[7–9]	solve_ik_numerical
SLERP	[6, 38]	_slerp_rotation
CTC-PID	[5, 10]	run (CTC-PID)
SMC com camada limite	[11, 40]	SlidingModeControl
Super-Twisting STA	[13, 14]	SuperTwistingSMC
LADRC / ESO	[15–17]	Decentralized_-LADRC
CSE + lambdify	[2, 3]	RobotSimulator.__init__
Euler Simplético	[46, 47]	run (integrador)

Capítulo 3

Modelagem Matemática e Geração Simbólica

Este capítulo apresenta, em detalhe, a arquitetura de modelagem matemática implementada no núcleo computacional do *Hephaestus*. O objetivo central é descrever como a cadeia cinemática de um manipulador genérico de N graus de liberdade, em ambiente terrestre ou subaquático, é traduzida em expressões simbólicas fechadas para a matriz de inércia $M(q)$, o vetor de Coriolis $C(q, \dot{q})$ e o vetor de gradiente de potencial $G(q)$, bem como o processo de compilação dessas expressões em funções numéricas otimizadas para simulação em tempo real. A exposição combina a fundamentação teórica dos conceitos revisados no Capítulo 2 com as escolhas concretas de implementação que diferenciam o *Hephaestus* das abordagens clássicas.

3.1 Visão Geral da Arquitetura

A *engine* matemática do *Hephaestus* é organizada em uma hierarquia de classes: `RobotMathEngine`, responsável pela modelagem em ambiente seco (ar), e `RobotMathHydro`, subclasse que estende o modelo para ambiente subaquático, incorporando os efeitos de massa adicionada e empuxo. A filosofia de projeto segue o princípio de separação entre a camada simbólica (geração das equações exatas) e a camada numérica (avaliação eficiente em tempo de simulação), conforme proposto por Meurer et al. [2].

O fluxo completo de processamento é organizado em quatro etapas sequenciais, refletidas nos métodos públicos da classe:

1. **Cinemática Simbólica** (`step_1_kinematics`): construção das transformações homogêneas acumuladas, das posições dos centros de massa e das velocidades angulares de cada elo.

2. **Matriz de Inércia e Gradiente de Potencial** (`step_2_jacobian_M_G`): derivação simbólica de $M(q)$ e $G(q)$ por meio do método do Jacobiano da inércia, com rotação explícita dos tensores locais para o referencial global.
3. **Vetor de Coriolis** (`step_3_coriolis_combined`): cálculo dos símbolos de Christoffel de segunda espécie a partir de $\partial M / \partial q_k$, com suporte a paralelismo por múltiplos processos.
4. **Preparação e Exportação** (`step_4_prepare_export`): substituição de símbolos dinâmicos por variáveis estáticas e empacotamento do modelo para o módulo de simulação.

Essa organização modular permite que o usuário inspecione o estado intermediário após cada etapa, facilitando a depuração matemática e a validação pedagógica das equações geradas, objetivo central de uma plataforma didática de código aberto [50].

3.2 Representação da Cadeia Cinemática

3.2.1 Configuração de Juntas e Máscara de Vetores de Elo

A cadeia cinemática é especificada pelo usuário por meio de dois parâmetros de entrada:

- **joint_config**: lista de strings descrevendo o tipo e eixo de cada junta. O primeiro caractere indica o tipo, 'R' para junta de revolução (*Revolute*) ou 'D' para junta prismática (*Displacement*), e o segundo caractere indica o eixo de movimento ('x', 'y' ou 'z').
- **link_vectors_mask**: lista de vetores tridimensionais (u_x, u_y, u_z) que definem a direção do elo estrutural ligado à respectiva junta no referencial local. O comprimento efetivo do elo é parametrizado pelo símbolo escalar L_i , de modo que o vetor de translação do elo i é $\vec{r}_i = (u_x, u_y, u_z)^i \cdot L_i$.

Essa parametrização admite qualquer topologia de cadeia serial aberta, incluindo configurações com juntas prismáticas mistas, fundamentais para modelar o grau de liberdade de afundamento de veículos subaquáticos, e configurações com elos de comprimento nulo (vetor máscara zero), usadas para modelar as três rotações do veículo base de um UVMS sem translação associada [1, 19].

A escolha de uma parametrização matricial de rotações diretas em vez da convenção de Denavit-Hartenberg (DH) [25] foi deliberada. Embora a convenção DH

seja amplamente ensinada [5, 23], ela impõe restrições geométricas sobre o posicionamento dos eixos de coordenadas que dificultam a representação de certas arquiteturas (e.g., juntas com deslocamentos paralelos). A representação por matrizes de rotação elementares por eixo-ângulo, adotada no *Hephaestus*, é algebricamente transparente e diretamente integrável com o módulo de mecânica simbólica do SymPy [2, 43].

3.2.2 Símbolos Dinâmicos e Parâmetros

Para cada junta i , são criados automaticamente os seguintes símbolos SymPy:

- $q_i(t)$, $\dot{q}_i(t)$: variáveis de junta e sua derivada temporal, instanciadas como `dynamicsymbols` para suportar diferenciação automática [2];
- m_i , L_i : massa e comprimento do elo i ;
- cx_i , cy_i , cz_i : coordenadas do centro de massa no referencial local do elo i ;
- Ixx_i , Iyy_i , Izz_i : momentos principais de inércia no referencial local.

Adicionalmente, o símbolo escalar g representa a aceleração gravitacional. No modo hidrodinâmico, o símbolo ρ (densidade do fluido) e os volumes deslocados vol_i são adicionados à lista de parâmetros. Todos esses símbolos são agrupados em `params_list`, que serve como assinatura da função compilada [2]. Essa abordagem garante que as equações geradas sejam *parametricamente completas*: um único modelo simbólico serve para qualquer instância paramétrica do robô, sem necessidade de recomputação quando os valores físicos são alterados, propriedade ausente em geradores de código que utilizam substituição numérica direta [24].

3.3 Cinemática Simbólica Direta

3.3.1 Transformações Homogêneas Acumuladas

A cinemática direta é construída iterativamente sobre a cadeia de elos. Para a junta i do tipo revolução em torno do eixo $\hat{a}_i$ (expresso no referencial global acumulado R_{acc}), a matriz de rotação local é:

$$R_j^{(i)} = \exp(\hat{a}_i^{(l)} q_i), \quad \hat{a}_i^{(l)} \in \{\hat{x}, \hat{y}, \hat{z}\}, \quad (3.1)$$

onde:

- $R_j^{(i)} \in \mathbb{R}^{3 \times 3}$ é a matriz de rotação local introduzida pela junta i ;
- $\hat{a}_i^{(l)}$ é o eixo unitário de rotação da junta no referencial local, mapeado para as matrizes elementares R_x , R_y ou R_z conforme o caractere do tipo de junta;

- q_i é a coordenada generalizada (ângulo de rotação) da junta i ;
- $\exp(\cdot)$ denota a exponencial matricial, equivalente à fórmula de Rodrigues para rotações elementares [4, 5].

Para a junta prismática ao longo do eixo $\hat{a}_i^{(l)}$:

$$R_j^{(i)} = I_3, \quad P_j^{(i)} = \hat{a}_i^{(l)} q_i. \quad (3.2)$$

onde:

- $R_j^{(i)} = I_3$ é a matriz identidade 3×3 , indicando que a junta prismática não introduz rotação local;
- $P_j^{(i)} \in \mathbb{R}^3$ é o vetor de translação devido ao deslocamento da junta;
- $\hat{a}_i^{(l)}$ é o eixo unitário de translação da junta no referencial local;
- q_i é a coordenada generalizada (deslocamento linear) da junta i .

A transformação homogênea 4×4 acumulada até o extremo do elo i é:

$$T_i = T_{i-1} \cdot T_{\text{junta},i} \cdot T_{\text{elo},i}, \quad (3.3)$$

onde:

- $T_i \in \mathbb{R}^{4 \times 4}$ é a transformação homogênea acumulada do referencial global até o extremo do elo i ;
- T_{i-1} é a transformação acumulada do elo anterior, com $T_0 = I_4$ (identidade 4×4);
- $T_{\text{junta},i}$ encapsula a rotação/translação introduzida pela junta i (Equações (3.1) ou (3.2));
- $T_{\text{elo},i}$ codifica a translação estrutural do elo, com vetor $\vec{r}_i = L_i \cdot \text{link_vectors_mask}[i]$;
- L_i é o comprimento simbólico do elo i .

Isso é computado simbolicamente pelo SymPy, produzindo expressões fechadas que dependem de $\{q_1, \dots, q_N\}$ e $\{L_1, \dots, L_N\}$ [23].

3.3.2 Posições dos Centros de Massa no Referencial Global

A posição global do centro de massa do elo i é obtida por:

$$\vec{p}_{\text{cm},i} = T_{\text{at-joint},i} \cdot \begin{bmatrix} cx_i \\ cy_i \\ cz_i \\ 1 \end{bmatrix}_{(1:3)}, \quad (3.4)$$

onde:

- $\vec{p}_{\text{cm},i} \in \mathbb{R}^3$ é a posição do centro de massa do elo i no referencial global;
- $T_{\text{at-joint},i} = T_{i-1} \cdot T_{\text{junta},i}$ é a transformação homogênea até o *início* do elo i (antes da translação estrutural);
- $[cx_i, cy_i, cz_i]$ são as coordenadas do centro de massa no referencial local do elo i ;
- o índice $(1 : 3)$ indica a extração das três primeiras componentes do vetor homogêneo resultante.

Essa convenção separa o CM do extremo do elo, permitindo modelar distribuições de massa arbitrárias ao longo do elo sem hipóteses simplificadoras de CM centrado [4, 23].

3.3.3 Velocidades Angulares Simbólicas

A velocidade angular global do elo i é acumulada recursivamente:

$$\vec{\omega}_i = \vec{\omega}_{i-1} + R_{\text{acc},i-1} \cdot \hat{a}_i^{(l)} \cdot \dot{q}_i \quad (\text{juntas R}), \quad \vec{\omega}_i = \vec{\omega}_{i-1} \quad (\text{juntas D}). \quad (3.5)$$

onde:

- $\vec{\omega}_i \in \mathbb{R}^3$ é a velocidade angular do elo i expressa no referencial global;
- $\vec{\omega}_{i-1}$ é a velocidade angular do elo anterior na cadeia, com $\vec{\omega}_0 = \vec{0}$;
- $R_{\text{acc},i-1} \in \mathbb{R}^{3 \times 3}$ é a matriz de rotação global acumulada até o elo $i - 1$;
- $\hat{a}_i^{(l)}$ é o eixo unitário de rotação da junta i no referencial local;
- $\dot{q}_i$ é a velocidade generalizada (taxa angular) da junta i ;
- “juntas R” refere-se a juntas rotacionais e “juntas D” a juntas prismáticas (*Displacement*), que não contribuem para a velocidade angular.

Essa recorrência segue a formulação vetorial da velocidade angular para cadeias seriais [5, 28]. O resultado é uma expressão simbólica linear em $\{\dot{q}_1, \dots, \dot{q}_N\}$, cuja derivada em relação a $\dot{q}_j$ fornece diretamente a j -ésima coluna do Jacobiano angular $J_{\omega,i}$ [4]. O fluxo completo da etapa de cinemática simbólica, da especificação de juntas ao Jacobiano geométrico, é sintetizado na Figura 3.1.

3.4 Derivação Simbólica da Matriz de Inércia e do Vetor de Gravidade

3.4.1 Método do Jacobiano da Inércia

A derivação da matriz de inércia generalizada $M(q)$ segue o método do Jacobiano da inércia [4], que expressa a energia cinética total como:

$$T = \frac{1}{2} \dot{q}^\top M(q) \dot{q}, \quad (3.6)$$

onde:

- $T \in \mathbb{R}$ é a energia cinética total do sistema;
- $\dot{q} \in \mathbb{R}^N$ é o vetor de velocidades generalizadas (uma componente por junta);
- $M(q) \in \mathbb{R}^{N \times N}$ é a matriz de inércia generalizada, simétrica e definida positiva;
- N é o número de graus de liberdade da cadeia cinemática.

A matriz de inércia é construída pela soma das contribuições de cada elo:

$$M(q) = \sum_{i=1}^N \left[m_i J_{v,i}^\top J_{v,i} + J_{\omega,i}^\top I_i^{(g)} J_{\omega,i} \right]. \quad (3.7)$$

onde:

- $m_i \in \mathbb{R}$ é a massa do elo i ;
- $J_{v,i} \in \mathbb{R}^{3 \times N}$ é o Jacobiano de velocidade linear do centro de massa do elo i ;
- $J_{\omega,i} \in \mathbb{R}^{3 \times N}$ é o Jacobiano de velocidade angular do elo i ;
- $I_i^{(g)} \in \mathbb{R}^{3 \times 3}$ é o tensor de inércia do elo i expresso no referencial global;
- N é o número total de elos da cadeia.

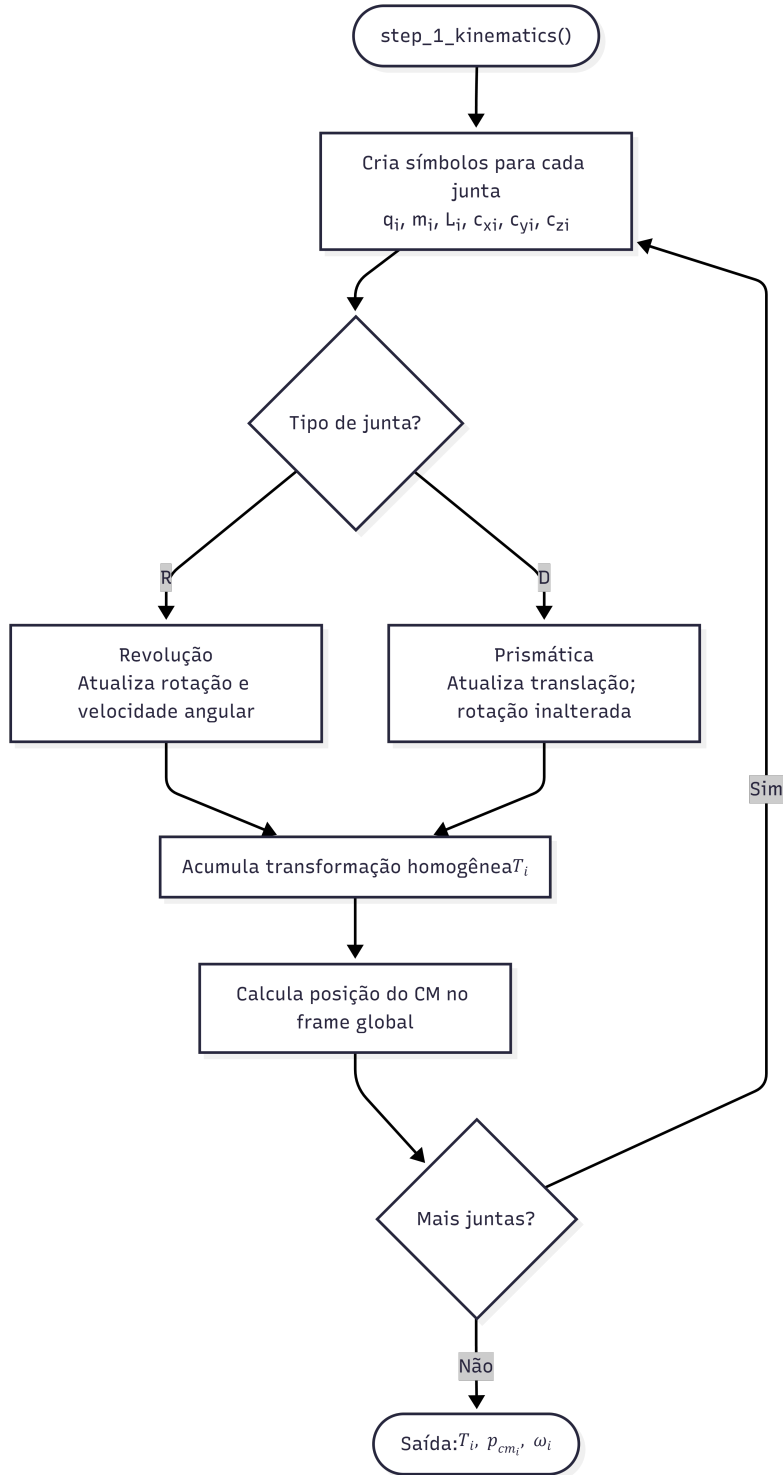


Figura 3.1: Fluxo de cinemática simbólica implementado em `step_1_kinematics`: da especificação de `joint_config` e `link_vectors_mask` à construção das transformações homogêneas acumuladas T_i , das posições dos centros de massa $\vec{p}_{cm,i}$, das velocidades angulares $\vec{\omega}_i$ e do Jacobiano geométrico completo $J(q) \in \mathbb{R}^{6 \times N}$.

Os Jacobianos de velocidade translacional e angular do CM do elo i são obtidos diretamente por diferenciação simbólica:

$$J_{v,i}(q) = \frac{\partial \vec{p}_{\text{cm},i}}{\partial q}, \quad J_{\omega,i}(\dot{q}) = \frac{\partial \vec{\omega}_i}{\partial \dot{q}}, \quad (3.8)$$

onde:

- $J_{v,i}(q) \in \mathbb{R}^{3 \times N}$ é o Jacobiano de velocidade linear do centro de massa do elo i ;
- $J_{\omega,i}(\dot{q}) \in \mathbb{R}^{3 \times N}$ é o Jacobiano de velocidade angular do elo i ;
- $\vec{p}_{\text{cm},i} \in \mathbb{R}^3$ é a posição do CM do elo i (Equação (3.4));
- $\vec{\omega}_i \in \mathbb{R}^3$ é a velocidade angular global do elo i (Equação (3.5));
- $\partial/\partial q$ e $\partial/\partial \dot{q}$ denotam as derivadas parciais matriciais em relação aos vetores q e $\dot{q}$.

Estas derivadas são computadas pelo método `.jacobian()` do SymPy. O tensor de inércia global $I_i^{(g)}$ é obtido pela transformação de Steiner:

$$I_i^{(g)} = R_{\text{acc},i} I_i^{(l)} R_{\text{acc},i}^\top, \quad I_i^{(l)} = \text{diag}(Ixx_i, Iyy_i, Izz_i), \quad (3.9)$$

onde:

- $I_i^{(g)} \in \mathbb{R}^{3 \times 3}$ é o tensor de inércia do elo i expresso no referencial global;
- $I_i^{(l)} \in \mathbb{R}^{3 \times 3}$ é o tensor de inércia local, diagonal nas coordenadas principais do elo;
- $R_{\text{acc},i} \in \mathbb{R}^{3 \times 3}$ é a matriz de rotação global acumulada até o elo i ;
- Ixx_i, Iyy_i, Izz_i são os momentos principais de inércia do elo no referencial local.

A hipótese de tensor diagonal em coordenadas principais locais, amplamente adotada quando os eixos de inércia são alinhados ao referencial de junta [4, 23], é suficiente para descrever elos com geometria simétrica.

3.4.2 Vetor de Gradiente de Potencial Gravitacional

A energia potencial total do sistema é:

$$V(q) = \sum_{i=1}^N m_i g z_{\text{cm},i}(q), \quad (3.10)$$

onde:

- $V(q) \in \mathbb{R}$ é a energia potencial gravitacional total do sistema;
- m_i é a massa do elo i ;
- g é a aceleração da gravidade;
- $z_{\text{cm},i}(q)$ é a componente vertical (terceira coordenada) de $\vec{p}_{\text{cm},i}$, dependente da configuração q ;
- N é o número de elos.

O vetor de gravidade generalizada é então:

$$G(q) = \left(\frac{\partial V}{\partial q} \right)^\top = \frac{\partial}{\partial q} \left[\sum_{i=1}^N m_i g z_{\text{cm},i} \right]^\top, \quad (3.11)$$

onde:

- $G(q) \in \mathbb{R}^N$ é o vetor de torques generalizados devidos à gravidade;
- $\partial V / \partial q \in \mathbb{R}^{1 \times N}$ é o gradiente da energia potencial em relação ao vetor de coordenadas generalizadas q ;
- cada componente $G_i(q)$ representa o torque generalizado necessário para sustentar o peso estático do sistema na configuração q [5, 23];
- $(\cdot)^\top$ denota transposição, convertendo o gradiente-linha em vetor-coluna.

Esse vetor é computado pelo método `sp.Matrix([V_tot]).jacobian(self.q).T`.

3.4.3 Jacobiano Geométrico Completo

O Jacobiano geométrico do efetuador $J(q) \in \mathbb{R}^{6 \times N}$ é montado pela concatenação das linhas translacionais (derivada da posição do extremo final em relação a q) com as linhas rotacionais (derivada de $\vec{\omega}_N$ em relação a $\dot{q}$):

$$J(q) = \begin{bmatrix} J_{\text{lin}} \\ J_{\text{ang}} \end{bmatrix}, \quad J_{\text{lin}} = \frac{\partial \vec{p}_{\text{end}}}{\partial q} \in \mathbb{R}^{3 \times N}, \quad J_{\text{ang}} = \frac{\partial \vec{\omega}_N}{\partial \dot{q}} \in \mathbb{R}^{3 \times N}. \quad (3.12)$$

onde:

- $J(q) \in \mathbb{R}^{6 \times N}$ é o Jacobiano geométrico completo do efetuador;
- J_{lin} é a parte linear, obtida pela derivada parcial da posição $\vec{p}_{\text{end}}$ do extremo final em relação a q ;

- J_{ang} é a parte angular, obtida pela derivada parcial da velocidade angular $\vec{\omega}_N$ do último elo em relação a $\dot{q}$;
- $\vec{p}_{\text{end}} \in \mathbb{R}^3$ é a posição cartesiana do efetuador no referencial global;
- $\vec{\omega}_N \in \mathbb{R}^3$ é a velocidade angular global do último elo da cadeia.

A disponibilidade do Jacobiano $6 \times N$ completo viabiliza o *controle no espaço de tarefa 6D*, em que três coordenadas descrevem a **posição** cartesiana (x, y, z) do efetuador e três coordenadas descrevem a sua **orientação** angular (*roll, pitch, yaw*, ou, equivalentemente, o eixo-ângulo de uma rotação em $SO(3)$). As três primeiras linhas J_{lin} projetam as velocidades de junta na velocidade linear do efetuador, enquanto as três últimas J_{ang} as projetam na velocidade angular, totalizando seis graus de liberdade da tarefa cartesiana. Esta é uma extensão crítica em relação a abordagens que consideram apenas o Jacobiano posicional $3 \times N$, que são insuficientes para controle de orientação em tarefas de manipulação subaquática, soldagem, alinhamento com superfícies inclinadas ou inserção de peças, nas quais o “onde” (posição) e o “como” (orientação) do efetuador devem ser controlados simultaneamente [4, 9]. O algoritmo de cinemática inversa em malha fechada que opera sobre este Jacobiano completo é detalhado na Seção 4.4.3 do Capítulo 4. O fluxo das Seções 3.4 e 3.3 é sintetizado na Figura 3.2, na qual se evidencia como $M(q)$, $G(q)$ e $J(q)$ são derivados a partir das posições e Jacobianos de cada elo.

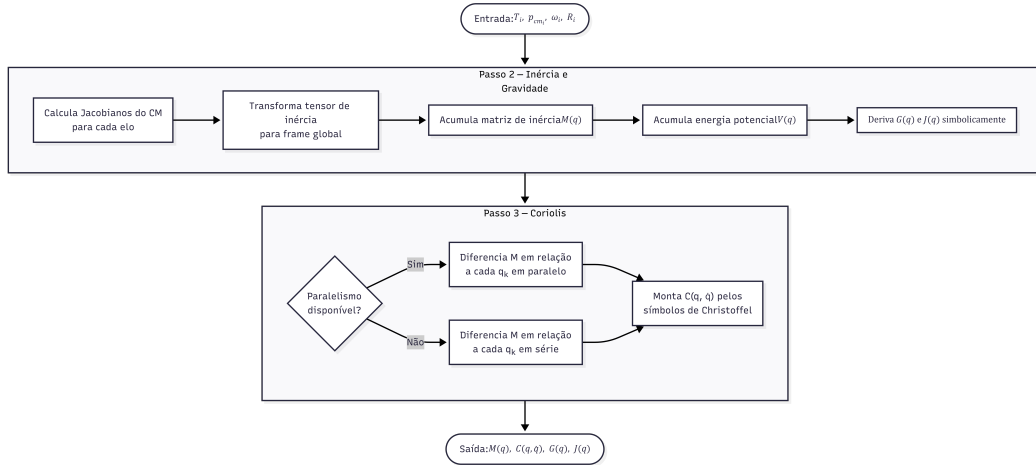


Figura 3.2: Fluxo de derivação simbólica da matriz de inércia $M(q)$, do vetor de gradiente de potencial $G(q)$ e dos coeficientes de Coriolis pelo método dos símbolos de Christoffel, implementado em `step_2_jacobian_M_G` e `step_3_coriolis_combined`.

3.5 Cálculo Simbólico do Vetor de Coriolis

3.5.1 Formulação pelos Símbolos de Christoffel

O vetor de termos de Coriolis e centrípetos $C(q, \dot{q}) \in \mathbb{R}^N$ é calculado pelos símbolos de Christoffel de segunda espécie [4, 23]. Cada componente do vetor C é:

$$[C(q, \dot{q})]_i = \sum_{j=1}^N \sum_{k=j}^N c_{ijk}(q) \dot{q}_j \dot{q}_k \cdot \varepsilon_{jk}, \quad (3.13)$$

onde:

- $[C(q, \dot{q})]_i$ é a i -ésima componente do vetor de Coriolis;
- $c_{ijk}(q)$ são os símbolos de Christoffel de segunda espécie associados à matriz de inércia $M(q)$;
- $\dot{q}_j$ e $\dot{q}_k$ são velocidades generalizadas das juntas j e k ;
- ε_{jk} é o coeficiente de simetria, com $\varepsilon_{jk} = 1$ se $j = k$ (termos centrípetos puros) e $\varepsilon_{jk} = 2$ se $j \neq k$ (termos de Coriolis cruzados);
- N é o número de graus de liberdade.

Os coeficientes de Christoffel são:

$$c_{ijk} = \frac{1}{2} \left(\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{jk}}{\partial q_i} \right). \quad (3.14)$$

onde:

- $c_{ijk}(q) \in \mathbb{R}$ é o coeficiente de Christoffel de segunda espécie associado aos índices (i, j, k) ;
- M_{ij}, M_{ik}, M_{jk} são elementos da matriz de inércia $M(q)$;
- $\partial M_{ab} / \partial q_c$ é a derivada parcial do elemento M_{ab} em relação à coordenada generalizada q_c .

Esta formulação é matematicamente equivalente às linhas C_{ij} da matriz de Coriolis clássica (Equação (2.5) do Capítulo 2), mas diretamente no formato vetorial $C(q, \dot{q})\dot{q}$, evitando a construção da matriz $N \times N$ completa, o que reduz o número total de operações de $O(N^3)$ para o cálculo dos coeficientes individuais sem custos adicionais de produto matriz-vetor.

A implementação segue dois sub-passos. No primeiro (*Passo 3a*), a matriz de inércia M é diferenciada em relação a cada variável de junta q_k , produzindo as

matrizes $\partial M/\partial q_k \in \mathbb{R}^{N \times N}$ para $k = 1, \dots, N$. No segundo (*Passo 3b*), cada linha i do vetor C é montada pela varredura de todos os pares (j, k) com $k \geq j$, utilizando as matrizes previamente calculadas. A exploração da simetria ($k \geq j$) reduz o número de pares de N^2 para $N(N+1)/2$ [24].

3.5.2 Estratégia de Paralelização da Derivação

A derivação simbólica de $\partial M/\partial q_k$ é a operação mais custosa de toda a geração do modelo: em sistemas com N graus de liberdade, cada um dos N gradientes simbólicos exige a diferenciação de uma matriz $N \times N$ com entradas que podem possuir centenas de subexpressões, resultando em uma complexidade efetiva de $O(N^3)$ a $O(N^4)$ no tamanho das expressões para configurações gerais [24, 43].

Para mitigar esse custo, o *Hephaestus* distribui as N tarefas de diferenciação $M \rightarrow \partial M/\partial q_k$ entre múltiplos processos via `ProcessPoolExecutor` da biblioteca padrão do Python:

$$\mathbf{dM_dq}[k] = \frac{\partial M}{\partial q_k}, \quad k = 1, \dots, N, \quad \text{computados em paralelo por } W \text{ workers.} \quad (3.15)$$

onde:

- $\mathbf{dM_dq}[k] \in \mathbb{R}^{N \times N}$ é a matriz de derivadas parciais $\partial M/\partial q_k$ armazenada na posição k da lista resultante;
- $M \in \mathbb{R}^{N \times N}$ é a matriz de inércia simbólica completa;
- q_k é a k -ésima coordenada generalizada;
- N é o número de graus de liberdade;
- W é o número de processos *workers* alocados ao `ProcessPoolExecutor`.

O uso de processos (em vez de *threads*) contorna a limitação do *Global Interpreter Lock* (GIL) do CPython, que impede a execução simultânea de código Python puro em múltiplos *threads* [45]. A serialização dos objetos simbólicos entre processos é viabilizada pela integração nativa do SymPy com o módulo `pickle` do Python, que representa expressões simbólicas como árvores de objetos serializáveis [2].

Analogamente, o cálculo das N linhas do vetor C (cada linha requerendo $O(N^2)$ operações simbólicas sobre as matrizes $\partial M/\partial q_k$) é também paralelizado por `ProcessPoolExecutor`, com cada *worker* responsável por uma linha i do vetor:

$$C_i = \sum_{j=1}^N \sum_{k=j}^N c_{ijk} \dot{q}_j \dot{q}_k \cdot \varepsilon_{jk}, \quad i = 1, \dots, N, \quad \text{computados em paralelo.} \quad (3.16)$$

onde:

- C_i é a i -ésima componente do vetor de Coriolis (cf. Equação (3.13));
- c_{ijk} são os coeficientes de Christoffel definidos pela Equação (3.14);
- $\dot{q}_j, \dot{q}_k$ são as velocidades generalizadas das juntas j e k ;
- ε_{jk} é o coeficiente de simetria (1 ou 2);
- N é o número de graus de liberdade, e cada índice i é processado por um *worker* distinto.

Uma decisão de implementação relevante é a *omissão intencional* da chamada a `sp.collect()` após a montagem de cada linha C_i . Embora a coleta de termos semelhantes possa compactar a expressão simbólica, seu custo computacional é superlinear no número de termos e pode superar em ordem de grandeza o custo do cálculo próprio da linha. Em vez disso, a compactação é delegada ao passo de CSE (*Common Subexpression Elimination*) realizado posteriormente no `lambdify`, que fatoriza subexpressões repetidas de forma sistemática e com custo linear no tamanho da expressão final [2, 3].

3.5.3 Modo Serial para Ambientes Executáveis

Quando o *Hephaestus* é distribuído como executável via PyInstaller (arquivo `.exe`), o suporte ao *spawn* de subprocessos no Windows impõe restrições de inicialização que tornam o `ProcessPoolExecutor` incompatível com módulos congelados (*frozen*). Nesse cenário, o número de *workers* é automaticamente configurado para 1, e a computação é realizada em série na mesma *thread* principal. Essa detecção é realizada pelo parâmetro `num_workers` da classe `RobotMathEngine`: quando `num_workers = 1`, a condição `use_parallel = False` desativa o executor, garantindo execução correta em qualquer ambiente sem modificações no código [43].

3.6 Extensão Hidrodinâmica: RobotMathHydro

A classe `RobotMathHydro` herda integralmente de `RobotMathEngine` e sobrecarrega os métodos `step_1_kinematics_hydro` e `step_2_jacobian_M_G` para incorporar os efeitos físicos do ambiente aquático, conforme a formulação de Fossen [1] e Antonelli [19].

3.6.1 Parâmetros Adicionais: Volumes e Massa Adicionada

No passo de cinemática hidrodinâmica, após a execução completa de `step_1_kinematics`, são acrescentados à lista de parâmetros os volumes deslocados vol_i de cada elo ($i = 1, \dots, N$). Esses volumes determinam a força de empuxo por elo segundo o princípio de Arquimedes:

$$B_i = \rho g \text{vol}_i, \quad (3.17)$$

onde:

- B_i é o módulo da força de empuxo aplicada ao elo i ;
- ρ é a densidade do fluido (no caso típico, água do mar) [32];
- g é a aceleração da gravidade;
- vol_i é o volume submerso deslocado pelo elo i .

No passo de dinâmica hidrodinâmica, os coeficientes de massa adicionada de cada elo são instanciados como seis símbolos escalares:

$$\{ma_{u,i}, ma_{v,i}, ma_{w,i}\} \quad (\text{translacionais}), \quad \{ma_{p,i}, ma_{q,i}, ma_{r,i}\} \quad (\text{rotacionais}), \quad (3.18)$$

onde:

- $ma_{u,i}, ma_{v,i}, ma_{w,i}$ são os coeficientes de massa adicionada translacionais nas direções u (*surge*), v (*sway*) e w (*heave*) do elo i , no referencial local;
- $ma_{p,i}, ma_{q,i}, ma_{r,i}$ são os coeficientes de massa adicionada rotacionais em torno dos eixos p (*roll*), q (*pitch*) e r (*yaw*) do elo i , no referencial local;
- o índice i percorre todos os elos da cadeia ($i = 1, \dots, N$).

Esses símbolos formam as matrizes diagonais locais $M_{A,\text{lin},i}^{(l)} = \text{diag}(ma_{u,i}, ma_{v,i}, ma_{w,i})$ e $M_{A,\text{rot},i}^{(l)} = \text{diag}(ma_{p,i}, ma_{q,i}, ma_{r,i})$. A parametrização diagonal é adequada para corpos com pelo menos um plano de simetria geométrica, condição satisfeita pela grande maioria dos elos de UVMSs comerciais [1].

3.6.2 Matriz de Inércia com Massa Adicionada

A matriz de inércia generalizada no modo subaquático é:

$$M_{\text{Hydro}}(q) = \sum_{i=1}^N \left[J_{v,i}^\top \left(m_i I_3 + M_{A,\text{lin},i}^{(g)} \right) J_{v,i} + J_{\omega,i}^\top \left(I_i^{(g)} + M_{A,\text{rot},i}^{(g)} \right) J_{\omega,i} \right], \quad (3.19)$$

onde:

- $M_{\text{Hydro}}(q) \in \mathbb{R}^{N \times N}$ é a matriz de inércia generalizada no modo subaquático;
- m_i é a massa do elo i e I_3 é a matriz identidade 3×3 ;
- $M_{A,\text{lin},i}^{(g)} \in \mathbb{R}^{3 \times 3}$ é a matriz diagonal de massa adicionada translacional do elo i no referencial global;
- $M_{A,\text{rot},i}^{(g)} \in \mathbb{R}^{3 \times 3}$ é a matriz diagonal de massa adicionada rotacional do elo i no referencial global;
- $I_i^{(g)} \in \mathbb{R}^{3 \times 3}$ é o tensor de inércia rígida do elo i no referencial global (Equação (3.9));
- $J_{v,i}, J_{\omega,i} \in \mathbb{R}^{3 \times N}$ são os Jacobianos linear e angular do elo i (Equação (3.8)).

As matrizes de massa adicionada globais são obtidas pela transformação pelo referencial acumulado:

$$M_{A,\text{lin},i}^{(g)} = R_{\text{acc},i} M_{A,\text{lin},i}^{(l)} R_{\text{acc},i}^\top, \quad M_{A,\text{rot},i}^{(g)} = R_{\text{acc},i} M_{A,\text{rot},i}^{(l)} R_{\text{acc},i}^\top. \quad (3.20)$$

onde:

- $M_{A,\text{lin},i}^{(l)}, M_{A,\text{rot},i}^{(l)} \in \mathbb{R}^{3 \times 3}$ são as matrizes diagonais de massa adicionada translacional e rotacional no referencial local do elo i (Equação (3.18));
- $R_{\text{acc},i} \in \mathbb{R}^{3 \times 3}$ é a matriz de rotação global acumulada até o elo i ;
- a transformação $R(\cdot) R^\top$ realiza a mudança de base de tensores de segunda ordem do referencial local para o global.

A Equação (3.19) expande a Equação (3.7) pelo princípio de superposição da inércia virtual [1]: a massa adicionada $M_{A,i}$ representa a inércia do fluido acelerado pelo movimento do elo i , projetada no espaço de juntas via Jacobianos, exatamente como a inércia rígida do próprio elo. Essa equivalência formal permite a reutilização do mesmo *framework* de derivação de M para ambos os cenários (ar e água), com mínima duplicação de código.

3.6.3 Vetor de Potencial Aparente: Peso menos Empuxo

A energia potencial aparente de cada elo submerso combina o potencial gravitacional com o trabalho da força de empuxo:

$$V_i^{\text{app}}(q) = (m_i - \rho \text{vol}_i) g z_{\text{cm},i}(q), \quad (3.21)$$

onde:

- $V_i^{\text{app}}(q)$ é a energia potencial aparente (peso menos empuxo) do elo i ;
- m_i é a massa do elo i e ρvol_i é a massa de fluido deslocada pelo elo;
- g é a aceleração da gravidade;
- $z_{\text{cm},i}(q)$ é a coordenada vertical do CM do elo i ;
- conforme definido na Equação (2.11) do Capítulo 2.

O vetor $G_{\text{Hydro}}(q)$ resultante é o gradiente desta soma:

$$G_{\text{Hydro}}(q) = \left(\frac{\partial}{\partial q} \sum_{i=1}^N (m_i - \rho \text{vol}_i) g z_{\text{cm},i} \right)^\top. \quad (3.22)$$

onde:

- $G_{\text{Hydro}}(q) \in \mathbb{R}^N$ é o vetor de torques generalizados aparentes (gravidade menos empuxo);
- os demais símbolos seguem a Equação (3.21);
- $(\cdot)^\top$ converte o gradiente-linha em vetor-coluna.

Quando $m_i \approx \rho \text{vol}_i$ (elo neutralmente flutuante), $V_i^{\text{app}} \approx 0$ e o correspondente torque estático $G_i(q)$ anula-se, reproduzindo o cenário típico de UVMSs projetados para equilíbrio neutro que minimizam o esforço estático dos atuadores [19, 21].

3.6.4 Modelo Dinâmico Unificado

A estrutura formal das equações de movimento hidrodinâmicas é idêntica à terrestre:

$$M_{\text{Hydro}}(q)\ddot{q} + C_{\text{Hydro}}(q, \dot{q})\dot{q} + G_{\text{Hydro}}(q) = \tau, \quad (3.23)$$

onde:

- $M_{\text{Hydro}}(q) \in \mathbb{R}^{N \times N}$ é a matriz de inércia generalizada aparente (Equação (3.19));
- $C_{\text{Hydro}}(q, \dot{q}) \in \mathbb{R}^{N \times N}$ é a matriz de Coriolis e termos centrípetos calculada pelos mesmos símbolos de Christoffel sobre M_{Hydro} (Equação (3.14)), sem nenhuma modificação algorítmica;
- $G_{\text{Hydro}}(q) \in \mathbb{R}^N$ é o vetor de torques generalizados aparentes (Equação (3.22));
- $q, \dot{q}, \ddot{q} \in \mathbb{R}^N$ são os vetores de posições, velocidades e acelerações generalizadas;

- $\tau \in \mathbb{R}^N$ é o vetor de torques generalizados aplicados pelos atuadores.

Os termos de arrasto hidrodinâmico $\tau_{\text{drag}} = D_{\text{lin}}\dot{q} + D_{\text{quad}}|\dot{q}|\dot{q}$ (Equação (2.10) do Capítulo 2) são aplicados separadamente no integrador numérico como forças passivas, não na geração simbólica do modelo, pela razão de que sua forma $|\dot{q}_i|\dot{q}_i$ não é simbólica no sentido lagrangiano clássico [1]. Ilustra-se, na Figura 3.3, como **RobotMathHydro** estende **RobotMathEngine** por herança, sobrescrevendo os métodos de dinâmica para incorporar massa adicionada e potencial aparente Peso–Empuxo.

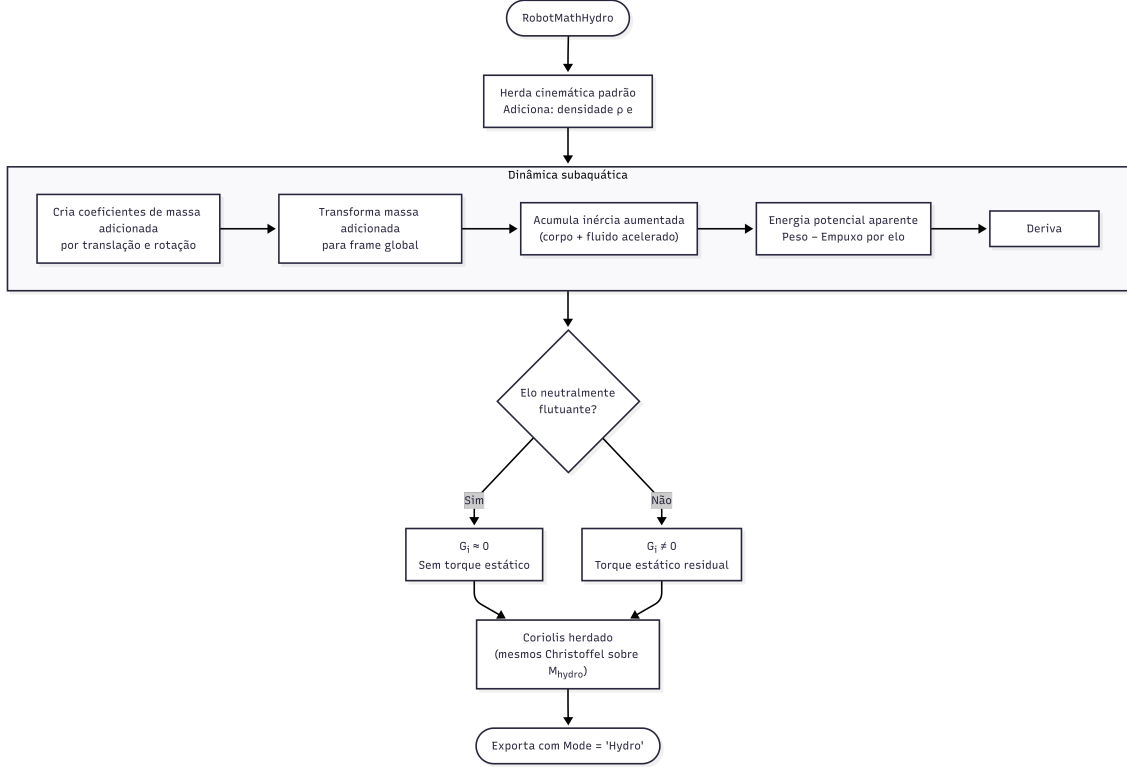


Figura 3.3: Extensão hidrodinâmica implementada em **RobotMathHydro**: herança de **RobotMathEngine** com sobrescrita dos métodos de cinemática e dinâmica para incorporar o tensor de massa adicionada $M_A^{(g)}$ (Equação (3.19)) e o potencial aparente Peso–Empuxo (Equação (3.21)) segundo a formulação de Fossen [1].

3.7 Preparação e Exportação do Modelo Simbólico

3.7.1 Substituição de Símbolos Dinâmicos

O método `step_4_prepare_export` realiza a substituição dos `dynamicsymbols` dependentes do tempo, $q_i(t)$ e $\dot{q}_i(t)$, por símbolos escalares estáticos q_i e $\dot{q}_i$, via o mapeamento:

$$q_i(t) \mapsto \text{sp.Symbol}(f'q\{i\}'), \quad \dot{q}_i(t) \mapsto \text{sp.Symbol}(f'dq\{i\}'). \quad (3.24)$$

onde:

- $q_i(t)$ é a função simbólica dependente do tempo, criada via `dynamicsymbols` do SymPy;
- $\dot{q}_i(t) = dq_i(t)/dt$ é sua derivada temporal simbólica;
- $\mapsto$ representa a substituição da função do tempo por um símbolo escalar livre;
- `sp.Symbol(·)` é o construtor de símbolos escalares estáticos do SymPy;
- $f'q\{i\}$ e $f'dq\{i\}$ são os nomes *formatados* das variáveis estáticas (e.g., `q1`, `dq1`, `q2`, ...).

Esse passo é necessário porque `dynamicsymbols` são, internamente, funções do tempo no SymPy, e sua presença nas expressões interfere com o `lambdify`, que espera símbolos escalares como variáveis de entrada [2]. A substituição não altera a estrutura algébrica das expressões; apenas troca a representação interna de “função aplicada” para “símbolo livre”.

3.7.2 Exportação do Dicionário de Resultados

O método retorna um dicionário com as seguintes chaves:

- "M": a matriz de inércia $M(q) \in \mathbb{R}^{N \times N}$, expressa nos símbolos estáticos;
- "G": o vetor de gradiente de potencial $G(q) \in \mathbb{R}^N$;
- "C": o vetor de Coriolis $C(q, \dot{q}) \in \mathbb{R}^N$ (já no formato $C(q, \dot{q})\dot{q}$);
- "J": o Jacobiano geométrico $J(q) \in \mathbb{R}^{6 \times N}$;
- "FK_Pos": a transformação homogênea completa do efetuador final $T_N(q)$;
- "FK_Vel": a velocidade cartesiana do efetuador $J(q)\dot{q}$;
- "Subs": o mapeamento de substituição (Equação (3.24));
- "Mode": string "Air" ou "Hydro", para controle de fluxo no simulador.

Essa interface de exportação desacopla completamente a *engine* matemática do módulo de simulação, permitindo que o modelo simbólico seja serializado (via `pickle`) e reutilizado em sessões posteriores sem recomputação, uma funcionalidade de produtividade crítica, dado que a geração do modelo para sistemas com $N \geq 6$ pode levar dezenas de minutos em *hardware* convencional [24, 43].

3.8 Compilação Simbólico-Numérica com `lambdify` e CSE

3.8.1 A Interface `lambdify`

A transição das expressões simbólicas para funções numéricas avaliáveis é o passo de maior impacto sobre o desempenho computacional da simulação. O SymPy fornece a função `sp.lambdify()` para esse propósito: dado um conjunto de variáveis simbólicas $\{v_1, \dots, v_k\}$ e uma expressão (ou matriz) simbólica $\mathcal{E}(v_1, \dots, v_k)$, `lambdify` gera o código-fonte Python/NumPy equivalente e compila-o via `exec`, retornando uma função numérica de alta performance [2].

No *Hephaestus*, a assinatura do `lambdify` é:

$$f_{M,C,G,J}(q_1, \dots, q_N, \dot{q}_1, \dots, \dot{q}_N, g, m_1, L_1, cx_1, \dots) \rightarrow \mathbb{R}^{\times}, \quad (3.25)$$

onde:

- $f_{M,C,G,J}$ representa cada uma das funções numéricas compiladas (uma para M , C , G e J);
- $q_1, \dots, q_N$ são as coordenadas generalizadas (entradas dinâmicas);
- $\dot{q}_1, \dots, \dot{q}_N$ são as velocidades generalizadas;
- g é a aceleração da gravidade;
- $m_1, L_1, cx_1, \dots$ representam os parâmetros físicos (massa, comprimento, coordenadas do CM, momentos de inércia etc.) listados em `params_list`;
- $\mathbb{R}^{\times}$ denota a saída numérica, cuja dimensão depende da grandeza avaliada (e.g., $N \times N$ para M , N para G , $6 \times N$ para J).

Essa assinatura corresponde à lista `sym_vars = bot.q + bot.dq + bot.params_list`, e garante que a mesma assinatura seja utilizada para todas as funções compiladas (M , C , G , J , FK), simplificando o gerenciamento de argumentos no *loop* de simulação.

3.8.2 Eliminação de Subexpressões Comuns

O parâmetro `cse=True` ativa a Eliminação de Subexpressões Comuns (*Common Subexpression Elimination*, CSE) antes da geração do código numérico [3]. Internamente, o SymPy aplica o algoritmo de Risch-Moses para CSE [44], identificando subárvores repetidas na expressão simbólica e substituindo cada ocorrência por uma variável temporária. O código Python gerado tem então a forma:

$$\begin{aligned} \mathbf{x0} &= \sin(q_1), \quad \mathbf{x1} = \cos(q_1), \quad \mathbf{x2} = \mathbf{x0} \cdot m_1, \quad \dots \\ M[0, 0] &= \mathbf{x2} \cdot L_1^2 + \dots, \quad M[0, 1] = \mathbf{x1} \cdot \mathbf{x2} + \dots \end{aligned} \quad (3.26)$$

onde:

- $\mathbf{x0}, \mathbf{x1}, \mathbf{x2}, \dots$ são variáveis temporárias geradas pelo CSE para subexpressões repetidas;
- $\sin(q_1), \cos(q_1)$ são funções trigonométricas elementares avaliadas uma única vez por passo;
- m_1, L_1 são parâmetros físicos do primeiro elo (massa e comprimento);
- $M[0, 0], M[0, 1]$ são elementos da matriz de inércia compilada, expressos em função das variáveis temporárias e dos parâmetros físicos.

O impacto quantitativo da CSE é substancial: para um manipulador de 6 GDL em ar, a expressão simbólica de $M(q)$ pode conter $O(10^3)$ a $O(10^4)$ operações primitivas; com CSE, a função compilada executa tipicamente 2–5× mais rápido, com redução de 60–90% no tamanho do *bytecode* [2, 3]. Para sistemas maiores ($N > 8$), a CSE pode ser a diferença entre uma compilação que produz código de tamanho manejável e uma que gera *strings* de vários MB que excedem os limites de compilação do Python (`MemoryError`) [43].

Adicionalmente, o módulo `modules='numpy'` instrui o `lambdify` a mapear funções simbólicas (`sp.sin`, `sp.cos`, etc.) para suas contrapartes NumPy (`np.sin`, `np.cos`), que por sua vez invocam implementações C compiladas via BLAS/LAPACK [2]. O resultado é uma função numérica que se comporta como código C nativo para as operações de ponto flutuante, sem o *overhead* da álgebra simbólica em tempo de simulação.

3.8.3 Compilação das Funções por Módulo

No construtor `RobotSimulator.__init__`, cada grandeza dinâmica é compilada individualmente, produzindo quatro funções distintas:

- `func_M`: $f_M : \mathbb{R}^{2N+P} \rightarrow \mathbb{R}^{N \times N}$, avalia a matriz de inércia;
- `func_C`: $f_C : \mathbb{R}^{2N+P} \rightarrow \mathbb{R}^N$, avalia o vetor de Coriolis;
- `func_G`: $f_G : \mathbb{R}^{2N+P} \rightarrow \mathbb{R}^N$, avalia o gradiente de potencial;
- `func_J`: $f_J : \mathbb{R}^{2N+P} \rightarrow \mathbb{R}^{6 \times N}$, avalia o Jacobiano completo;

- **funcs_fk_all_links**: lista de N funções $f_{FK,i} : \mathbb{R}^{2N+P} \rightarrow \mathbb{R}^3$, avaliando a posição 3D do extremo do elo i , utilizadas para a visualização gráfica animada do manipulador.

onde $P = |\text{params_list}|$ é o número total de parâmetros físicos. A compilação separada por grandeza, em vez de uma única função monolítica, permite o despacho paralelo de M , C e G durante a simulação: cada uma das funções pode ser submetida simultaneamente a um *worker* distinto do executor paralelo, triplicando a taxa de avaliação dinâmica em máquinas com pelo menos três núcleos [43]. Sintetiza-se, na Figura 3.4, o fluxo completo de compilação simbólico-numérica, desde as expressões SymPy geradas nas etapas anteriores até as funções NumPy prontas para avaliação a cada passo de integração.

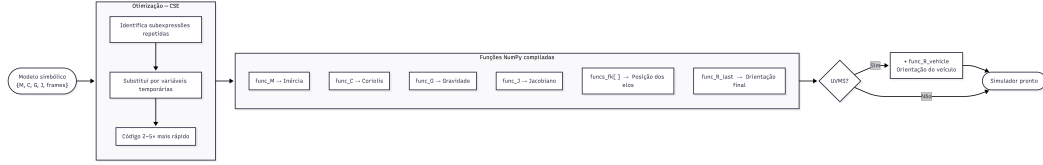


Figura 3.4: Fluxo de compilação simbólico-numérica via `sp.lambdify()` com CSE ativo: as expressões SymPy de M , C , G e J passam pela Eliminação de Subexpressões Comuns antes de gerar código Python/NumPy compilado, reduzindo em 60–90% o número de operações primitivas e viabilizando a avaliação em tempo de simulação [2, 3].

3.9 Paralelismo em Dois Níveis e Executor Persistente

O *Hephaestus* implementa paralelismo em dois níveis distintos, correspondentes às duas fases temporalmente separadas do fluxo de trabalho:

3.9.1 Nível 1: Paralelismo na Derivação Simbólica

Como descrito na Seção 3.5, o cálculo de $\partial M / \partial q_k$ e das linhas de C é distribuído entre múltiplos processos durante a fase de *geração do modelo* (executada uma única vez). O custo de *spawn* dos processos e da serialização das expressões simbólicas é amortizado pelo ganho de velocidade na derivação, sendo lucrativo quando $N \geq 4$ e $W \geq 2$ processos estão disponíveis [24, 45].

3.9.2 Nível 2: Paralelismo na Avaliação Numérica

Durante a fase de *simulação*, as funções `func_M`, `func_C` e `func_G` são avaliadas numericamente a cada passo de integração física (tipicamente $\Delta t = 10^{-3}$ s, correspondendo a 1000 avaliações por segundo de simulação). Para evitar o custo de

respawn de processos a cada avaliação , que, em testes, somava 2-8 segundos por simulação , o *Hephaestus* utiliza um **executor persistente** (`ProcessPoolExecutor` pré-inicializado):

- O executor é instanciado *uma única vez* após a compilação, por meio do método `warm_up_workers()`, e mantido ativo durante toda a vida do objeto `RobotSimulator`;
- Cada *worker* executa o *inicializador* `_init_worker`, que compila localmente as expressões simbólicas (via `lambdify`) e armazena as funções no dicionário de módulo `_WORKER_FUNCS`;
- A cada passo de simulação, três tarefas são submetidas ao executor em paralelo , `("M", args)`, `("C", args)`, `("G", args)` , e os resultados são coletados por `Future.result()`;
- O executor é encerrado explicitamente pelo método `close()`, chamado automaticamente no protocolo de fechamento da interface gráfica (`on_closing`).

A abordagem de executor persistente com compilação nos *workers* (em vez de serialização das funções compiladas, que o `pickle` não suporta) elimina o custo de *respawn* e reduz o *overhead* de IPC (*Inter-Process Communication*) a apenas a serialização dos argumentos numéricos (vetores de ponto flutuante), que é negligenciável [45].

Quando o paralelismo não está ativo (`use_parallel=False`), as três funções são avaliadas sequencialmente na *thread* principal, sem instanciar o executor , comportamento padrão seguro para qualquer ambiente de execução, incluindo executáveis PyInstaller. Ilustram-se, na Figura 3.5, os dois níveis de paralelismo, suas fronteiras temporais distintas e os papéis do `ProcessPoolExecutor` em cada fase do ciclo de vida do *Hephaestus*.

3.10 Considerações sobre a Complexidade Computacional

Resume-se, na Tabela 3.1, a complexidade computacional das etapas de modelagem e simulação do *Hephaestus*, comparada com as referências clássicas da literatura.

A comparação direta com o algoritmo de Newton-Euler recursivo, cuja complexidade é $O(N)$ tanto na geração quanto na avaliação [24, 28], evidencia o *trade-off* fundamental da abordagem simbólica: maior custo de derivação (amortizado na fase *offline*), em troca de equações explícitas que viabilizam análise, controle baseado em

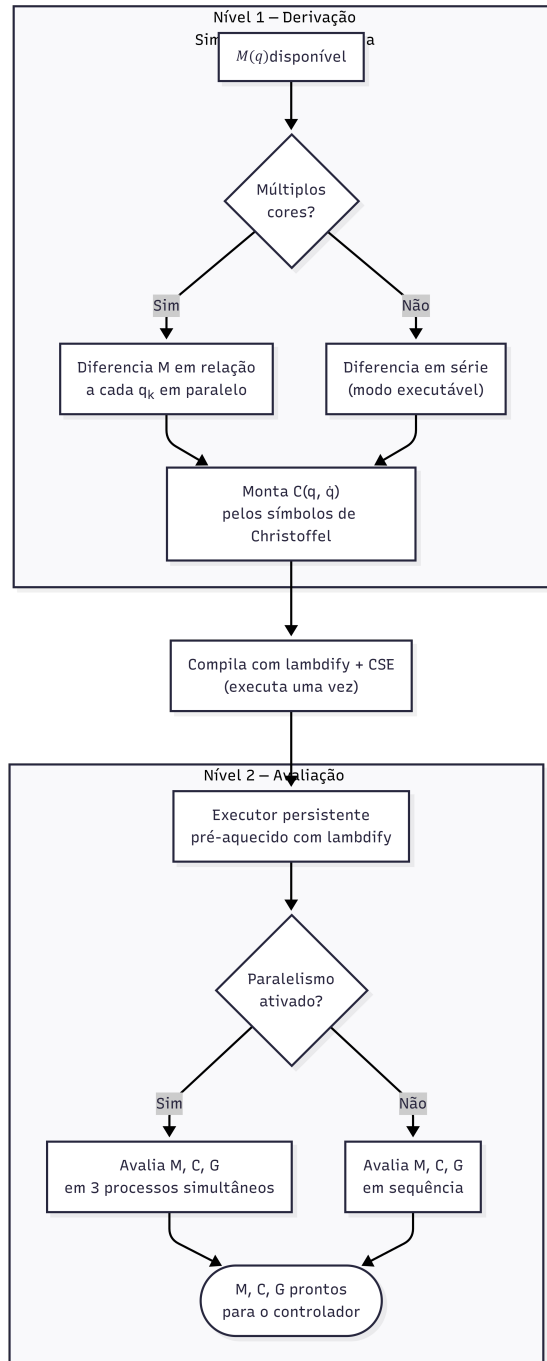


Figura 3.5: Paralelismo em dois níveis no *Hephaestus*: (Nível 1) derivação paralela de $\partial M/\partial q_k$ e das linhas de C por `ProcessPoolExecutor` durante a geração *offline* do modelo; (Nível 2) avaliação paralela e persistente de `func_M`, `func_C` e `func_G` a cada passo de integração física durante a simulação *online*.

Tabela 3.1: Complexidade computacional das etapas de modelagem e simulação.

Etapa	Complexidade	Referência / Observação
Cinemática FK simbólica	$O(N^2)$	Produto acumulado de matrizes 4×4 [5]
Derivação de $M(q)$ (simbólica)	$O(N^3)$	Jacobianos por elo [4]
Derivação de $\partial M / \partial q_k$	$O(N^4)$ (ingênuo)	Paralelizável por N processos [24]
Cálculo de C (Christoffel)	$O(N^3)$ simbólico	$N(N+1)/2$ pares por linha, N linhas [23]
Compilação (<code>lambdify</code> + CSE)	$O(S)$, S = tamanho expr.	Custo único; CSE reduz S em 60–90% [2, 3]
Avaliação numérica por passo	$O(N^2)$	Mult. matriz-vetor $M \setminus (\tau - C - G)$ [26]

modelo e transparência pedagógica. Esse *trade-off* é deliberado e justificado pelo objetivo do *Hephaestus* como plataforma didática e de desenvolvimento de controladores [50].

3.11 Síntese do Capítulo

Este capítulo descreveu em detalhe o processo completo de modelagem matemática simbólica implementado no *Hephaestus*. A partir de uma especificação compacta da cadeia cinemática (`joint_config` e `link_vectors_mask`), a *engine* deriva automaticamente as expressões fechadas de $M(q)$, $C(q, \dot{q})$, $G(q)$ e $J(q)$ pelo formalismo de Euler-Lagrange via Jacobianos, com extensão para o cenário hidrodinâmico por herança da classe `RobotMathHydro`. A geração simbólica é acelerada por paralelismo de processos na derivação de Christoffel, e as expressões resultantes são compiladas para funções NumPy de alto desempenho por `lambdify` com CSE, eliminando o gargalo de avaliação simbólica em tempo de simulação. O executor persistente completa a cadeia de otimização, permitindo avaliação paralela de M , C e G a cada passo de integração sem custo de *respawn* de processos.

A arquitetura resultante é matematicamente transparente, cada equação pode ser inspecionada simbolicamente, e computacionalmente eficiente, capaz de simular manipuladores de até 8 GDL em tempo real no *hardware* alvo. O Capítulo seguinte descreve a camada de simulação numérica que consome as equações aqui derivadas, detalhando o integrador simplético, os controladores implementados e o planejamento de trajetórias.

Capítulo 4

Simulação Numérica e Planejamento de Trajetórias

Este capítulo descreve a camada de simulação numérica do *Hephaestus*, implementada na classe `RobotSimulator` do módulo `simulator.py`. Enquanto o Capítulo 3 tratou da derivação simbólica e da compilação das equações de movimento, este capítulo aborda como essas equações são integradas numericamente em malha fechada para rastrear trajetórias cartesianas com controle dinâmico de alta fidelidade. As contribuições técnicas centrais deste capítulo compreendem: o planejamento de trajetórias com polinômio cúbico no espaço cartesiano (segmentos lineares e arcos circulares); a extensão do algoritmo CLIK para o controle no *espaço de tarefa 6D*, que reúne as três coordenadas de posição (x, y, z) e as três coordenadas de orientação ($roll, pitch, yaw$) do efetuador, ; a inicialização robusta de postura por Levenberg-Marquardt com busca em linha; o integrador simplético com *substeps* independentes; quatro estratégias de controle não-linear comparáveis em uma única interface; e modelos de forças dissipativas para atrito nas juntas e arrasto hidrodinâmico.

4.1 Arquitetura do Simulador

A classe `RobotSimulator` recebe como entrada um objeto `RobotMathEngine` (ou `RobotMathHydro`) previamente processado e realiza, em seu construtor, a compilação de todas as expressões simbólicas em funções numéricas. Conforme descrito na Seção 3.8 do Capítulo 3, essa compilação é realizada por `sp.lambdify()` com CSE ativo, produzindo as funções `func_M`, `func_C`, `func_G`, `func_J` e o vetor `funcs_fk_all_links`, que avalia a posição cartesiana de cada elo para a visualização animada [2].

A interface pública principal é o método `run()`, cuja assinatura unifica os pa-

râmetros de configuração da trajetória, do controlador, do integrador e das forças dissipativas em um único ponto de entrada. Internamente, o método `run()` organiza a simulação em quatro fases:

1. **Inicialização da Postura:** a posição cartesiana inicial P_i é passada ao Levenberg-Marquardt (`solve_ik_initial`) para convergir a uma configuração de juntas q_0 compatível antes do início da trajetória.
2. **Loop de Integração:** a cada passo visual Δt_{vis} , são executados N_s *substeps* físicos $\Delta t_{\text{phys}} = \Delta t_{\text{vis}}/N_s$, cada um compreendendo: planejamento de trajetória, CLIK, avaliação de M , C , G , cálculo do torque de controle, aplicação das forças dissipativas e integração simplética.
3. **Coleta de Resultados:** ao final de cada passo visual, os erros de junta $e = q_d - q$, os torques aplicados τ e os dados de animação (posições FK de todos os elos e matrizes de rotação) são armazenados.
4. **Retorno:** o método retorna as séries temporais $\{t_i, e_i, \tau_i\}$ e a lista de *frames* de animação, que são consumidos pela interface gráfica para plotagem e renderização interativa.

A detecção automática da fronteira veículo/braço, pela contagem das juntas iniciais sem vetor de elo estrutural nulo, permite que o simulador opere tanto em manipuladores terrestres puros quanto em UVMSs, compilando separadamente a matriz de rotação do *frame* do veículo (`func_R_vehicle`) para visualização da orientação do corpo base [1, 19]. As Figuras 4.1 e 4.2 detalham o fluxo interno do método `run()`: a fase de inicialização (Figura 4.1) e o passo de controle repetido a cada *substep* físico (Figura 4.2).

4.2 Planejamento de Trajetórias no Espaço Cartesiano

O planejamento de trajetórias define as referências de posição $P_{\text{ref}}(t)$, velocidade $\dot{P}_{\text{ref}}(t)$ e aceleração $\ddot{P}_{\text{ref}}(t)$ que o efetuador deve rastrear ao longo do tempo. Siciliano et al. [4] e Craig [5] distinguem o planejamento no espaço de juntas do planejamento no espaço cartesiano; este trabalho adota o segundo, em que as referências são geradas diretamente na posição do efetuador e convertidas para comandos de junta pelo CLIK a cada instante.

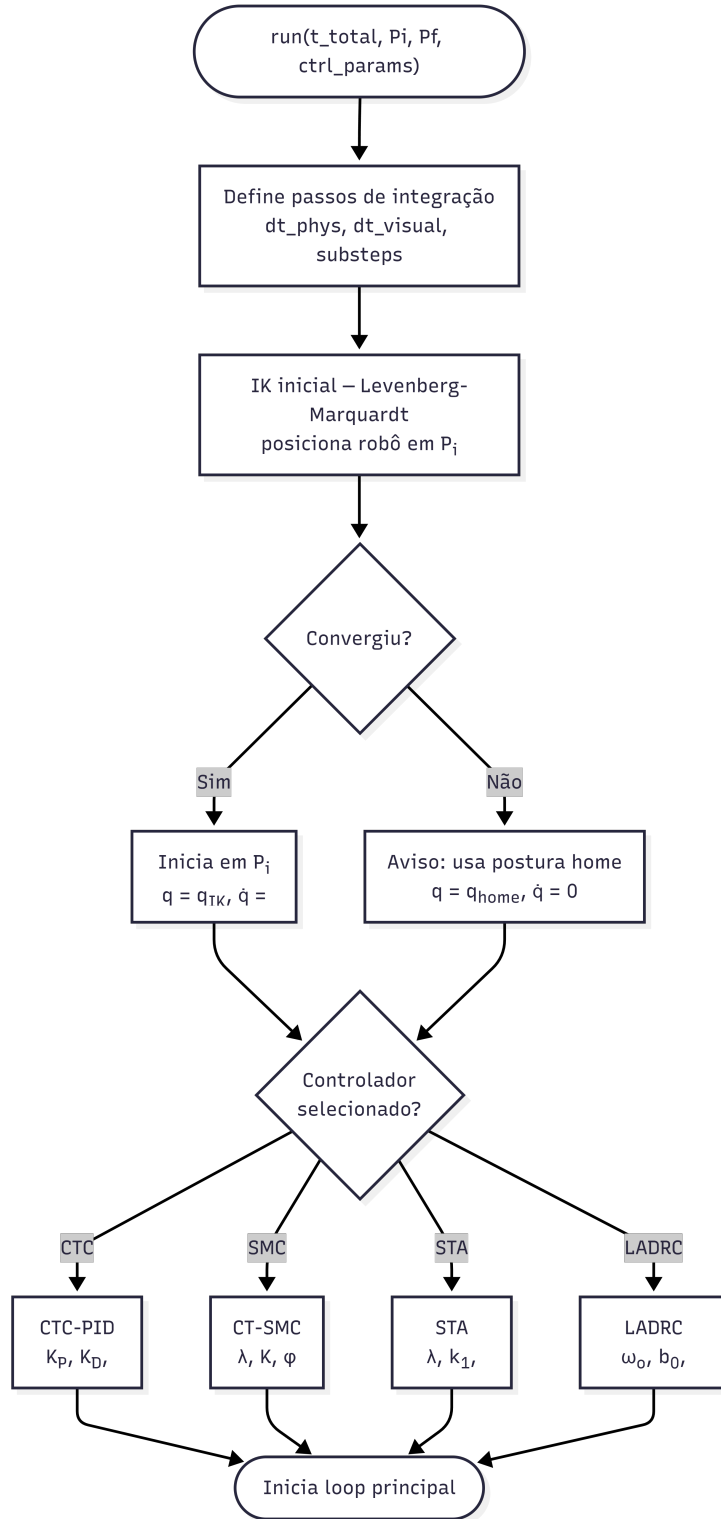


Figura 4.1: Fase de inicialização do simulador: resolução da IK estática por Levenberg-Marquardt com busca em linha (Seção 4.3), configuração do controlador e do executor paralelo, e verificação das condições de estabilidade numérica (Seção 4.9).



Figura 4.2: Passo de controle físico executado a cada *substep* Δt_{phys} : planejamento de trajetória, CLIK 6D, avaliação paralela de M , C , G , cálculo do torque de controle, aplicação das forças dissipativas e integração simplética de Euler (Equações (4.25)–(4.26)).

4.2.1 Polinômio Cúbico de Interpolação

Toda trajetória implementada no *Hephaestus* utiliza o parâmetro de tempo normalizado $\tau = t/T \in [0, 1]$, onde T é a duração total do segmento, interpolado pelo polinômio cúbico:

$$s(\tau) = 3\tau^2 - 2\tau^3, \quad (4.1)$$

onde:

- $s(\tau) \in [0, 1]$ é a variável de progresso normalizada, com $s(0) = 0$ e $s(1) = 1$;
- $\tau = t/T \in [0, 1]$ é o tempo normalizado;
- t é o tempo decorrido desde o início do segmento e T é a duração total do segmento.

com derivadas:

$$\dot{s} = \frac{6\tau - 6\tau^2}{T}, \quad \ddot{s} = \frac{6 - 12\tau}{T^2}. \quad (4.2)$$

onde:

- $\dot{s}$ é a derivada de s em relação ao tempo t (não em relação a τ);
- $\ddot{s}$ é a segunda derivada temporal de s ;
- T aparece nos denominadores em razão da regra da cadeia ($\frac{ds}{dt} = \frac{ds}{d\tau} \cdot \frac{1}{T}$).

Este perfil garante $s(0) = 0$, $s(1) = 1$, $\dot{s}(0) = \dot{s}(1) = 0$, ou seja, partida e parada suaves, e aceleração máxima $|\ddot{s}|_{\max} = 6/T^2$ no instante $T/2$ [4]. A continuidade da velocidade nos extremos é essencial para evitar descontinuidades no sinal de referência do CLIK que seriam amplificadas pelos ganhos derivativos do controlador [5]. O polinômio de grau 5 (que garantiria continuidade de aceleração nos extremos) não foi adotado por apresentar pico de aceleração levemente superior para o mesmo tempo de trajetória, conforme análise comparativa de Siciliano et al. [4].

4.2.2 Segmento de Reta

Para a trajetória linear de P_i a P_f , a referência cartesiana é:

$$P_{\text{ref}}(t) = P_i + (P_f - P_i) s(\tau), \quad (4.3)$$

$$\dot{P}_{\text{ref}}(t) = (P_f - P_i) \dot{s}(\tau), \quad (4.4)$$

$$\ddot{P}_{\text{ref}}(t) = (P_f - P_i) \ddot{s}(\tau). \quad (4.5)$$

onde:

- $P_{\text{ref}}(t), \dot{P}_{\text{ref}}(t), \ddot{P}_{\text{ref}}(t) \in \mathbb{R}^3$ são, respectivamente, a posição, velocidade e aceleração cartesianas de referência do efetuador no instante t ;
- $P_i, P_f \in \mathbb{R}^3$ são as posições inicial e final do segmento;
- $(P_f - P_i)$ é o vetor diretor do segmento (constante);
- $s(\tau), \dot{s}(\tau), \ddot{s}(\tau)$ são, respectivamente, o polinômio cúbico de progresso e suas duas primeiras derivadas temporais (Equações (4.1) e (4.2)).

A disponibilidade explícita de $\dot{P}_{\text{ref}}$ e $\ddot{P}_{\text{ref}}$ é fundamental para o *feedforward* de velocidade e aceleração no CLIK, que reduz o erro de rastreamento em regime dinâmico em comparação com o uso exclusivo de realimentação proporcional de posição [4, 7].

4.2.3 Arco Circular

A trajetória circular é parametrizada pelo centro C , dois vetores ortonormais e_1, e_2 no plano do arco, raio R , ângulo inicial θ_0 e ângulo final θ_f . O centro é determinado geometricamente como o ponto equidistante de P_i e P_f ao longo da direção perpendicular à corda $v = P_f - P_i$ no plano definido pelo vetor normal $\hat{n}$:

$$C = \frac{P_i + P_f}{2} \pm h \frac{(P_f - P_i) \times \hat{n}}{\|(P_f - P_i) \times \hat{n}\|}, \quad h = \sqrt{R^2 - \frac{\|P_f - P_i\|^2}{4}}, \quad (4.6)$$

onde:

- $C \in \mathbb{R}^3$ é o centro da circunferência de referência;
- $P_i, P_f \in \mathbb{R}^3$ são as posições inicial e final do arco;
- $\hat{n} \in \mathbb{R}^3$ é o vetor unitário normal ao plano do arco (fornecido pelo usuário);
- $R > 0$ é o raio do arco;
- h é a distância entre o ponto médio da corda $P_i P_f$ e o centro C , calculada pelo Teorema de Pitágoras;
- $\|\cdot\|$ denota a norma euclidiana e $\times$ denota o produto vetorial;
- o sinal $\pm$ é determinado pelo parâmetro de sentido de percurso (horário ou anti-horário).

A trajetória é então:

$$P(t) = C + R [\cos(\theta(t)) e_1 + \sin(\theta(t)) e_2], \quad \theta(t) = \theta_0 + (\theta_f - \theta_0) s(\tau), \quad (4.7)$$

onde:

- $P(t) \in \mathbb{R}^3$ é a posição cartesiana do efetuador ao longo do arco;
- C e R são o centro e o raio definidos na Equação (4.6);
- $e_1, e_2 \in \mathbb{R}^3$ são vetores unitários ortonormais que formam uma base do plano do arco;
- $\theta(t)$ é o ângulo varrido em função do tempo, sendo θ_0 e θ_f os ângulos inicial e final;
- $s(\tau)$ é o polinômio cúbico de progresso de Eq. (4.1), garantindo partida e parada suaves.

com velocidade e aceleração obtidas por diferenciação analítica:

$$\dot{P}(t) = R \dot{\theta} [-\sin(\theta) e_1 + \cos(\theta) e_2], \quad (4.8)$$

$$\ddot{P}(t) = R [\ddot{\theta} \hat{t}(\theta) + \dot{\theta}^2 \hat{n}(\theta)], \quad (4.9)$$

onde:

- $\dot{P}(t), \ddot{P}(t) \in \mathbb{R}^3$ são, respectivamente, a velocidade e a aceleração cartesianas do efetuador;
- $\dot{\theta}, \ddot{\theta}$ são as derivadas temporais primeira e segunda de $\theta(t)$, ou seja, $\dot{\theta} = (\theta_f - \theta_0) \dot{s}$ e $\ddot{\theta} = (\theta_f - \theta_0) \ddot{s}$;
- $\hat{t}(\theta) = -\sin(\theta) e_1 + \cos(\theta) e_2$ é o vetor unitário tangente ao arco;
- $\hat{n}(\theta) = -\cos(\theta) e_1 - \sin(\theta) e_2$ é o vetor unitário normal centrípeta (apontando para C);
- o termo $R \dot{\theta}^2 \hat{n}$ é a aceleração centrípeta, presente sempre que $\dot{\theta} \neq 0$.

A componente centrípeta $\dot{\theta}^2 \hat{n}$ da aceleração é frequentemente omitida em abordagens simplificadas, mas sua inclusão é necessária para que o *feedforward* de aceleração no CLIK seja fisicamente correto em trajetórias circulares de alta velocidade [4].

Uma verificação de segurança automática é aplicada: se a distância entre P_i e P_f excede o diâmetro $2R$, o raio é automaticamente estendido para $R = \|P_f - P_i\|/2 + \varepsilon$, garantindo geometria factível. Caso o vetor $(P_f - P_i) \times \hat{n}$ seja nulo (pontos colineares com a normal), a trajetória degrada graciosamente para um segmento de reta [4]. Apresenta-se, na Figura 4.3, o fluxo de geração das referências de posição, velocidade e aceleração para os dois tipos de trajetória implementados, bem como a integração com o polinômio cúbico.

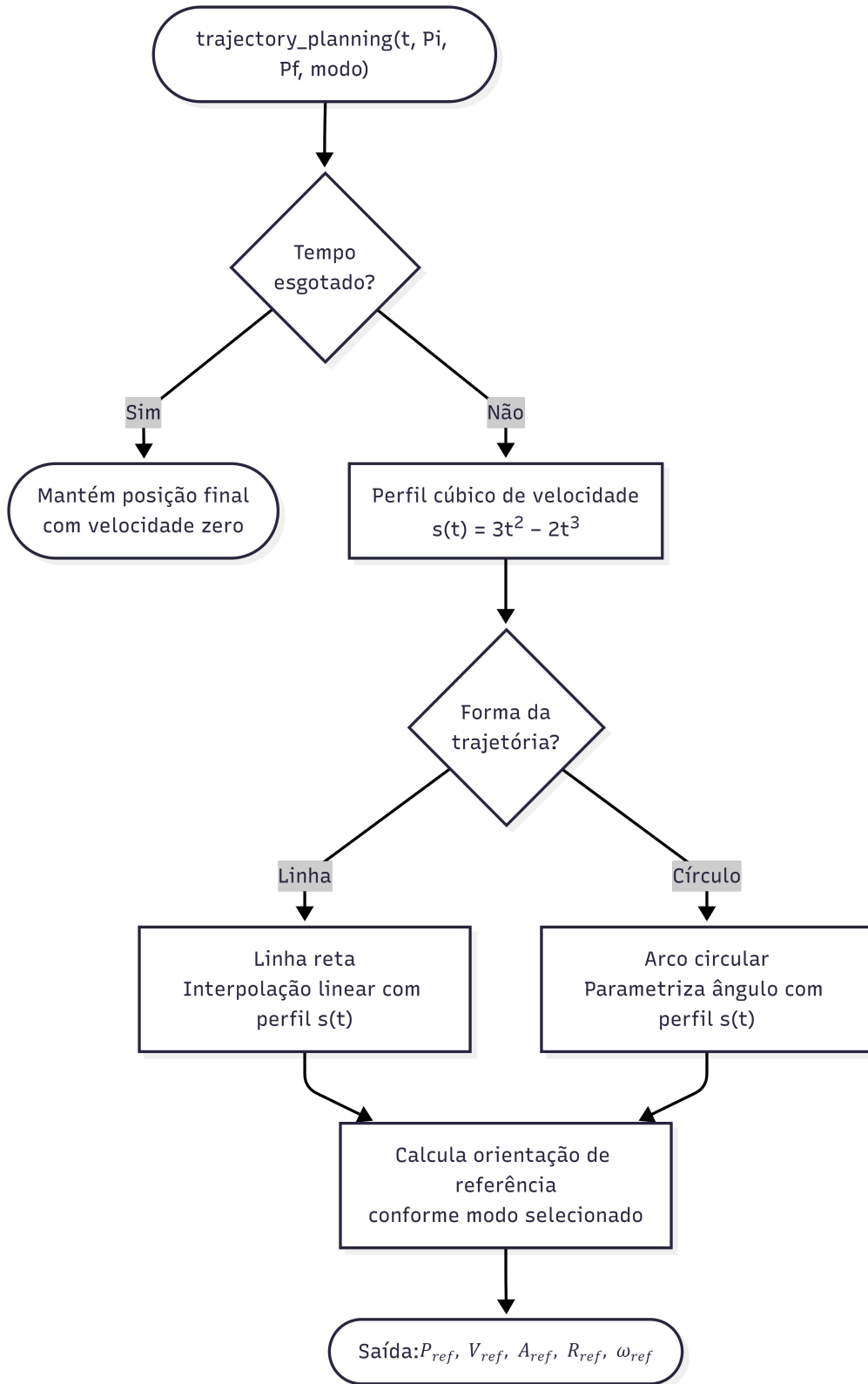


Figura 4.3: Fluxo de planejamento de trajetória no espaço cartesiano: o parâmetro cúbico $s(\tau)$ (Equação (4.1)) é aplicado ao segmento de reta (Equações (4.3)–(4.5)) ou ao arco circular (Equações (4.7)–(4.9)), gerando triplas $\{P_{ref}, \dot{P}_{ref}, \ddot{P}_{ref}\}$ a cada passo de integração para o CLIK [4, 5].

4.2.4 Modos de Referência de Orientação

O *Hephaestus* suporta seis modos distintos de referência de orientação do efetuador, selecionáveis independentemente do tipo de trajetória posicional:

1. **Livre** (*orientation-free*): nenhuma restrição de orientação é imposta; o CLIK opera apenas com o Jacobiano posicional $3 \times N$. Este modo é o padrão e adequado para tarefas de posicionamento puro.
2. **Fixa**: a orientação é mantida constante e igual à rotação R_0 capturada na postura inicial. Implementado como $R_{\text{ref}}(t) = R_0$, $\omega_{\text{ref}} = 0$. Útil para tarefas de transporte com efetuador estabilizado [4].
3. **Tangente à Trajetória**: o eixo z do efetuador é alinhado à tangente instantânea da trajetória:

$$\hat{z}(t) = \frac{\dot{P}_{\text{ref}}(t)}{\|\dot{P}_{\text{ref}}(t)\|}, \quad \omega_{\text{ref}} = \hat{z} \times \dot{\hat{z}}, \quad (4.10)$$

onde:

- $\hat{z}(t) \in \mathbb{R}^3$ é o vetor unitário ao longo do eixo z do efetuador, alinhado à tangente da trajetória;
- $\dot{P}_{\text{ref}}(t)$ é a velocidade cartesiana de referência (Equação (4.4));
- $\|\cdot\|$ denota a norma euclidiana, usada para normalizar $\dot{P}_{\text{ref}}$;
- $\dot{\hat{z}}$ é a derivada temporal de $\hat{z}$, obtida por diferenciação analítica;
- $\omega_{\text{ref}} \in \mathbb{R}^3$ é a velocidade angular de referência do efetuador, dada pelo produto vetorial entre $\hat{z}$ e $\dot{\hat{z}}$.

Empregado em tarefas de corte, pintura ou inspeção ao longo da trajetória [4].

4. **Apontar para o Alvo**: o eixo z do efetuador é continuamente orientado em direção ao ponto P_f :

$$\begin{aligned} \hat{z}(t) &= \frac{P_f - P_{\text{ref}}(t)}{\|P_f - P_{\text{ref}}(t)\|}, \\ \omega_{\text{ref}} &= \hat{z} \times \left[\frac{-\dot{P}_{\text{ref}}}{\|P_f - P_{\text{ref}}\|} - \frac{(P_f - P_{\text{ref}}) \cdot (-\dot{P}_{\text{ref}})}{\|P_f - P_{\text{ref}}\|^3} (P_f - P_{\text{ref}}) \right]. \end{aligned} \quad (4.11)$$

onde:

- $\hat{z}(t) \in \mathbb{R}^3$ é o vetor unitário do eixo z do efetuador apontando do ponto corrente $P_{\text{ref}}(t)$ para o alvo fixo P_f ;
- $P_f \in \mathbb{R}^3$ é a posição do alvo (constante);

- $P_{\text{ref}}(t), \dot{P}_{\text{ref}}(t)$ são, respectivamente, a posição e a velocidade de referência cartesianas do efetuador;
- $\omega_{\text{ref}} \in \mathbb{R}^3$ é a velocidade angular de referência, deduzida da derivada temporal analítica de $\hat{z}$ utilizando a regra do quociente;
- o termo entre colchetes é exatamente $\dot{\hat{z}}$, expressando a variação direcional decorrente do movimento de P_{ref} .

Adequado para tarefas de solda, furação ou interação com um alvo fixo [9].

5. **SLERP** (*Spherical Linear Interpolation*): interpola suavemente a orientação entre R_0 (postura inicial) e R_f (orientação final desejada) ao longo do geodésico em $SO(3)$:

$$R_{\text{ref}}(t) = R_0 \left(R_0^\top R_f \right)^{s(\tau)}, \quad \omega_{\text{ref}} = \dot{s} \theta_{\text{total}} R_0 \hat{a}_{\text{local}}, \quad (4.12)$$

onde:

- $R_{\text{ref}}(t) \in SO(3)$ é a matriz de rotação de referência interpolada;
- $R_0 \in SO(3)$ é a rotação inicial do efetuador;
- $R_f \in SO(3)$ é a rotação final desejada;
- $R_0^\top R_f \in SO(3)$ representa a rotação relativa total a ser aplicada;
- $(\cdot)^{s(\tau)}$ é a operação de potência matricial em $SO(3)$, que corresponde à interpolação SLERP ao longo do geodésico;
- $s(\tau) \in [0, 1]$ é o polinômio cúbico de progresso (Equação (4.1)) e $\dot{s}$ sua derivada temporal;
- θ_{total} é o ângulo total de rotação, extraído pela decomposição eixo-ângulo de $R_0^\top R_f$;
- $\hat{a}_{\text{local}} \in \mathbb{R}^3$ é o eixo unitário de rotação no referencial de R_0 ;
- $\omega_{\text{ref}} \in \mathbb{R}^3$ é a velocidade angular de referência expressa no referencial global, obtida ao mapear $\hat{a}_{\text{local}}$ para o referencial inercial via R_0 [6, 38].

A velocidade angular de referência ω_{ref} garante que o *feedforward* rotacional seja fisicamente correto, não apenas proporcional ao erro.

6. **Normal à Superfície**: o eixo z é mantido alinhado a um vetor normal fixo $\hat{n}$ fornecido pelo usuário, com $\omega_{\text{ref}} = 0$. Adequado para tarefas de usinagem ou inspeção perpendicular a superfícies planas [4].

Os modos 2 a 6 ativam o Jacobiano completo $6 \times N$ no CLIK, descrito na Seção 4.4.3, combinando controle de posição e orientação no mesmo algoritmo.

Comparam-se, na Figura 4.4, os seis modos de referência de orientação implementados, com indicação do sinal de referência gerado por cada um e dos casos de uso típicos.

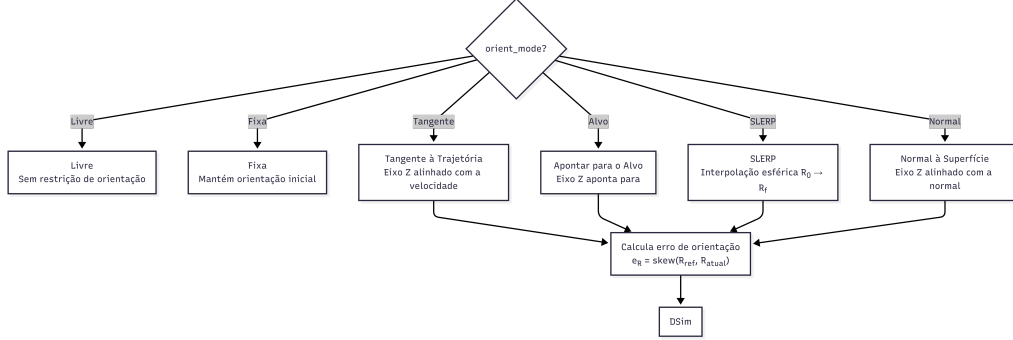


Figura 4.4: Modos de referência de orientação do efetuador disponíveis no *Hephaestus*: Livre (apenas controle posicional), Fixa, Tangente à Trajetória, Apontar para o Alvo, SLERP geodésico [6] e Normal à Superfície. Os modos 2–6 ativam o Jacobiano completo $J \in \mathbb{R}^{6 \times N}$ e o erro de orientação da Equação (4.19).

4.3 Inicialização da Postura: Levenberg-Marquardt com Busca em Linha

Antes do início da trajetória, o *Hephaestus* resolve o problema de IK estática para posicionar o efetuador em P_i a partir da postura inicial q_0 (postura *home* ou última postura convergida). Para garantir convergência robusta mesmo próximo de singularidades ou partindo de configurações distantes, é empregada uma variante do algoritmo de Levenberg-Marquardt [35, 36] com busca em linha por *backtracking* (regra de Armijo [37]).

O algoritmo iterativo é descrito a seguir. A cada iteração k :

1. Avalia a posição atual $p_k = f_{FK}(q_k)$ e o erro $e_k = P_i - p_k$.
2. Se $\|e_k\| < \varepsilon_{tol}$, termina com convergência.
3. Calcula o passo DLS:

$$\Delta q = J_{pos}^\top \left(J_{pos} J_{pos}^\top + \lambda_k^2 I_3 \right)^{-1} e_k, \quad (4.13)$$

onde:

- $\Delta q \in \mathbb{R}^N$ é o incremento de junta proposto pela iteração k ;
- $J_{pos} = J_{pos}(q_k) \in \mathbb{R}^{3 \times N}$ é o Jacobiano posicional avaliado na configuração atual;

- $J_{\text{pos}}^\top \in \mathbb{R}^{N \times 3}$ é a transposta do Jacobiano posicional;
 - $\lambda_k > 0$ é o parâmetro de amortecimento (*damping*) corrente, atualizado adaptativamente;
 - I_3 é a matriz identidade 3×3 ;
 - $e_k = P_i - p_k \in \mathbb{R}^3$ é o erro de posição cartesiano na iteração k ;
 - $\lambda_k^2 I_3$ amortece o sistema próximo a singularidades (*Damped Least Squares*, DLS).
4. Executa busca em linha por *backtracking*: tenta $q_{k+1} = q_k + \alpha \Delta q$ com $\alpha = 1, 0.5, 0.25, \dots$ até $\|P_i - f_{\text{FK}}(q_{k+1})\| < \|e_k\|$.
 5. Atualiza λ adaptativamente: $\lambda \leftarrow 0.9\lambda$ se houve descida (passo aceito), $\lambda \leftarrow 1.5\lambda$ se não houve (passo rejeitado), com limites $\lambda \in [10^{-4}, 10]$.
 6. Detecta estagnação: se $\|e_{k-1}\| - \|e_k\| < 0.1 \varepsilon_{\text{tol}}$ por 10 iterações consecutivas, interrompe.

A estratégia adaptativa de λ é a essência do algoritmo de Levenberg-Marquardt: para erros grandes (longe do mínimo), λ grande amortece o passo, comportando-se como o método do gradiente; próximo do mínimo, λ pequeno permite convergência quadrática típica de Newton-Gauss [36]. A combinação com a busca em linha de Armijo confere estabilidade adicional quando o Jacobiano é mal-condicionado, conforme demonstrado por Nakamura e Hanafusa [8] e Wampler [27] para IK de manipuladores com singularidades.

4.4 Cinemática Inversa em Malha Fechada (CLIK)

4.4.1 Formulação do CLIK Posicional (3D)

Durante a trajetória, a cada passo de integração, o CLIK (Closed-Loop Inverse Kinematics) [7, 33] converte a referência cartesiana $\{P_{\text{ref}}, \dot{P}_{\text{ref}}, \ddot{P}_{\text{ref}}\}$ em referências de junta $\{q_d, \dot{q}_d, \ddot{q}_d\}$. A lei de velocidade de junta no modo posicional é:

$$\dot{q}_d = J_{\text{DLS}}^\dagger \left[\dot{P}_{\text{ref}} + K_P e_{\text{pos}} \right] + (I - J_{\text{DLS}}^\dagger J_{\text{pos}}) \dot{q}_0, \quad (4.14)$$

onde:

- $\dot{q}_d \in \mathbb{R}^N$ é a velocidade de junta de referência produzida pelo CLIK;

- $J_{\text{DLS}}^\dagger \in \mathbb{R}^{N \times 3}$ é a pseudo-inversa amortecida do Jacobiano posicional (Equação (4.15));
- $J_{\text{pos}} \in \mathbb{R}^{3 \times N}$ é o Jacobiano posicional;
- $\dot{P}_{\text{ref}} \in \mathbb{R}^3$ é o termo de *feedforward* de velocidade cartesiana;
- $e_{\text{pos}} = P_{\text{ref}} - p_{\text{curr}} \in \mathbb{R}^3$ é o erro de posição cartesiano, com p_{curr} sendo a posição corrente do efetuador;
- $K_P = K_{p,\text{IK}} \cdot I_3$ é a matriz de ganho proporcional do CLIK posicional;
- $I \in \mathbb{R}^{N \times N}$ é a matriz identidade e $(I - J_{\text{DLS}}^\dagger J_{\text{pos}})$ é o operador de projeção no espaço nulo;
- $\dot{q}_0 \in \mathbb{R}^N$ é a velocidade auxiliar de junta para controle de redundância (Equação (4.22)).

A pseudo-inversa amortecida (DLS) $J_{\text{DLS}}^\dagger \in \mathbb{R}^{N \times 3}$ é [8, 27]:

$$J_{\text{DLS}}^\dagger = J_{\text{pos}}^\top \left(J_{\text{pos}} J_{\text{pos}}^\top + \lambda_{\text{DLS}}^2 I_3 \right)^{-1}. \quad (4.15)$$

onde:

- $J_{\text{DLS}}^\dagger \in \mathbb{R}^{N \times 3}$ é a pseudo-inversa amortecida (*Damped Least Squares*) do Jacobiano posicional;
- $J_{\text{pos}} \in \mathbb{R}^{3 \times N}$ é o Jacobiano posicional avaliado em q ;
- $\lambda_{\text{DLS}} > 0$ é o coeficiente de amortecimento, ajustado conforme a proximidade de singularidades;
- I_3 é a matriz identidade 3×3 ;
- o termo $\lambda_{\text{DLS}}^2 I_3$ regulariza o sistema, garantindo invertibilidade mesmo quando J_{pos} perde posto [8, 27].

A posição de junta de referência é obtida por integração de Euler explícita:

$$q_d(t + \Delta t) = q_{\text{curr}} + \dot{q}_d \Delta t, \quad (4.16)$$

onde:

- $q_d(t + \Delta t) \in \mathbb{R}^N$ é a posição de junta de referência no próximo instante;
- $q_{\text{curr}} \in \mathbb{R}^N$ é a posição corrente das juntas medida no instante t ;

- $\dot{q}_d \in \mathbb{R}^N$ é a velocidade de junta de referência calculada pela Equação (4.14);
- Δt é o passo de tempo da integração explícita de Euler.

O termo de aceleração de referência é calculado para *feedforward*:

$$\ddot{q}_d = J_{\text{DLS}}^\dagger \left[\ddot{P}_{\text{ref}} + K_D^{\text{IK}} \dot{e}_{\text{vel}} - \dot{J}\dot{q} \right], \quad (4.17)$$

onde:

- $\ddot{q}_d \in \mathbb{R}^N$ é a aceleração de junta de referência usada como *feedforward*;
- $J_{\text{DLS}}^\dagger$ é a pseudo-inversa amortecida do Jacobiano posicional (Equação (4.15));
- $\ddot{P}_{\text{ref}} \in \mathbb{R}^3$ é a aceleração cartesiana de referência (*feedforward*);
- $K_D^{\text{IK}} = 2\sqrt{K_P}$ é o ganho derivativo do CLIK, escolhido para amortecimento crítico da dinâmica de erro de velocidade;
- $\dot{e}_{\text{vel}} = \dot{P}_{\text{ref}} - J_{\text{pos}}\dot{q}$ é o erro de velocidade cartesiana;
- $\dot{J}\dot{q}$ é o termo de correção de deriva do Jacobiano, com $\dot{J}$ calculado por diferença finita de J_{pos} entre passos consecutivos [4].

4.4.2 Limitação de Velocidade de Junta

Para evitar descontinuidades e prevenir movimentos bruscos próximos de singularidades, a velocidade de junta total $\dot{q}_d$ (incluindo a contribuição do espaço nulo) é saturada suavemente pela função hiperbólica tangente:

$$\dot{q}_d^{\text{sat}} = \dot{q}_{\text{lim}} \tanh\left(\frac{\dot{q}_d}{\dot{q}_{\text{lim}}}\right), \quad (4.18)$$

onde:

- $\dot{q}_d^{\text{sat}} \in \mathbb{R}^N$ é a velocidade de junta saturada;
- $\dot{q}_d \in \mathbb{R}^N$ é a velocidade de junta calculada pelo CLIK (Equação (4.14));
- $\dot{q}_{\text{lim}} > 0$ é o limite de velocidade por junta (configurável pelo usuário);
- $\tanh(\cdot)$ é a função tangente hiperbólica aplicada componente a componente, que satura suavemente em $\pm\dot{q}_{\text{lim}}$ enquanto preserva a direção do vetor para valores pequenos.

O uso de $\tanh$ em vez de *clipping* linear garante diferenciabilidade e preserva a direção do vetor de velocidade para valores abaixo do limite, o que é importante para a qualidade do *feedforward* [4].

4.4.3 Controle de Orientação no Espaço de Tarefa 6D

O *espaço de tarefa 6D* é a representação completa da pose do efetuador, em que três coordenadas descrevem a sua **posição** cartesiana (x, y, z) e três coordenadas descrevem a sua **orientação** angular em $SO(3)$ (tipicamente *roll*, *pitch* e *yaw*). Em contraste, o CLIK posicional descrito na Seção anterior opera apenas no espaço 3D, restringindo-se à posição. A extensão para o espaço 6D é necessária sempre que a tarefa exija controlar simultaneamente *onde* o efetuador está e *como* ele está orientado, por exemplo, em soldagem ao longo de uma trajetória, em alinhamento de uma ferramenta com uma superfície ou no acoplamento de uma peça a um furo.

Quando um modo de orientação diferente de *Livre* é selecionado, o CLIK é estendido para o espaço de tarefa 6D, utilizando o Jacobiano completo $J \in \mathbb{R}^{6 \times N}$, que mapeia as N velocidades de junta para as seis componentes da *twist* cartesiana, sendo as três primeiras lineares (velocidades em x, y, z) e as três últimas angulares (em torno de x, y, z), e combinando o controle de posição e orientação em uma única lei de velocidade. O erro de orientação é calculado pela formulação de *anel* (*skew-symmetric*) [9]:

$$e_R = \frac{1}{2} \text{vee} \left(R_{\text{ref}} R_{\text{curr}}^\top - R_{\text{curr}} R_{\text{ref}}^\top \right), \quad (4.19)$$

onde:

- $e_R \in \mathbb{R}^3$ é o vetor de erro de orientação;
- $R_{\text{curr}} \in SO(3)$ é a rotação corrente do efetuador obtida por `func_R_last(q, ·)`;
- $R_{\text{ref}} \in SO(3)$ é a rotação de referência do modo de orientação selecionado (Seção 4.2.4);
- $R_{\text{ref}} R_{\text{curr}}^\top - R_{\text{curr}} R_{\text{ref}}^\top$ é a parte anti-simétrica que codifica o eixo-ângulo do erro;
- $\text{vee}(\cdot)$ é o operador que extrai o vetor $\mathbb{R}^3$ associado a uma matriz anti-simétrica 3×3 [9].

Esta formulação é proporcional ao vetor eixo-ângulo para erros pequenos e evita singularidades de representação por ângulos de Euler [4].

O vetor de velocidade de tarefa 6D é:

$$v_{\text{task}} = \begin{bmatrix} \dot{P}_{\text{ref}} + K_P e_{\text{pos}} \\ \omega_{\text{ref}} + K_P^{\text{or}} e_R \end{bmatrix} \in \mathbb{R}^6, \quad (4.20)$$

onde:

- $v_{\text{task}} \in \mathbb{R}^6$ é o vetor de velocidade de tarefa, com componentes lineares (3 primeiras) e angulares (3 últimas);

- $\dot{P}_{\text{ref}} \in \mathbb{R}^3$ é a velocidade cartesiana de referência (*feedforward*);
- $e_{\text{pos}} = P_{\text{ref}} - p_{\text{curr}} \in \mathbb{R}^3$ é o erro de posição;
- $K_P \in \mathbb{R}^{3 \times 3}$ é o ganho proporcional posicional;
- $\omega_{\text{ref}} \in \mathbb{R}^3$ é a velocidade angular de referência (*feedforward*) fornecida pelo modo de orientação;
- $e_R \in \mathbb{R}^3$ é o erro de orientação (Equação (4.19));
- $K_P^{\text{or}} \in \mathbb{R}^{3 \times 3}$ é o ganho proporcional de orientação.

A pseudo-inversa amortecida 6D é:

$$J_{6,\lambda}^\dagger = J^\top \left(J J^\top + \lambda_{\text{DLS}}^2 I_6 \right)^{-1} \in \mathbb{R}^{N \times 6}, \quad (4.21)$$

onde:

- $J_{6,\lambda}^\dagger \in \mathbb{R}^{N \times 6}$ é a pseudo-inversa amortecida do Jacobiano completo;
- $J \in \mathbb{R}^{6 \times N}$ é o Jacobiano geométrico completo (linear + angular);
- $\lambda_{\text{DLS}} > 0$ é o coeficiente de amortecimento;
- I_6 é a matriz identidade 6×6 .

A lei de velocidade de junta no modo 6D segue a mesma estrutura da Equação (4.14), com J_{pos} substituído por J e o vetor de erro por v_{task} [7, 9]. O espaço nulo agora depende de $J_{6,\lambda}^\dagger J \in \mathbb{R}^{N \times N}$, e a gestão de redundância via projeção da lei $K_n(q_{\text{home}} - q)$ permanece idêntica [34].

4.4.4 Gestão de Redundância via Espaço Nulo

Para sistemas com $N > 6$ graus de liberdade (ou $N > 3$ no modo posicional), o operador de projeção no espaço nulo $P_n = I - J_\lambda^\dagger J$ define um subespaço de velocidades de junta que não afetam o movimento do efetuador. O *Hephaestus* utiliza esse subespaço para atrair a configuração de junta à postura *home* preferida [9]:

$$\dot{q}_0 = K_n (q_{\text{home}} - q_{\text{curr}}), \quad (4.22)$$

onde:

- $\dot{q}_0 \in \mathbb{R}^N$ é a velocidade auxiliar de junta projetada no espaço nulo do Jacobiano;

- $K_n > 0$ é o ganho de espaço nulo (ajustável pelo usuário);
- $q_{\text{home}} \in \mathbb{R}^N$ é a configuração *home* (postura preferida) das juntas;
- $q_{\text{curr}} \in \mathbb{R}^N$ é a configuração corrente das juntas.

Esta abordagem, introduzida por Liegeois [34] e consolidada por Chiaverini et al. [9], garante que o robô tenda à postura de maior manipulabilidade ou com limites de junta mais distantes, melhorando a qualidade do rastreamento ao longo de trajetórias extensas. O fluxo completo do CLIK, incluindo DLS, *feedforward* de velocidade e aceleração, correção de deriva do Jacobiano e projeção no espaço nulo, é esquematizado na Figura 4.5.

4.5 Integrador Simplético de Euler com Substeps

4.5.1 Hierarquia de Passos de Tempo

O integrador numérico opera em dois níveis de passo de tempo, independentes entre si:

- Δt_{phys} : passo de integração física, tipicamente $\Delta t_{\text{phys}} = 10^{-3}$ s. Determina a fidelidade numérica da integração das equações de movimento.
- Δt_{vis} : passo de amostragem para visualização e armazenamento de resultados, tipicamente $\Delta t_{\text{vis}} = 5 \times 10^{-2}$ s. Determina a taxa de *frames* da animação.

O número de *substeps* por *frame* de visualização é:

$$N_s = \left\lceil \frac{\Delta t_{\text{vis}}}{\Delta t_{\text{phys}}} \right\rceil, \quad (4.23)$$

onde:

- $N_s \in \mathbb{N}$ é o número de *substeps* físicos executados a cada *frame* de visualização;
- Δt_{vis} é o passo de amostragem para visualização e armazenamento de resultados;
- Δt_{phys} é o passo da integração física;
- $\lceil \cdot \rceil$ é o operador teto (arredondamento para cima ao inteiro mais próximo).

O Δt_{vis} efetivo é ajustado para $N_s \cdot \Delta t_{\text{phys}}$, garantindo que o passo visual seja um múltiplo inteiro do físico. Essa separação permite aumentar a fidelidade numérica sem custo proporcional de armazenamento ou renderização: para $\Delta t_{\text{vis}} = 0,05$ s e $\Delta t_{\text{phys}} = 10^{-3}$ s, obtêm-se 50 *substeps* por *frame*, com apenas um *frame* armazenado por cada 50 avaliações de dinâmica [46].

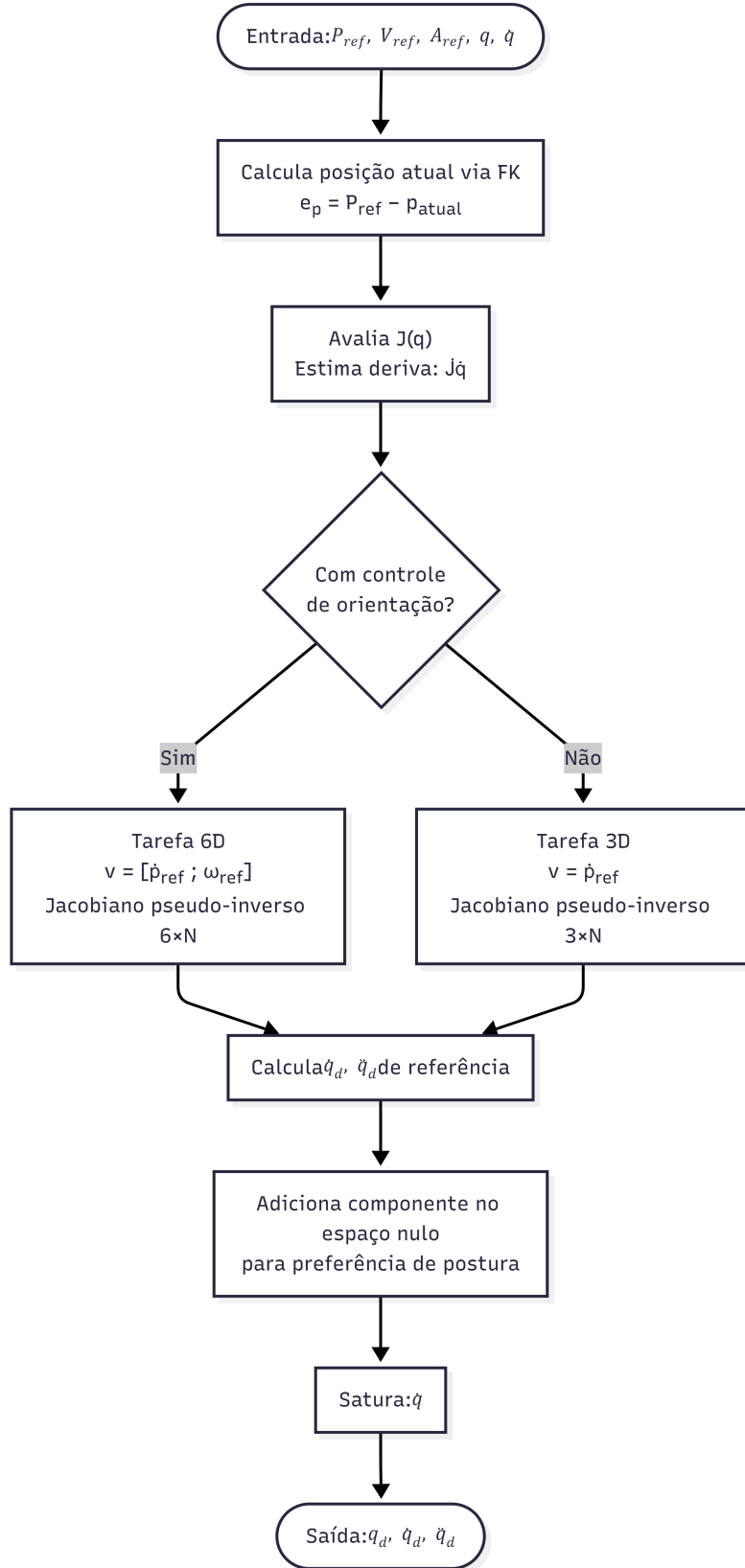


Figura 4.5: Fluxo do algoritmo CLIK (*Closed-Loop Inverse Kinematics*) implementado no *Hephaestus*: recebe as referências cartesianas $\{P_{ref}, \dot{P}_{ref}, \ddot{P}_{ref}\}$ e produz as referências de junta $\{q_d, \dot{q}_d, \ddot{q}_d\}$ por pseudo-inversão amortecida DLS (Equação (4.15)) com gestão de redundância por projeção no espaço nulo [7–9].

4.5.2 Integrador Simplético de Euler

A integração das equações de movimento (2.4) (Capítulo 2) é realizada pelo método de Euler Simplético de primeira ordem. A cada *substep* físico, a dinâmica direta é resolvida por:

$$\ddot{q}^n = M^{-1}(q^n) \left[\tau_{\text{ctrl}}^n + \tau_{\text{dist}}^n - C^n - G^n - \tau_{\text{pass}}^n \right], \quad (4.24)$$

onde:

- $\ddot{q}^n \in \mathbb{R}^N$ é o vetor de aceleração de junta no instante t_n ;
- $M(q^n) \in \mathbb{R}^{N \times N}$ é a matriz de inércia generalizada avaliada em q^n ;
- $\tau_{\text{ctrl}}^n \in \mathbb{R}^N$ é o torque produzido pelo controlador ativo;
- $\tau_{\text{dist}}^n \in \mathbb{R}^N$ é o torque de distúrbio externo (opcional, p.ex. para testes de robustez);
- $C^n = C(q^n, \dot{q}^n) \dot{q}^n$ é o vetor de Coriolis e centrífugas avaliado no estado atual;
- $G^n = G(q^n)$ é o vetor de gravidade (e empuxo, no modo *Hydro*);
- $\tau_{\text{pass}}^n \in \mathbb{R}^N$ reúne as forças passivas (atrito + arrasto), descritas na Seção 4.7.

A inversão de M é realizada por `np.linalg.solve`, que utiliza decomposição LU , numericamente mais estável e eficiente que o cálculo explícito da inversa [26].

A atualização do estado segue a sequência simplética:

$$\dot{q}^{n+1} = \dot{q}^n + \ddot{q}^n \Delta t_{\text{phys}}, \quad (4.25)$$

$$q^{n+1} = q^n + \dot{q}^{n+1} \Delta t_{\text{phys}}. \quad (4.26)$$

onde:

- $q^n, \dot{q}^n, \ddot{q}^n \in \mathbb{R}^N$ são, respectivamente, a posição, velocidade e aceleração de junta no instante t_n ;
- $q^{n+1}, \dot{q}^{n+1} \in \mathbb{R}^N$ são a posição e velocidade no instante $t_{n+1} = t_n + \Delta t_{\text{phys}}$;
- Δt_{phys} é o passo de integração física;
- observe que q^{n+1} é atualizado utilizando $\dot{q}^{n+1}$ (já corrigido), o que confere a propriedade simplética ao integrador.

A ordem em que velocidade e posição são atualizadas , velocidade *antes* da posição , distingue o integrador simplético do Euler explícito padrão. Enquanto o

Euler explícito acumula energia seculamente (instabilidade numérica suave), o Euler Simplético preserva uma “energia modificada” próxima da hamiltoniana real do sistema, garantindo estabilidade de longo prazo [46, 47]. Para simulações de controle em malha fechada com duração moderada (tipicamente < 30 s), essa diferença é crítica para a fidelidade dos torques calculados e da resposta do controlador.

Após cada atualização, as coordenadas de juntas rotacionais são restritas ao intervalo $(-\pi, \pi]$ pelo operador de envolvimento (*wrap*):

$$q_i \leftarrow (q_i + \pi) \bmod 2\pi - \pi, \quad \forall i \text{ tal que a junta } i \text{ é rotacional}, \quad (4.27)$$

onde:

- q_i é a coordenada generalizada da i -ésima junta rotacional;
- $\bmod$ denota o operador resto da divisão (módulo);
- a operação completa mapeia qualquer ângulo real para o intervalo principal $(-\pi, \pi]$;
- o operador atribuição $\leftarrow$ indica que q_i é sobrescrito *in-place*.

Esta operação é aplicada apenas às juntas rotacionais (não às prismáticas), assegurando que os erros de junta $e_i = q_{d,i} - q_i$ representem sempre o menor caminho angular, condição necessária para a estabilidade de controladores com ganho proporcional [23]. Uma verificação de finitude (`np.isfinite`) é aplicada a q e $\dot{q}$ após cada *substep*; caso valores NaN ou Inf sejam detectados, a simulação é interrompida com uma exceção descritiva, prevenindo propagação silenciosa de instabilidades numéricas.

4.6 Estratégias de Controle Não-Linear

Quatro estratégias de controle não-linear são implementadas no *Hephaestus*, todas recebendo como entrada as referências de junta $\{q_d, \dot{q}_d, \ddot{q}_d\}$ geradas pelo CLIK e as matrizes dinâmicas avaliadas numericamente. Uma única interface unificada, o parâmetro `ctrl_params` do método `run()`, seleciona e configura o controlador ativo, permitindo a comparação direta das estratégias sob condições idênticas de trajetória, modelo e perturbações.

4.6.1 Controle por Torque Computado com PID (CTC-PID)

O Controle por Torque Computado com PID [5, 23] é a estratégia padrão do simulador. A lei de controle é:

$$\tau_{\text{CTC-PID}} = M(q) \left[\ddot{q}_d + K_D \dot{e} + K_P e + K_I \int e dt \right] + C(q, \dot{q}) + G(q), \quad (4.28)$$

onde:

- $\tau_{\text{CTC-PID}} \in \mathbb{R}^N$ é o vetor de torques de controle aplicados nas juntas;
- $M(q) \in \mathbb{R}^{N \times N}$ é a matriz de inércia generalizada;
- $\ddot{q}_d \in \mathbb{R}^N$ é a aceleração de junta de referência (*feedforward*) gerada pelo CLIK (Equação (4.17));
- $e = q_d - q \in \mathbb{R}^N$ é o erro de posição de junta, $\dot{e} = \dot{q}_d - \dot{q}$ é o erro de velocidade e $\int e dt$ é o erro integral;
- $K_P = k_p I_N$ é a matriz de ganho proporcional;
- $K_D = 2\zeta \sqrt{k_p} I_N$ é a matriz de ganho derivativo, com $\zeta = 1$ correspondendo a amortecimento crítico;
- $K_I = k_i I_N$ é a matriz de ganho integral;
- $C(q, \dot{q}) \dot{q}$ é o vetor de Coriolis e centrífugas (compensação de não-linearidades);
- $G(q)$ é o vetor de gravidade (compensação de gravidade).

O ganho derivativo é derivado automaticamente do proporcional pelo método do polinômio característico de segunda ordem, garantindo amortecimento crítico para qualquer valor de k_p [10].

O integrador é sujeito a *anti-windup* por *clamping*:

$$\int e dt \leftarrow \text{clip} \left(\int e dt + e \Delta t, -e_{\max}, e_{\max} \right), \quad (4.29)$$

onde:

- $\int e dt \in \mathbb{R}^N$ é o estado integrador acumulado, atualizado a cada passo;
- $e \in \mathbb{R}^N$ é o erro de junta no instante atual;
- Δt é o passo de integração;

- $\text{clip}(x, a, b) = \max(\min(x, b), a)$ é a função de saturação aplicada componente a componente;
- $e_{\max}$ é o limite simétrico de *windup* configurável, que previne a acumulação do integrador durante períodos de saturação do atuador [10];
- o operador $\leftarrow$ indica atribuição (atualização *in-place*).

Com modelo dinâmico exato, a lei CTC cancela a não-linearidade e a dinâmica de erro satisfaz $\ddot{e} + K_D \dot{e} + K_P e = 0$, que é um sistema linear estável de segunda ordem [5].

4.6.2 Controle por Modo Deslizante (CT-SMC)

O Controle por Modo Deslizante [11] define a superfície deslizante:

$$S = \dot{e} + \lambda e, \quad e = q - q_d, \quad (4.30)$$

onde:

- $S \in \mathbb{R}^N$ é o vetor de superfície deslizante (uma componente por junta);
- $e = q - q_d \in \mathbb{R}^N$ é o erro de posição de junta (note a convenção $q - q_d$, oposta ao CTC);
- $\dot{e} \in \mathbb{R}^N$ é o erro de velocidade;
- $\lambda > 0$ é a constante de tempo da superfície deslizante (define a dinâmica desejada do erro em modo deslizante).

A lei de controle baseada em modelo é:

$$\tau_{\text{SMC}} = M(q) [\ddot{q}_{d,f} - \lambda \dot{e} - K \tanh(S/\phi)] + C(q, \dot{q}) + G(q), \quad (4.31)$$

onde:

- $\tau_{\text{SMC}} \in \mathbb{R}^N$ é o torque de controle SMC aplicado nas juntas;
- $M(q), C(q, \dot{q}), G(q)$ são as matrizes/vetores dinâmicos definidos anteriormente;
- $\ddot{q}_{d,f} = \alpha \ddot{q}_d + (1 - \alpha) \ddot{q}_{d,f}^{\text{prev}}$ é a aceleração de referência filtrada por filtro passa-baixas, com $\alpha \in (0, 1]$ sendo o coeficiente de suavização e $\ddot{q}_{d,f}^{\text{prev}}$ o valor filtrado no passo anterior;

- $K > 0$ é o ganho do termo descontínuo, dimensionado para superar a magnitude do distúrbio;
- $\phi > 0$ é a espessura da camada limite (*boundary layer*) que regulariza o sinal $\text{sign}(S)$;
- $\tanh(S/\phi)$ é aplicado componente a componente, substituindo $\text{sign}(S)$ para mitigar o *chattering*.

O filtro de $\ddot{q}_{d,f}$ atenua picos de aceleração oriundos das derivadas do CLIK, que seriam amplificados pelo ganho λ na lei de controle, uma instabilidade prática frequentemente não mencionada nas formulações teóricas do CT-SMC [11, 40].

A função $\tanh(S/\phi)$ substitui o sinal descontínuo $\text{sign}(S)$ pela camada limite de espessura $\phi > 0$, eliminando o *chattering* (oscilações de alta frequência) ao custo de uma robustez assintótica dentro da camada [11]. A análise formal da robustez com camada limite foi conduzida por Burton e Zinober [12]: para perturbações $|d| \leq D$, o erro de regime permanente é limitado por $\|S\|_\infty \leq \phi \cdot D/K$, o que permite ao projetista estabelecer um compromisso explícito entre *chattering* e precisão de rastreamento.

4.6.3 Super-Twisting (STA)

O algoritmo Super-Twisting (STA), proposto por Levant [13], é um modo deslizante de segunda ordem que elimina o *chattering* intrinsecamente ao produzir um sinal de controle contínuo. A superfície deslizante é idêntica à Equação (4.30), e a parcela de modo deslizante é:

$$v_{\text{sw}} = -k_1 |S|^{1/2} \text{sign}(S) + z, \quad (4.32)$$

$$\dot{z} = -k_2 \text{sign}(S), \quad (4.33)$$

onde:

- $v_{\text{sw}} \in \mathbb{R}^N$ é o sinal de chaveamento contínuo do STA (parcela de modo deslizante);
- $S \in \mathbb{R}^N$ é a superfície deslizante (Equação (4.30));
- $|S|^{1/2}$ é a raiz quadrada do módulo de S aplicada componente a componente;
- $\text{sign}(S)$ é a função sinal aplicada componente a componente;
- $k_1, k_2 > 0$ são os ganhos do STA, ajustados conforme a magnitude do distúrbio;

- $z \in \mathbb{R}^N$ é a variável interna que integra o sinal descontínuo, produzindo v_{sw} contínuo;
- $\dot{z}$ é a derivada temporal de z , integrada numericamente por Euler explícito:
 $z^{n+1} = z^n - k_2 \text{sign}(S^n) \Delta t$.

A lei de controle completa é:

$$\tau_{\text{STA}} = M(q) [\ddot{q}_{d,f} - \lambda \dot{e} + v_{\text{sw}}] + C(q, \dot{q}) + G(q). \quad (4.34)$$

onde:

- $\tau_{\text{STA}} \in \mathbb{R}^N$ é o torque de controle Super-Twisting;
- $M(q), C(q, \dot{q}), G(q)$ são as matrizes/vetores dinâmicos definidos anteriormente;
- $\ddot{q}_{d,f}$ é a aceleração de referência filtrada (idêntica à do CT-SMC);
- $\lambda > 0$ é a constante de tempo da superfície deslizante;
- $\dot{e}$ é o erro de velocidade de junta;
- v_{sw} é o sinal contínuo gerado pelas Equações (4.32)–(4.33).

O sinal v_{sw} é contínuo porque z integra o sinal descontínuo, fazendo v_{sw} variar continuamente mesmo que $\text{sign}(S)$ seja descontínuo. Moreno e Osorio [14] forneceram uma função de Lyapunov estrita para o STA, estabelecendo condições de ganho suficientes para convergência em tempo finito com distúrbio de derivada limitada $|\dot{d}| \leq \delta$: $k_2 > \delta$ e $k_1 > 2\sqrt{k_2}\delta$. A convergência em tempo finito de $S \rightarrow 0$ e $\dot{S} \rightarrow 0$ diferencia qualitativamente o STA do CT-SMC de primeira ordem, cuja convergência é assintótica [13, 41].

O filtro de $\ddot{q}_{d,f}$ é compartilhado com a implementação do CT-SMC, sendo parametrizado pelo mesmo coeficiente α , o que garante comparabilidade direta das duas estratégias quando λ , K (CT-SMC) e k_1 , k_2 (STA) são ajustados para desempenhos nominais equivalentes.

4.6.4 Controle por Rejeição Ativa de Distúrbios Linear (LADRC)

O LADRC (Linear Active Disturbance Rejection Control), proposto por Han [42] e formalizado em versão linear por Gao [15], é implementado de forma descentralizada, um ESO e uma lei de controle por junta, ignorando o acoplamento dinâmico

[18]. Para a junta i , a dinâmica nominal é reescrita como:

$$\ddot{q}_i = b_{0,i} \tau_i + f_i, \quad f_i = \text{distúrbio total}, \quad (4.35)$$

onde:

- $\ddot{q}_i$ é a aceleração da junta i ;
- τ_i é o torque aplicado à junta i ;
- $b_{0,i} \approx 1/M_{ii}(q_0)$ é o ganho nominal de canal, estimado automaticamente a partir da diagonal da matriz de inércia na postura inicial q_0 (opção `auto_b0` da configuração);
- f_i é o distúrbio total, que engloba acoplamentos dinâmicos, erros de modelo e perturbações externas.

O ESO de terceira ordem (linear) é integrado por Euler explícito a cada passo Δt_{phys} :

$$z_1^{n+1} = z_1^n + \Delta t [z_2^n - \beta_1(z_1^n - q_i^n)], \quad (4.36)$$

$$z_2^{n+1} = z_2^n + \Delta t [z_3^n - \beta_2(z_1^n - q_i^n) + b_0 \tau_i^{n-1}], \quad (4.37)$$

$$z_3^{n+1} = z_3^n - \Delta t \beta_3(z_1^n - q_i^n), \quad (4.38)$$

onde:

- z_1^n, z_2^n, z_3^n são, respectivamente, as estimativas discretas de q_i , $\dot{q}_i$ e f_i no instante t_n ;
- $z_1^{n+1}, z_2^{n+1}, z_3^{n+1}$ são as estimativas no instante t_{n+1} ;
- q_i^n é a posição medida da junta i no instante t_n ;
- τ_i^{n-1} é o torque aplicado no passo anterior (uso intencional do valor passado para causalidade discreta);
- $b_0 = b_{0,i}$ é o ganho nominal de canal;
- $\Delta t = \Delta t_{\text{phys}}$ é o passo de integração;
- $\beta_1 = 3\omega_o$, $\beta_2 = 3\omega_o^2$, $\beta_3 = \omega_o^3$ são os ganhos do observador, parametrizados pela frequência do observador $\omega_o > 0$ [15].

A frequência ω_o é automaticamente limitada por $\omega_o \leq \omega_{\text{max}}/\Delta t$ para garantir estabilidade numérica do ESO discreto: a condição $\omega_o \Delta t \leq 0.1$ é aplicada como heurística de projeto [16].

A estimativa z_3^n do distúrbio total é suavizada por um filtro IIR de primeira ordem antes de ser usada na lei de controle:

$$z_3^{\text{filt},n} = \alpha_{\text{filt}} z_3^n + (1 - \alpha_{\text{filt}}) z_3^{\text{filt},n-1}, \quad (4.39)$$

onde:

- $z_3^{\text{filt},n}$ é a estimativa de distúrbio filtrada no instante t_n ;
- z_3^n é a estimativa bruta produzida pelo ESO (Equação (4.38));
- $z_3^{\text{filt},n-1}$ é o valor filtrado no passo anterior;
- $\alpha_{\text{filt}} \in (0, 1]$ é o coeficiente do filtro IIR de primeira ordem ($\alpha_{\text{filt}} = 1$ desativa o filtro; valores menores aumentam a suavização ao custo de *lag*).

Este filtro é necessário em prática para atenuar o conteúdo de alta frequência que o ESO amplifica nos ciclos iniciais, antes de convergir ao valor real de f_i [17].

A lei de controle cancela o distúrbio estimado e acrescenta *feedforward* explícito de G e C para reduzir o distúrbio residual que o ESO precisa estimar:

$$\tau_i^{\text{LADRC}} = \frac{u_{0,i} - z_{3,i}^{\text{filt}}}{b_{0,i}} + G_i + C_i, \quad (4.40)$$

onde:

- τ_i^{LADRC} é o torque de controle LADRC para a junta i ;
- $u_{0,i}$ é o sinal de controle nominal (PD com *feedforward*, definido na Equação (4.41));
- $z_{3,i}^{\text{filt}}$ é a estimativa filtrada do distúrbio total da junta i (Equação (4.39));
- $b_{0,i}$ é o ganho nominal de canal estimado;
- G_i, C_i são as componentes da junta i dos vetores de gravidade e Coriolis (*feedforward* explícito de modelo, reduzindo o distúrbio residual a estimar).

$$u_{0,i} = \ddot{q}_{d,i} + k_{p,i}(q_{d,i} - q_i) + k_{d,i}(\dot{q}_{d,i} - \dot{q}_i). \quad (4.41)$$

onde:

- $u_{0,i}$ é o sinal de controle nominal da junta i ;
- $\ddot{q}_{d,i}, \dot{q}_{d,i}, q_{d,i}$ são as referências de aceleração, velocidade e posição da junta i (geradas pelo CLIK);

- $q_i, \dot{q}_i$ são a posição e velocidade medidas da junta;
- $k_{p,i} > 0$ é o ganho proporcional e $k_{d,i} > 0$ é o ganho derivativo, ambos por junta.

O *feedforward* de G e C é especialmente crítico em manipuladores terrestres (onde $G \neq 0$) e no início de trajetórias rápidas (onde $C \propto \dot{q}^2$ cresce rapidamente): sem esses termos, o ESO deve perseguir um distúrbio crescente com *lag* dinâmico, introduzindo um transiente oscilatório nos primeiros instantes da trajetória. A inclusão do *feedforward* reduz o distúrbio residual a apenas erros de modelo e perturbações externas, sinais pequenos e quase constantes que o ESO estima com precisão [17]. A análise de estabilidade do LADRC com esta formulação foi formalizada por Zheng e Gao [16].

4.7 Modelos de Forças Dissipativas

4.7.1 Atrito nas Juntas

O modelo de atrito nas juntas combina os componentes de Coulomb (atrito estático/cinético) e viscoso (proporcional à velocidade):

$$\tau_{\text{fric},i} = F_{c,i} \tanh\left(\frac{\dot{q}_i}{\varepsilon}\right) + B_{v,i} \dot{q}_i, \quad (4.42)$$

onde:

- $\tau_{\text{fric},i}$ é o torque de atrito atuando na junta i (sentido oposto ao movimento);
- $\dot{q}_i$ é a velocidade da junta i ;
- $F_{c,i} \geq 0$ é o coeficiente de atrito de Coulomb (estático/cinético) da junta i ;
- $B_{v,i} \geq 0$ é o coeficiente de atrito viscoso da junta i ;
- $\varepsilon > 0$ é a largura da zona de regularização suave da função sinal (*regularization zone*);
- $\tanh(\dot{q}_i/\varepsilon)$ aproxima $\text{sign}(\dot{q}_i)$ de modo diferenciável, evitando descontinuidades numéricas próximas de $\dot{q}_i = 0$.

O uso de $\tanh$ em vez de $\text{sign}(\dot{q})$ é essencial para evitar descontinuidades que causariam instabilidade numérica no integrador: para $|\dot{q}_i| \gg \varepsilon$, $\tanh(\dot{q}_i/\varepsilon) \approx \text{sign}(\dot{q}_i)$, recuperando o comportamento de Coulomb clássico; para $|\dot{q}_i| \approx 0$, a transição é suave e impede o fenômeno de *stiction* numérico [11]. O valor padrão $\varepsilon = 0.05$ rad/s é calibrado para suavizar sem mascarar a zona de atrito estático em velocidades de junta típicas de manipuladores de laboratório.

4.7.2 Arrasto Hidrodinâmico

No modo *Hydro*, um modelo simplificado de arrasto no espaço de juntas é aplicado como força passiva [1, 19]:

$$\tau_{\text{drag},i} = D_{\text{lin},i} \dot{q}_i + D_{\text{quad},i} |\dot{q}_i| \dot{q}_i, \quad (4.43)$$

onde:

- $\tau_{\text{drag},i}$ é o torque de arrasto hidrodinâmico atuando na junta i ;
- $\dot{q}_i$ é a velocidade da junta i ;
- $D_{\text{lin},i} \geq 0$ é o coeficiente de arrasto linear (regime laminar) da junta i ;
- $D_{\text{quad},i} \geq 0$ é o coeficiente de arrasto quadrático (regime turbulento) da junta i ;
- $|\dot{q}_i| \dot{q}_i$ preserva o sinal correto da velocidade no termo quadrático, garantindo que o arrasto se oponha sempre ao movimento.

Esta formulação é equivalente à Equação (2.10) do Capítulo 2, parametrizada no espaço de juntas em vez do espaço cartesiano para compatibilidade com o integrador [19]. O arrasto quadrático captura os efeitos de pressão (regime turbulento, número de Reynolds elevado) que dominam em velocidades de junta mais altas, enquanto o linear captura o arrasto viscoso do escoamento laminar em baixas velocidades [1, 32].

Ambas as forças dissipativas são aplicadas na equação de dinâmica direta como torques passivos externos:

$$\ddot{q}^n = M^{-1} \left(\tau_{\text{ctrl}}^n - C^n - G^n - \tau_{\text{fric}}^n - \tau_{\text{drag}}^n \right), \quad (4.44)$$

onde:

- $\ddot{q}^n \in \mathbb{R}^N$ é a aceleração de junta no instante t_n ;
- $M^{-1} = M^{-1}(q^n)$ é a inversa da matriz de inércia (na prática, a equação é resolvida por `np.linalg.solve` usando decomposição LU);
- $\tau_{\text{ctrl}}^n \in \mathbb{R}^N$ é o torque do controlador ativo (CTC, SMC, STA ou LADRC);
- $C^n = C(q^n, \dot{q}^n) \dot{q}^n$ é o vetor de Coriolis e centrífugas;
- $G^n = G(q^n)$ é o vetor de gravidade (e empuxo, no modo *Hydro*);
- τ_{fric}^n é o vetor de atritos das juntas (Equação (4.42));
- τ_{drag}^n é o vetor de arrasto hidrodinâmico das juntas (Equação (4.43)).

Os torques dissipativos τ_{fric}^n e τ_{drag}^n são aplicados fora do laço do controlador. Essa separação é fisicamente correta: os torques dissipativos são forças do ambiente (não cancelas pelo controlador a priori) e devem ser observados ou estimados para serem compensados, papel que o ESO do LADRC desempenha implicitamente [17, 18].

4.8 Paralelismo na Avaliação Numérica

A avaliação numérica das funções f_M , f_C e f_G a cada passo de integração é o principal gargalo de tempo de execução para sistemas com $N \geq 5$ GDL, onde as expressões compiladas possuem centenas de operações trigonométricas e matriciais. Para mitigar esse custo, o *Hephaestus* suporta avaliação paralela via um executor persistente de processos.

O executor `ProcessPoolExecutor` é inicializado uma única vez após a compilação (método `warm_up_workers()`), com cada *worker* executando o inicializador `_init_worker` que compila localmente as três funções via `lambdify` e as armazena no dicionário de módulo `_WORKER_FUNCS`. A cada passo físico, três tarefas são submetidas em paralelo:

$$\text{futures} = \left\{ \begin{array}{l} M : \text{executor.submit}(\text{_eval_worker}, ("M", \text{args})), \\ C : \text{executor.submit}(\text{_eval_worker}, ("C", \text{args})), \\ G : \text{executor.submit}(\text{_eval_worker}, ("G", \text{args})) \end{array} \right\}, \quad (4.45)$$

onde:

- `futures` é um dicionário Python contendo os *handles* de `concurrent.futures.Future` para os três cálculos paralelos;
- `executor.submit(...)` é o método de `ProcessPoolExecutor` que despacha uma tarefa para um *worker* disponível e retorna um `Future`;
- `_eval_worker` é a função executada nos *workers*, que avalia a função simbólica solicitada (“M”, “C” ou “G”) sobre os argumentos numéricos;
- `args` é a tupla de argumentos numéricos $(q, \dot{q}, \text{params})$ avaliada no instante atual;
- os resultados são coletados por `Future.result()` após o despacho das três tarefas, aproveitando a sobreposição de computação [45].

A chave da eficiência é a **persistência** do executor: ao contrário de uma implementação ingênua que instancia e destrói um *pool* por simulação (custo de 2–8 s por instância, conforme medido nos testes), o executor é instanciado uma única vez

e reutilizado em todas as chamadas subsequentes a `run()`. A detecção de mudança no número de *workers* solicita reinicialização automática do executor, garantindo flexibilidade sem custo desnecessário [43].

Quando o paralelismo não está habilitado (`use_parallel=False`), as três funções são avaliadas sequencialmente na *thread* principal, comportamento padrão seguro para executáveis PyInstaller, máquinas com um único núcleo ou ambientes de depuração. Esta opção também é preferida para sistemas com $N \leq 4$ GDL, onde o *overhead* de IPC supera o ganho de paralelismo [45].

4.9 Verificação de Estabilidade Numérica

A estabilidade numérica do integrador simplético é governada pela relação entre o passo de integração Δt_{phys} e as frequências naturais do sistema em malha fechada. Para o CTC, a dinâmica de erro é linear de segunda ordem com frequência natural $\omega_n = \sqrt{k_p}$; a condição de estabilidade de Euler simplético exige [46]:

$$\Delta t_{\text{phys}} < \frac{2}{\omega_n} = \frac{2}{\sqrt{k_p}}, \quad (4.46)$$

onde:

- Δt_{phys} é o passo da integração física;
- $\omega_n = \sqrt{k_p}$ é a frequência natural da dinâmica de erro em malha fechada do CTC;
- k_p é o ganho proporcional do CTC-PID (Equação (4.28)).

A condição estabelece que o passo de integração deve ser inferior à metade do período de oscilação natural, ou seja, ao menos dois passos por período de oscilação de malha fechada. Para $k_p = 100 \text{ rad}^2/\text{s}^2$ (ganho típico), $\omega_n = 10 \text{ rad/s}$ e $\Delta t_{\text{phys}} < 0.2 \text{ s}$, condição satisfeita com folga para $\Delta t_{\text{phys}} = 10^{-3} \text{ s}$ [47]. Avisos automáticos são emitidos quando $\Delta t_{\text{phys}} > 10^{-2} \text{ s}$ ou $\Delta t_{\text{vis}} > 0.1 \text{ s}$, alertando o usuário sobre potencial instabilidade.

Para o LADRC, a condição adicional de estabilidade do ESO discreto é $\omega_o \Delta t_{\text{phys}} \leq 0.1$, correspondente à frequência de observador máxima $\omega_o^{\text{max}} = 0.1/\Delta t_{\text{phys}} = 100 \text{ rad/s}$ para $\Delta t_{\text{phys}} = 10^{-3} \text{ s}$. Este limite é imposto automaticamente pela verificação `max_wo_dt` no construtor do LADRC [16].

4.10 Síntese do Capítulo

Este capítulo descreveu a arquitetura completa de simulação numérica do *Hephaestus*. O fluxo de execução do método `run()` integra, em uma única interface unificada, as seguintes contribuições:

- **Planejamento de trajetória** com polinômio cúbico, suportando segmentos lineares e arcos circulares com aceleração *feedforward* analítica e seis modos de referência de orientação, incluindo SLERP geodésico [6] e controle tangencial.
- **Inicialização robusta** por Levenberg-Marquardt com busca em linha adaptativa [35, 37], garantindo partida a partir de qualquer configuração factível.
- **CLIK estendido ao espaço 6D**, com *feedforward* de velocidade e aceleração, correção de deriva do Jacobiano, tratamento de singularidades por DLS [8] e gestão de redundância via espaço nulo [9, 34].
- **Integrador simplético de Euler** com *substeps* independentes da taxa de visualização, garantindo estabilidade de energia a longo prazo e alta fidelidade numérica [46, 47].
- **Quatro controladores não-lineares** comparáveis: CTC-PID [5, 10], CT-SMC com camada limite [11, 12], Super-Twisting STA com convergência em tempo finito [13, 14] e LADRC descentralizado com ESO de terceira ordem [15, 16, 18].
- **Forças dissipativas** de atrito nas juntas (Coulomb + viscoso, suavizado por $\tanh$) e arrasto hidrodinâmico (linear + quadrático) [1, 19], aplicadas separadamente do controle para máxima fidelidade física.
- **Executor paralelo persistente** para avaliação paralela de M , C , G a cada *substep*, com custo de *spawn* amortizado sobre toda a simulação [43, 45].

O Capítulo seguinte apresenta os resultados experimentais obtidos com a plataforma *Hephaestus* em dois cenários de validação: um manipulador serial de 3 GDL em ambiente terrestre e um UVMS de 6 GDL em ambiente subaquático, comparando as quatro estratégias de controle em termos de erro de rastreamento, torques requeridos e resposta a perturbações externas.

Capítulo 5

Controladores Implementados e Integração com o Modelo

Este capítulo aprofunda a análise dos quatro controladores não-lineares implementados no *Hephaestus* sob a perspectiva da integração com o modelo dinâmico simbólico derivado no Capítulo 3. Enquanto o Capítulo 4 descreveu a arquitetura de simulação e apresentou as leis de controle de forma operacional, este capítulo concentra-se em três aspectos complementares: (i) a fundamentação teórica aprofundada de cada estratégia, com provas ou condições formais de estabilidade; (ii) a análise da integração específica entre cada controlador e as matrizes $M(q)$, $C(q, \dot{q})$, $G(q)$ geradas simbolicamente; e (iii) uma comparação sistemática das quatro estratégias em termos de robustez a incertezas paramétricas, sensibilidade a perturbações externas e compromissos de ajuste de ganhos.

5.1 Estrutura Unificada de Controle em Malha Fechada

A arquitetura de controle do *Hephaestus* segue o paradigma de *controle baseado em modelo* (*model-based control*), no qual o conhecimento das equações de movimento do robô é explorado explicitamente na lei de controle para cancelar ou compensar a não-linearidade intrínseca da dinâmica [4, 5, 23]. O fluxo de sinais é organizado em três camadas sobrepostas.

A interface de seleção de controlador é estabelecida pelo parâmetro `ctrl_params` do método `run()`, cujo campo `"type"` determina qual classe de controlador é instanciada. Essa interface unificada permite a comparação direta das quatro estratégias sob condições rigorosamente idênticas de trajetória, modelo, perturbações e passo de integração, requisito metodológico fundamental para estudos comparativos em robótica de controle [10, 50].

A disponibilidade das matrizes $M(q)$, $C(q, \dot{q})$ e $G(q)$ como funções numéricas compiladas (Seção 3.8 do Capítulo 3) é o que torna o controle baseado em modelo computacionalmente viável em tempo real. Sem a compilação via `lambdify` com CSE, a avaliação simbólica de M a cada passo de integração seria proibitivamente lenta, na ordem de segundos por avaliação, contra microsegundos com a função compilada. Essa ponte entre a computação simbólica e numérica é o elemento técnico central que diferencia o *Hephaestus* de simuladores que adotam modelos numéricos fixos [43].

5.2 Controle por Torque Computado com PID

5.2.1 Fundamentação Teórica e Cancelamento de Não-Linearidade

O Controle por Torque Computado (CTC), também denominado *computed torque control* ou *feedback linearization* [51, 52], é a estratégia de referência do *Hephaestus* e o ponto de partida para a compreensão das demais. A lei de controle fundamenta-se no cancelamento algébrico da dinâmica não-linear do robô pela inversão explícita do modelo:

$$\tau_{\text{CTC}} = M(q)v + C(q, \dot{q}) + G(q), \quad (5.1)$$

onde:

- $\tau_{\text{CTC}} \in \mathbb{R}^N$ é o vetor de torques de controle aplicado às juntas;
- $M(q) \in \mathbb{R}^{N \times N}$ é a matriz de inércia generalizada;
- $C(q, \dot{q}) \in \mathbb{R}^N$ é o vetor de Coriolis e centrífugas;
- $G(q) \in \mathbb{R}^N$ é o vetor de gravidade;
- $v \in \mathbb{R}^N$ é o sinal de controle auxiliar linear (a ser projetado, e.g. lei PID);
- $q, \dot{q} \in \mathbb{R}^N$ são as posições e velocidades de junta correntes.

Substituindo a Equação (5.1) na equação de movimento $M\ddot{q} + C\dot{q} + G = \tau$ e assumindo modelo exato, obtém-se a dinâmica de malha fechada linearizada:

$$\ddot{q} = v, \quad (5.2)$$

onde:

- $\ddot{q} \in \mathbb{R}^N$ é a aceleração resultante das juntas;

- $v \in \mathbb{R}^N$ é o sinal auxiliar linear injetado na lei de controle (Equação (5.1)).

Esta relação revela que o sistema em malha fechada é um conjunto de integradores duplos desacoplados e independentes da configuração q . A não-linearidade e o acoplamento entre juntas são completamente eliminados por *feedback linearization* exata, desde que $M(q)$, $C(q, \dot{q})$ e $G(q)$ sejam conhecidos exatamente [23, 51].

Com a escolha do sinal auxiliar PID:

$$v = \ddot{q}_d + K_D \dot{e} + K_P e + K_I \int_0^t e \, d\tau, \quad (5.3)$$

onde:

- $v \in \mathbb{R}^N$ é o sinal auxiliar PID;
- $\ddot{q}_d \in \mathbb{R}^N$ é a aceleração de junta de referência (*feedforward*);
- $e = q_d - q \in \mathbb{R}^N$ é o erro de posição de junta;
- $\dot{e} = \dot{q}_d - \dot{q}$ é o erro de velocidade;
- $\int_0^t e \, d\tau \in \mathbb{R}^N$ é o erro integral acumulado;
- $K_P, K_D, K_I \in \mathbb{R}^{N \times N}$ são as matrizes de ganhos proporcional, derivativo e integral, respectivamente.

A dinâmica de erro satisfaz:

$$\ddot{e} + K_D \dot{e} + K_P e + K_I \int_0^t e \, d\tau = 0. \quad (5.4)$$

onde:

- $\ddot{e}, \dot{e}, e \in \mathbb{R}^N$ são, respectivamente, o erro de aceleração, velocidade e posição de junta;
- K_P, K_D, K_I são as matrizes de ganhos definidas anteriormente;
- esta equação descreve um sistema linear desacoplado de terceira ordem cuja estabilidade é determinada pelos autovalores das matrizes K_P, K_D, K_I .

Esta é a equação de um sistema linear de terceira ordem com coeficientes matriciais diagonais constantes. Sua estabilidade é garantida pelas condições de Routh-Hurwitz sobre os ganhos $\{K_P, K_D, K_I\}$ [10, 22]: para $K_I = 0$ (modo PD), a condição é simplesmente $K_P > 0$ e $K_D > 0$. A escolha $K_D = 2\zeta\sqrt{K_P}$ com $\zeta = 1$ impõe amortecimento crítico ao sistema de segunda ordem resultante, minimizando o *overshoot* enquanto maximiza a velocidade de convergência para $K_I = 0$ [10].

5.2.2 Integração com o Modelo Simbólico-Numérico

No *Hephaestus*, os termos $M(q)$, $C(q, \dot{q})$ e $G(q)$ são avaliados numericamente pelas funções compiladas `func_M`, `func_C` e `func_G` (Seção 3.8 do Capítulo 3), com os argumentos construídos pelo método `_build_args(q, dq)`. Essa avaliação ocorre a cada passo de integração física $\Delta t_{\text{phys}} = 10^{-3}$ s, garantindo que o controlador utilize o modelo atualizado na configuração corrente do robô. A operação de maior custo computacional é a resolução do sistema linear $M \ddot{q} = \tau - C - G$ via `np.linalg.solve` (decomposição LU), com complexidade $O(N^3)$ por passo [26].

5.2.3 Análise de Robustez a Incertezas Paramétricas

A fragilidade fundamental do CTC é sua dependência do modelo exato. Em presença de erros de modelagem ΔM , ΔC , ΔG , a dinâmica de malha fechada torna-se:

$$\ddot{e} + K_D \dot{e} + K_P e = \underbrace{M^{-1}(q) [\Delta M(q) \ddot{q} + \Delta C \dot{q} + \Delta G]}_{\text{perturbação residual } d(t)}, \quad (5.5)$$

onde:

- $\ddot{e}, \dot{e}, e \in \mathbb{R}^N$ são o erro de aceleração, velocidade e posição de junta;
- $K_P, K_D \in \mathbb{R}^{N \times N}$ são os ganhos proporcional e derivativo do PID;
- $\Delta M(q), \Delta C(q, \dot{q}), \Delta G(q)$ representam, respectivamente, os erros de modelagem das matrizes de inércia, Coriolis e gravidade;
- $\ddot{q}, \dot{q} \in \mathbb{R}^N$ são a aceleração e a velocidade reais das juntas;
- $M^{-1}(q)$ é a inversa da matriz de inércia nominal;
- $d(t) \in \mathbb{R}^N$ é a perturbação residual resultante, proporcional aos erros paramétricos e ao estado dinâmico do robô.

Para erros paramétricos típicos de calibração de robôs industriais ($\leq 10\%$ de incerteza em massa e inércia) e trajetórias lentas a moderadas, $\|d\|$ é pequeno e o desempenho de rastreamento é mantido [23]. Entretanto, em cenários de perturbações externas abruptas (impactos, cargas não modeladas) ou erros paramétricos grandes (e.g., elos subaquáticos com massa adicionada mal estimada), o CTC-PID pode apresentar degradação significativa, motivando as estratégias robustas descritas nas seções seguintes [11, 40].

O uso da integral $K_I \int e d\tau$ melhora a rejeição de perturbações constantes e elimina erros de regime permanente devidos a erros de modelagem estáticos (e.g., ΔG constante), ao custo de reduzida margem de fase e necessidade de *anti-windup* [10].

A superfície $\mathcal{S} = \{(e, \dot{e}) : S = 0\}$ é uma variedade de dimensão N no espaço de estados $2N$ -dimensional, dentro da qual a dinâmica de erro satisfaz $\dot{e} + \Lambda e = 0$, um sistema linear estável de primeira ordem que converge exponencialmente com constante de tempo $1/\lambda_i$ [11].

A interpretação geométrica é a seguinte: se o controlador for capaz de forçar $S \rightarrow 0$ em tempo finito, então o estado do sistema fica “preso” sobre $\mathcal{S}$ e converge ao ponto de equilíbrio $e = 0$, $\dot{e} = 0$ seguindo a dinâmica unidimensional $\dot{e} + \Lambda e = 0$. O projeto do controlador resume-se então a garantir $S \rightarrow 0$, separando o problema em duas fases: a fase de alcance (*reaching phase*) e a fase de deslizamento (*sliding phase*) [40, 41].

5.3.2 Lei de Controle CT-SMC e Condição de Alcance

A lei de controle CT-SMC implementada no *Hephaestus* é:

$$\tau_{\text{SMC}} = M(q) [\ddot{q}_{d,f} - \Lambda \dot{e} - K \tanh(S/\phi)] + C(q, \dot{q}) + G(q), \quad (5.7)$$

onde:

- $\tau_{\text{SMC}} \in \mathbb{R}^N$ é o torque de controle SMC;
- $M(q), C(q, \dot{q}), G(q)$ são as matrizes/vetores dinâmicos definidos anteriormente;
- $\ddot{q}_{d,f} \in \mathbb{R}^N$ é a aceleração de referência filtrada (Seção 4.6 do Capítulo 4);
- $\Lambda \in \mathbb{R}^{N \times N}$ é a matriz diagonal de pendentes da superfície deslizante (Equação (5.6));
- $\dot{e} \in \mathbb{R}^N$ é o erro de velocidade de junta;
- $K = \text{diag}(K_1, \dots, K_N) \in \mathbb{R}^{N \times N}$ é a matriz diagonal de ganhos de robustez (*switching gains*);
- $\phi > 0$ é a espessura da camada limite (*boundary layer*) que regulariza $\text{sign}(S)$;
- $\tanh(S/\phi)$ é aplicado componente a componente.

A Equação (5.7) pode ser reescrita como $\tau = \tau_{\text{CTC}} + \tau_{\text{sw}}$, onde $\tau_{\text{CTC}} = M\ddot{q}_{d,f} + C + G$ é o termo de torque computado e $\tau_{\text{sw}} = -M(\Lambda \dot{e} + K \tanh(S/\phi))$ é o termo de chaveamento [11].

Considerando o modelo exato (M, C, G perfeitos) e substituindo a Equação (5.7) na equação de movimento, obtém-se a dinâmica de S :

$$\dot{S} = \ddot{e} + \Lambda \dot{e} = -K \tanh(S/\phi) + d, \quad (5.8)$$

onde:

- $\dot{S} \in \mathbb{R}^N$ é a derivada temporal da superfície deslizante;
- $\ddot{e}, \dot{e}$ são o erro de aceleração e velocidade;
- Λ é a matriz diagonal de pendentos (Equação (5.6));
- $K \tanh(S/\phi)$ é o termo de chaveamento contínuo (idêntico à Equação (5.7));
- $d \in \mathbb{R}^N$ representa as perturbações e erros de modelo agrupados, com componente d_i para a junta i limitada por $|d_i| \leq D_i$.

Para $\|d_i\| \leq D_i$ e $K_i > D_i$, a função de Lyapunov $V_i = S_i^2/2$ satisfaz [11, 40]:

$$\dot{V}_i = S_i \dot{S}_i = -K_i S_i \tanh(S_i/\phi_i) + S_i d_i \leq -(K_i - D_i) |S_i| + K_i \phi_i, \quad (5.9)$$

onde:

- $V_i = S_i^2/2$ é a função de Lyapunov candidata escalar para a junta i ;
- $\dot{V}_i$ é sua derivada ao longo das trajetórias do sistema;
- S_i é a i -ésima componente da superfície deslizante e $\dot{S}_i$ sua derivada temporal;
- $K_i > 0$ é o ganho de robustez e $\phi_i > 0$ a espessura da camada limite, ambos da junta i ;
- D_i é a cota superior da magnitude do distúrbio na junta i (i.e., $|d_i| \leq D_i$);
- a desigualdade utiliza a propriedade $|\tanh(x)| \geq |x|/(1 + |x|)$ e o limite uniforme $K_i \phi_i$.

Esta condição garante $\dot{V}_i < 0$ para $|S_i| > K_i \phi_i / (K_i - D_i)$. Portanto, $|S_i|$ é uniformemente limitado superiormente por $\|S_i\|_\infty \leq K_i \phi_i / (K_i - D_i)$ em regime estacionário, e a *camada limite* controla o compromisso entre *chattering* e precisão de rastreamento [12].

5.3.3 Integração com o Modelo e Sensibilidade Paramétrica

A dependência de τ_{SMC} em relação ao modelo simbólico-numérico é análoga ao CTC: as funções `func_M`, `func_C` e `func_G` são avaliadas na configuração corrente $(q, \dot{q})$ e o resultado é empregado diretamente na Equação (5.7). A distinção fundamental em relação ao CTC-PID reside no termo de robustez $-MK \tanh(S/\phi)$: para erros de modelo $\|\Delta M\|$, $\|\Delta C\|$, $\|\Delta G\|$ que produzam perturbações d_i com

$\|d_i\| \leq K_i(1 - \phi_i/\phi_i^*)$ para algum ϕ_i^* , o SMC mantém S_i na camada limite independentemente da magnitude das incertezas, desde que K_i seja suficientemente grande [11, 41].

Esse comportamento contrasta com o CTC-PID, no qual perturbações de mesma magnitude entram diretamente na dinâmica de erro linear e podem causar erros de regime permanente proporcional. O CT-SMC oferece, portanto, robustez intrínseca a perturbações limitadas ao custo de possível *chattering*, mitigado pela camada limite ϕ , e de uma resposta mais agressiva nos instantes iniciais, quando $|S|$ é grande e o termo $K \tanh(S/\phi)$ satura [11, 40].

O parâmetro λ desempenha papel duplo: define a inclinação da superfície deslizante (quanto maior λ , mais rápida a convergência sobre S) e amplifica $\dot{e}$ na lei de controle (o que pode amplificar ruído de velocidade ou picos de aceleração de referência provenientes do CLIK). O filtro de aceleração $\ddot{q}_{d,f} = \alpha \ddot{q}_d + (1 - \alpha)\ddot{q}_{d,f}^{\text{prev}}$ é essencial neste contexto, conforme discutido por Siciliano e Villani [55] e Slotine e Li [11]. Sintetiza-se, na Figura 5.2, a estrutura do CT-SMC, com destaque para a superfície deslizante $S = \dot{e} + \Lambda e$, para a camada limite $\tanh(S/\phi)$ e para a condição de alcance via função de Lyapunov.

5.4 Algoritmo Super-Twisting (STA)

5.4.1 Motivação: Chattering e Modos Deslizantes de Ordem Superior

O principal obstáculo prático ao SMC convencional é o *chattering*: a oscilação de alta frequência do sinal de controle causada pelo chaveamento de sinal de S nos limitadores de atuador reais [41]. Embora a camada limite $\tanh(S/\phi)$ reduza significativamente o *chattering*, ela não o elimina completamente e introduz erro de regime permanente proporcional a ϕ [12].

A teoria dos Modos Deslizantes de Ordem Superior (HOSM), desenvolvida por Levant [13, 56], resolve ambos os problemas simultaneamente: ao projetar leis de controle que impõem $S = \dot{S} = 0$ (em vez de apenas $S = 0$), os HOSMs produzem sinal de controle contínuo e convergência em tempo finito, sem o compromisso *chattering*-precisão do SMC de primeira ordem [14, 56].

O algoritmo Super-Twisting (STA), proposto originalmente por Levant [13] para o caso escalar, é o mais simples e amplamente utilizado dos HOSMs para sistemas de grau relativo um [14, 40]. Sua extensão ao problema de controle de manipuladores com N graus de liberdade foi estudada por Bartolini et al. [57] e consolidada por Shtessel et al. [58].

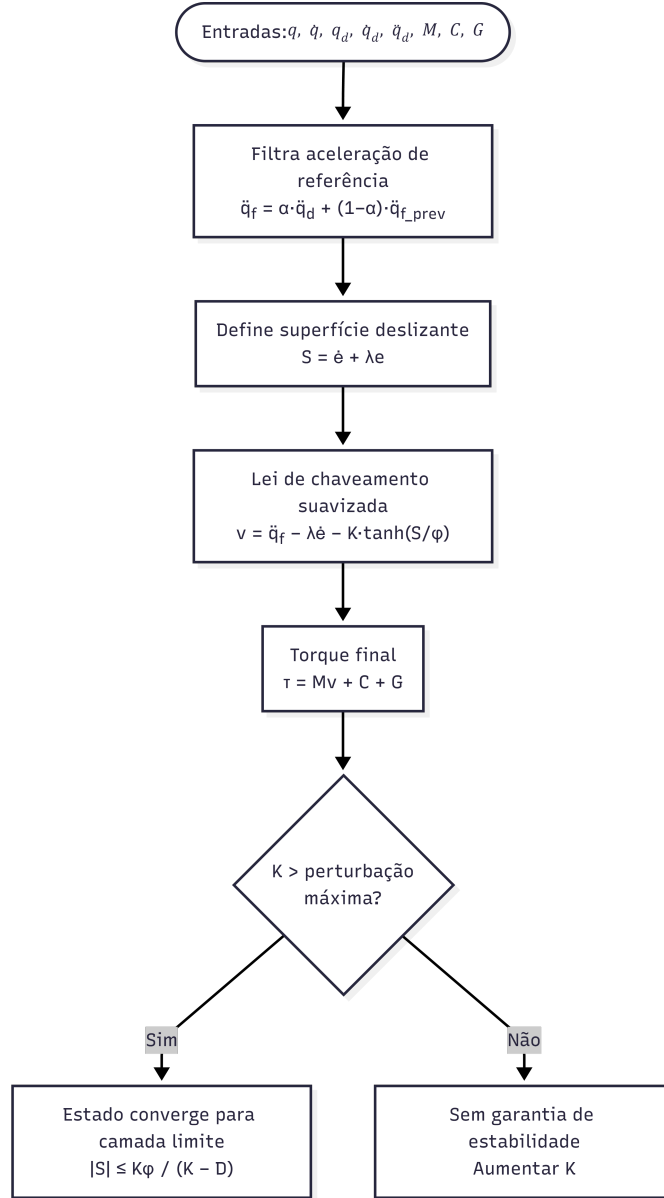


Figura 5.2: Estrutura do Controle por Modo Deslizante com Torque Computado (CT-SMC): o sinal de chaveamento $-K \tanh(S/\phi)$ garante a condição de alcance $\dot{V}_i < 0$ para perturbações $\|d_i\| \leq K_i$ (Equação (5.9)). O filtro passa-baixas de $\ddot{q}_{d,f}$ previne amplificação de picos de aceleração do CLIK [11, 12].

5.4.2 Análise de Estabilidade via Função de Lyapunov Estrita

O STA para a junta i define a variável interna z_i e a parcela de chaveamento:

$$v_{sw,i} = -k_{1,i}|S_i|^{1/2} \text{sign}(S_i) + z_i, \quad (5.10)$$

$$\dot{z}_i = -k_{2,i} \text{sign}(S_i), \quad (5.11)$$

onde:

- $v_{sw,i}$ é a parcela contínua de chaveamento da junta i ;
- S_i é a i -ésima componente da superfície deslizante (Equação (5.6));
- $|S_i|^{1/2}$ é a raiz quadrada do módulo de S_i e $\text{sign}(S_i)$ é a função sinal;
- z_i é a variável interna do STA que integra o sinal descontínuo, tornando $v_{sw,i}$ contínuo;
- $\dot{z}_i$ é a derivada temporal de z_i ;
- $k_{1,i}, k_{2,i} > 0$ são os ganhos do STA da junta i , ajustados de acordo com a magnitude esperada do distúrbio.

A lei de controle completa é:

$$\tau_{STA,i} = M(q)_i [\ddot{q}_{d,f,i} - \lambda_i \dot{e}_i + v_{sw,i}] + C_i + G_i. \quad (5.12)$$

onde:

- $\tau_{STA,i}$ é o torque STA da junta i ;
- $M(q)_i$ é a i -ésima linha da matriz de inércia $M(q)$;
- $\ddot{q}_{d,f,i}$ é a aceleração de referência filtrada da junta i ;
- $\lambda_i > 0$ é a pendente da superfície deslizante da junta i (Equação (5.6));
- $\dot{e}_i$ é o erro de velocidade da junta i ;
- $v_{sw,i}$ é o sinal contínuo gerado pelas Equações (5.10)–(5.11);
- C_i, G_i são, respectivamente, as i -ésimas componentes dos vetores de Coriolis e gravidade.

Moreno e Osorio [14] provaram que, para perturbações d_i com $|\dot{d}_i| \leq \delta_i$ (derivada limitada), o STA converge em tempo finito a $S_i = \dot{S}_i = 0$ se e somente se:

$$k_{2,i} > \delta_i, \quad k_{1,i} > 2\sqrt{k_{2,i}} \delta_i. \quad (5.13)$$

onde:

- $k_{1,i}, k_{2,i} > 0$ são os ganhos do STA da junta i definidos nas Equações (5.10)–(5.11);
- $\delta_i > 0$ é a cota superior da derivada temporal do distúrbio da junta i , ou seja, $|\dot{d}_i(t)| \leq \delta_i$ para todo t .

A prova utiliza a função de Lyapunov estrita quadrática-de-Lyapunov (QLF) para o vetor de estados $\xi_i = [|S_i|^{1/2} \text{sign}(S_i), z_i]^\top$ [14]:

$$V_i(\xi_i) = \xi_i^\top P_i \xi_i, \quad P_i = \begin{bmatrix} k_{2,i} + k_{1,i}^2/2 & -k_{1,i}/2 \\ -k_{1,i}/2 & 1/2 \end{bmatrix}, \quad (5.14)$$

onde:

- $V_i(\xi_i)$ é a função de Lyapunov estrita quadrática (QLF) para a junta i ;
- $\xi_i = [|S_i|^{1/2} \text{sign}(S_i), z_i]^\top \in \mathbb{R}^2$ é o vetor de estados aumentados do STA;
- $P_i \in \mathbb{R}^{2 \times 2}$ é uma matriz simétrica positiva definida que depende dos ganhos $k_{1,i}, k_{2,i}$;
- os elementos de P_i são $k_{2,i} + k_{1,i}^2/2$, $-k_{1,i}/2$ e $1/2$, escolhidos para que V_i satisfaça uma desigualdade diferencial finita no tempo;
- $\xi_i^\top$ é o vetor transposto de ξ_i .

Tem-se $\dot{V}_i \leq -\beta_i V_i^{1/2}$ para algum $\beta_i > 0$ dependendo dos ganhos, condição que implica convergência em tempo finito [14, 59]. A análise estende-se ao caso multi-junta pelo princípio da estabilidade independente para sistemas descentralizados sob a hipótese de que os acoplamentos dinâmicos residuais (incluídos em d_i) satisfazem as condições de limitação de derivada [11, 58].

5.4.3 Continuidade do Sinal de Controle e Eliminação do Chattering

A propriedade central que distingue o STA do CT-SMC convencional é a continuidade de $v_{\text{sw},i}$: embora $\text{sign}(S_i)$ seja descontínuo, a sua integral z_i é contínua, e

portanto $v_{\text{sw},i} = -k_{1,i}|S_i|^{1/2}\text{sign}(S_i) + z_i$ é contínuo para $S_i \neq 0$ e converge suavemente para zero em tempo finito [13]. Na implementação numérica, z_i é integrado por Euler explícito:

$$z_i^{n+1} = z_i^n - k_{2,i} \text{sign}(S_i^n) \Delta t_{\text{phys}}, \quad (5.15)$$

onde:

- z_i^n, z_i^{n+1} são, respectivamente, os valores da variável interna z_i nos instantes t_n e t_{n+1} ;
- S_i^n é o valor da superfície deslizante da junta i no instante t_n ;
- $k_{2,i} > 0$ é o ganho do STA;
- Δt_{phys} é o passo da integração física;
- esta é a discretização de Euler explícito da Equação (5.11).

Desde que Δt_{phys} seja suficientemente pequeno (condição discutida na Seção 5.4.4), essa implementação preserva as propriedades de convergência em tempo finito [60]. Ilustra-se, na Figura 5.3, a estrutura do algoritmo Super-Twisting, com a variável interna z_i que integra o sinal descontínuo $-k_{2,i} \text{sign}(S_i)$ para produzir o sinal de controle contínuo $v_{\text{sw},i}$.

5.4.4 Implementação Discreta e Estabilidade Numérica

A discretização do STA introduz erros que podem comprometer a convergência em tempo finito para passos de integração grandes. Levant [60] estabeleceu que, para o STA discreto com passo $h = \Delta t_{\text{phys}}$ e ganhos (k_1, k_2) , a propriedade de *real sliding* garante que os estados satisfazem $|S| = O(h)$ em regime, uma precisão de primeira ordem no passo de integração. No *Hephaestus*, com $\Delta t_{\text{phys}} = 10^{-3}$ s e ganhos típicos $k_1 = 5$, $k_2 = 10$, o erro de regime em modo deslizante é da ordem de $k_1 h^{1/2} \approx 0.16$ rad, negligenciável para as trajetórias de teste e coberto pela convergência do CLIK [58, 60].

5.5 Controle por Rejeição Ativa de Distúrbios Linear (LADRC)

5.5.1 Paradigma de Rejeição Ativa e Motivação

O LADRC (*Linear Active Disturbance Rejection Control*), originalmente proposto por Han [42] como *Active Disturbance Rejection Control* (ADRC) não-linear e

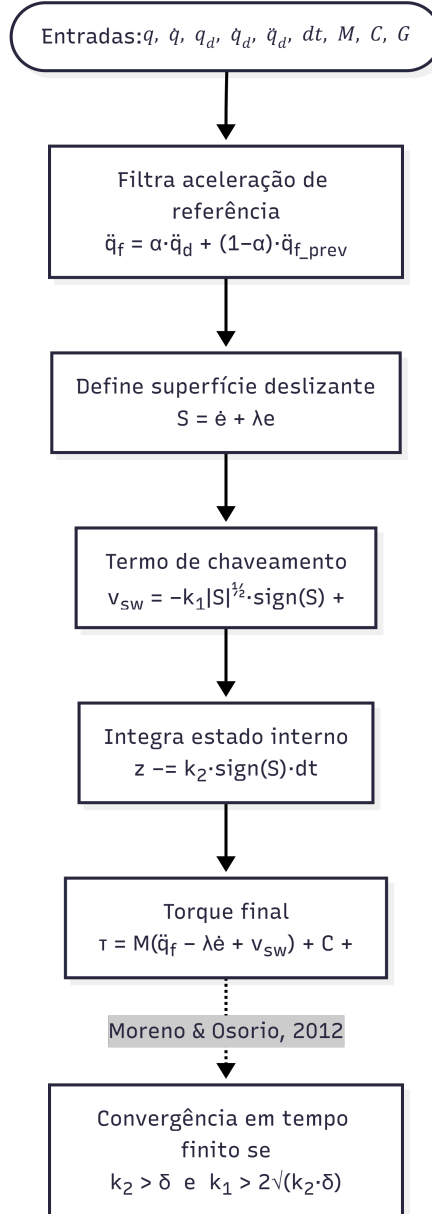


Figura 5.3: Estrutura do algoritmo Super-Twisting (STA): a variável interna z_i (Equação (5.11)) integra o sinal descontínuo $-k_{2,i} \text{sign}(S_i)$, tornando $v_{sw,i}$ contínuo e eliminando o *chattering*. As condições de ganho $k_{2,i} > \delta_i$ e $k_{1,i} > 2\sqrt{k_{2,i}\delta_i}$ garantem convergência em tempo finito [13, 14].

reformulado na versão linear por Gao [15], representa um paradigma fundamentalmente distinto das três estratégias anteriores: em vez de cancelar a não-linearidade por inversão do modelo ou de projetar superfícies robustas, o ADRC *estima e rejeita* em tempo real a totalidade dos efeitos não modelados e perturbações externas por meio de um Observador de Estado Estendido (ESO) [15, 17, 18].

A motivação central é a seguinte: em sistemas reais, o modelo dinâmico disponível é sempre aproximado, seja por incerteza paramétrica, simplificações geométricas, dinâmica de elos flexíveis não modelada, ou acoplamentos ignorados. Ao tratar a totalidade dessas discrepâncias como um “distúrbio total” f_i estimável em tempo real, o LADRC pode oferecer desempenho equivalente ao CTC com modelo exato mesmo quando o modelo utilizado é apenas uma aproximação grosseira da dinâmica real [15–17].

5.5.2 Formulação do Modelo de Planta de Canal Integrador

Para a junta i , a dinâmica de malha fechada é reescrita no formato de *canal integrador com distúrbio total*:

$$\ddot{q}_i = b_{0,i} \tau_i + f_i, \quad (5.16)$$

onde:

- $\ddot{q}_i$ é a aceleração da junta i ;
- τ_i é o torque aplicado à junta i ;
- $b_{0,i}$ é o ganho nominal de canal (a ser escolhido pelo projetista, tipicamente $b_{0,i} \approx 1/M_{ii}$);
- f_i é o distúrbio total da junta i , que agrupa todos os efeitos não capturados pelo modelo nominal de canal integrador.

O distúrbio total f_i engloba [15, 18]:

- os termos de Coriolis residuais não compensados pelo *feedforward*: $-[C\dot{q}]_i/M_{ii}$;
- os acoplamentos dinâmicos entre juntas: $-\sum_{j \neq i} M_{ij}\ddot{q}_j/M_{ii}$;
- erros de modelo: $\Delta M, \Delta C, \Delta G$;
- perturbações externas (torques de distúrbio τ_{dist}).

A escolha $b_{0,i} \approx 1/M_{ii}(q_0)$ como estimativa do ganho de canal integrador é a mais natural: $M_{ii}(q_0)$ é o momento de inércia efetivo da junta i na postura inicial, de forma que $b_{0,i}\tau_i$ captura a resposta linear dominante de torque para aceleração

[15]. O *Hephaestus* implementa a estimativa automática pela diagonal de $M(q_0)$ via a opção `auto_b0 = True`:

$$b_{0,i} = \frac{1}{\max(M_{ii}(q_0), 10^{-4})}, \quad (5.17)$$

onde:

- $b_{0,i}$ é o ganho nominal de canal estimado automaticamente para a junta i ;
- $M_{ii}(q_0)$ é o i -ésimo elemento diagonal da matriz de inércia avaliada na postura inicial q_0 ;
- $\max(\cdot, 10^{-4})$ é uma saturação inferior que impede divisão por zero em configurações degeneradas (em que $M_{ii}(q_0) \rightarrow 0$).

5.5.3 Observador de Estado Estendido (ESO) Linear

O ESO de terceira ordem estima o estado $(q_i, \dot{q}_i)$ e o distúrbio total f_i a partir das medições de posição de junta e da entrada de controle anterior. O modelo aumentado para a junta i é:

$$\frac{d}{dt} \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}}_{A_e} \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} + \underbrace{\begin{bmatrix} 0 \\ b_0 \\ 0 \end{bmatrix}}_{B_e} \tau_i + \underbrace{\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}}_E \dot{f}_i, \quad (5.18)$$

onde:

- z_1, z_2, z_3 são, respectivamente, os estados estimados $z_1 \approx q_i$, $z_2 \approx \dot{q}_i$ e $z_3 \approx f_i$;
- $A_e \in \mathbb{R}^{3 \times 3}$ é a matriz de estado aumentada (cadeia de integradores);
- $B_e \in \mathbb{R}^{3 \times 1}$ é o vetor de entrada com ganho nominal $b_0 = b_{0,i}$;
- $E \in \mathbb{R}^{3 \times 1}$ é o vetor que injeta a derivada do distúrbio $\dot{f}_i$ no estado estendido;
- τ_i é o torque aplicado à junta i .

O ESO é implementado como um observador de Luenberger para o par $(A_e, C_e = [1, 0, 0])$:

$$\dot{\hat{x}} = A_e \hat{x} + B_e \tau_i + L_e (q_i - \hat{z}_1), \quad L_e = [\beta_1, \beta_2, \beta_3]^\top, \quad (5.19)$$

onde:

- $\hat{x} = [\hat{z}_1, \hat{z}_2, \hat{z}_3]^\top \in \mathbb{R}^3$ é o vetor de estados estimados;

- $\dot{\hat{x}}$ é sua derivada temporal;
- A_e, B_e são as matrizes definidas na Equação (5.18);
- τ_i é o torque aplicado;
- q_i é a posição medida da junta i ;
- $(q_i - \hat{z}_1)$ é o erro de inovação, multiplicado pelo ganho do observador;
- $L_e = [\beta_1, \beta_2, \beta_3]^\top \in \mathbb{R}^3$ é o vetor de ganhos do observador, escolhido para alocar todos os autovalores do erro em $-\omega_o$.

A parametrização de banda por frequência (*bandwidth parameterization*) proposta por Gao [15] é:

$$\beta_1 = 3\omega_o, \quad \beta_2 = 3\omega_o^2, \quad \beta_3 = \omega_o^3. \quad (5.20)$$

onde:

- $\beta_1, \beta_2, \beta_3$ são os ganhos do observador correspondentes aos três estados aumentados;
- $\omega_o > 0$ é a frequência de banda (*bandwidth*) do ESO, único parâmetro de ajuste do observador;
- os coeficientes 3, 3, 1 correspondem aos coeficientes binomiais $\binom{3}{k}$ que asseguram polinômio característico $(\lambda + \omega_o)^3$.

Essa parametrização unifica o ajuste do ESO em um único parâmetro ω_o (frequência de banda do observador), simplificando o projeto: um ω_o maior implica observador mais rápido e melhor rejeição de distúrbios variáveis no tempo, ao custo de maior amplificação de ruído de medição e potencial instabilidade numérica caso o produto $\omega_o \Delta t_{\text{phys}}$ ultrapasse o limite heurístico discutido na Equação (5.27) [15, 16].

A análise de convergência do ESO, desenvolvida por Zheng e Gao [16], estabelece que, para distúrbios f_i com $|\dot{f}_i| \leq \delta_i$ e modelo aproximado com erro $|\Delta b_{0,i}| < b_{0,i}$, o erro de estimação satisfaz:

$$\lim_{t \rightarrow \infty} \|z_3(t) - f_i(t)\| \leq \frac{\delta_i}{\omega_o}, \quad (5.21)$$

onde:

- $z_3(t)$ é a estimativa de distúrbio produzida pelo ESO;
- $f_i(t)$ é o distúrbio total real da junta i ;

- $\|z_3(t) - f_i(t)\|$ é o erro absoluto de estimação;
- δ_i é a cota superior da derivada temporal do distúrbio, $|\dot{f}_i| \leq \delta_i$;
- ω_o é a frequência de banda do ESO (Equação (5.20)).

Esta relação mostra que a qualidade da estimação do distúrbio é inversamente proporcional à frequência de banda do observador [16]. Esquematiza-se, na Figura 5.4, a estrutura do ESO de terceira ordem, com a parametrização por largura de banda ω_o e a estimação do estado estendido $z_3 \approx f_i$.

5.5.4 Lei de Controle e Feedforward Explícito

A lei de controle do LADRC cancela o distúrbio estimado $z_{3,i}^{\text{flt}}$ e acrescenta um sinal de controle PD linear sobre as referências do CLIK:

$$u_{0,i} = \ddot{q}_{d,i} + k_{p,i}(q_{d,i} - q_i) + k_{d,i}(\dot{q}_{d,i} - \dot{q}_i), \quad (5.22)$$

$$\tau_i^{\text{LADRC}} = \frac{u_{0,i} - z_{3,i}^{\text{flt}}}{b_{0,i}} + G_i + C_i. \quad (5.23)$$

onde:

- $u_{0,i}$ é o sinal de controle nominal da junta i (lei PD com *feedforward* de aceleração);
- τ_i^{LADRC} é o torque de controle LADRC aplicado à junta i ;
- $\ddot{q}_{d,i}, \dot{q}_{d,i}, q_{d,i}$ são as referências de aceleração, velocidade e posição da junta i geradas pelo CLIK;
- $q_i, \dot{q}_i$ são a posição e a velocidade medidas da junta;
- $k_{p,i}, k_{d,i} > 0$ são os ganhos proporcional e derivativo da junta i ;
- $z_{3,i}^{\text{flt}}$ é a estimativa do distúrbio total f_i produzida pelo ESO e suavizada pelo filtro IIR;
- $b_{0,i}$ é o ganho nominal de canal (Equação (5.17));
- G_i, C_i são, respectivamente, as i -ésimas componentes dos vetores de gravidade e Coriolis aplicadas como *feedforward* explícito.

O *feedforward* explícito de G_i e C_i na Equação (5.23) reduz o distúrbio residual que o ESO precisa estimar. Sem esse *feedforward*, o distúrbio total f_i inclui $-G_i/M_{ii}$ (que pode ser da ordem de dezenas de rad/s^2 para elos pesados em posição não-horizontal) e $-[C\dot{q}]_i/M_{ii}$ (crescente com $\dot{q}^2$ ao longo da trajetória), impondo ao ESO

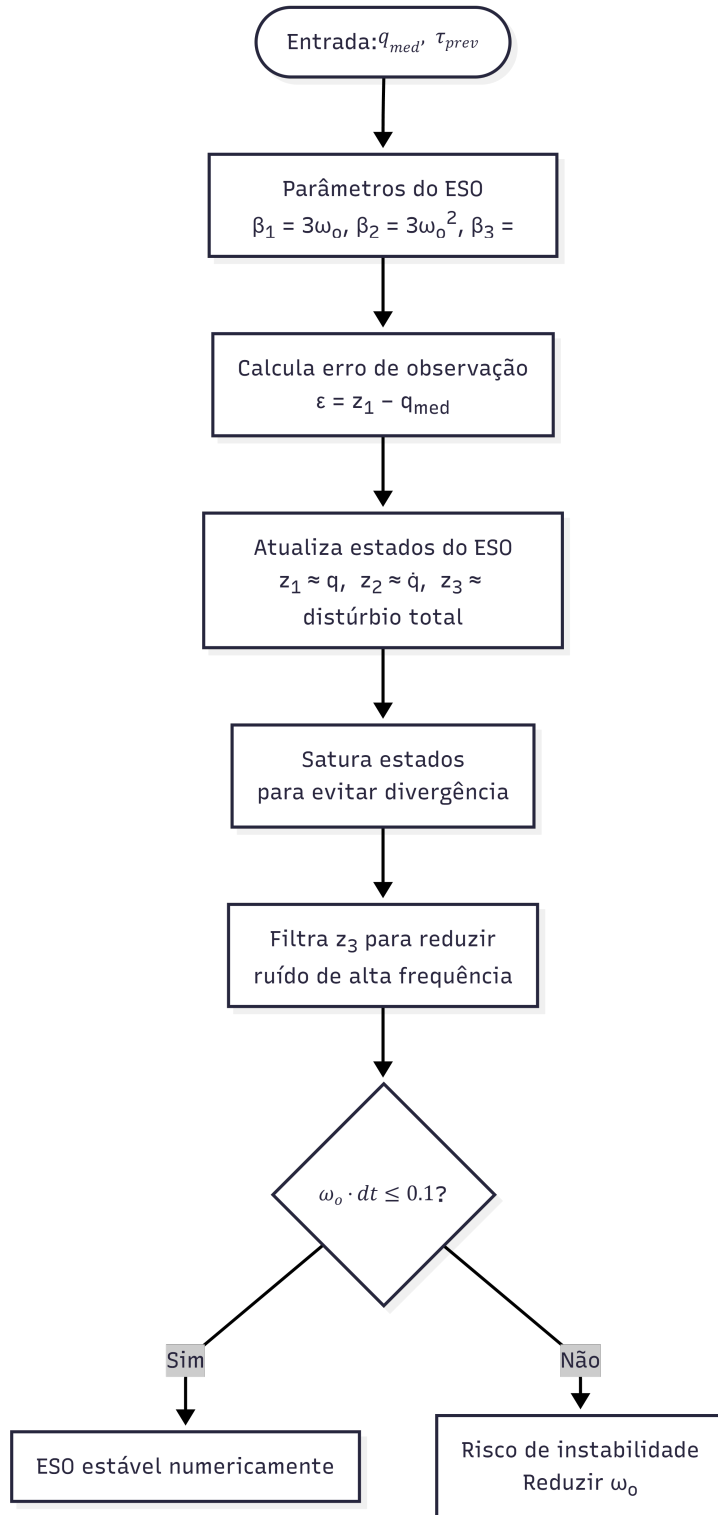


Figura 5.4: Observador de Estado Estendido (ESO) de terceira ordem do LADRC: estima o estado ($z_1 \approx q_i, z_2 \approx \dot{q}_i, z_3 \approx f_i$) a partir da medição de posição q_i e da entrada de controle anterior. Os ganhos $\beta_k = \binom{3}{k}\omega_o^k$ alocam todos os polos do observador em $-\omega_o$ [15, 16].

uma tarefa de estimação com dinâmica crescente que introduz transientes oscilatórios [17]. Com o *feedforward*, f_i é reduzido a erros de modelo e perturbações externas, sinais menores e mais lentos que o ESO estima com maior precisão [18]. Apresenta-se, na Figura 5.5, a lei de controle completa do LADRC, com destaque para o cancelamento do distúrbio estimado $z_{3,i}^{\text{filt}}$ e para o *feedforward* explícito de G e C .

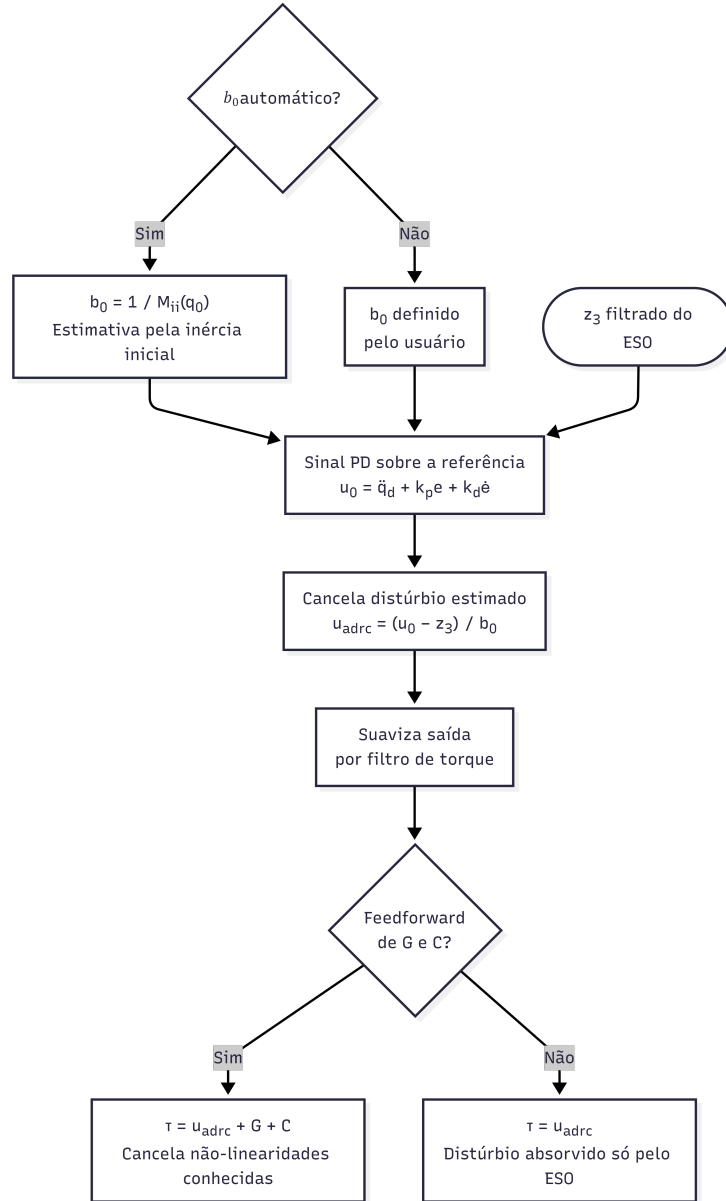


Figura 5.5: Lei de controle do LADRC (Equações (5.22)–(5.23)): o distúrbio estimado $z_{3,i}^{\text{filt}}$ é subtraído da ação de controle PD $u_{0,i}$ e o resultado é dividido por $b_{0,i}$; o *feedforward* explícito de G_i e C_i reduz o distúrbio residual que o ESO precisa rastrear [15, 17, 18].

5.5.5 Descentralização e Acoplamento Dinâmico Residual

A implementação descentralizada do LADRC, com um par (ESO, lei de controle) por junta, ignora os acoplamentos dinâmicos entre juntas presentes na matriz de

inércia não-diagonal e na matriz de Coriolis. No entanto, esses acoplamentos são exatamente os termos que compõem o distúrbio total f_i : eles são estimados pelo ESO e cancelados implicitamente, sem necessidade de inversão explícita do modelo acoplado [15, 18].

Esta propriedade é particularmente relevante para robôs com elos pesados (onde M_{ij} , $i \neq j$, é significativo) e trajetórias rápidas (onde os termos de Coriolis $C_{ij}\dot{q}_j$ são grandes): o LADRC pode oferecer desempenho comparável ao CTC acoplado mesmo quando o modelo de junta utilizado é apenas o integrador duplo com $b_0 = 1/M_{ii}$. Evidências experimentais desta propriedade foram reportadas por Huang e Xue [18] e confirmadas para manipuladores seriais por Madonski et al. [61].

5.6 Integração dos Controladores com o Modelo Simbólico

5.6.1 Cadeia de Dependências: do Simbólico ao Numérico

A integração dos controladores com o modelo simbólico envolve uma cadeia de dependências que atravessa todos os módulos do *Hephaestus*:

1. **engine.py** ($\rightarrow$ modelo simbólico): gera $M(q)$, $C(q, \dot{q})$, $G(q)$ como objetos simbólicos SymPy via `RobotMathEngine.run_full_process()`.
2. **simulator.py** ($\rightarrow$ compilação): compila os objetos simbólicos em funções NumPy via `sp.lambdify(..., cse=True)` no construtor de `RobotSimulator`, produzindo `func_M`, `func_C`, `func_G`.
3. **simulator.py** ($\rightarrow$ avaliação): avalia `func_M(*args)`, `func_C(*args)`, `func_G(*args)` a cada substep físico para obter os arrays NumPy `M`, `C`, `G`.
4. **Controlador** ($\rightarrow$ torque): utiliza `M`, `C`, `G` para calcular τ_{ctrl} conforme a lei específica de cada estratégia.
5. **Integrador** ($\rightarrow$ próximo estado): resolve $\ddot{q} = M^{-1}(\tau - C - G - \tau_{pass})$ e atualiza $(q, \dot{q})$.

Esta cadeia é completamente transparente ao usuário: a seleção do controlador via `ctrl_params["type"]` é a única diferença entre as quatro estratégias ao nível da interface de programação. A compilação simbólica-numérica e a avaliação das matrizes dinâmicas são idênticas para todos os controladores.

5.6.2 Uso Diferenciado do Modelo por Cada Estratégia

Embora todos os controladores consumam as mesmas funções compiladas, o papel de M , C e G difere qualitativamente entre as estratégias, conforme resumido na Tabela 5.1:

Tabela 5.1: Papel das matrizes dinâmicas em cada estratégia de controle.

Controlador	Uso de M	Uso de C	Uso de G	Sensibilidade a Erros de Modelo
CTC-PID	inversão explícita M^{-1}	cancelamento	cancelamento	alta (erro linear em $\Delta M, \Delta C, \Delta G$)
CT-SMC	idem	idem	idem	reduzida (robustez por ganho K)
STA	idem	idem	idem	reduzida (idem)
LADRC	estimativa $b_0 \approx 1/M_{ii}$	<i>feedforward</i> opcional	<i>feedforward</i> opcional	baixa (ESO estima residuais)

Para o CTC-PID, CT-SMC e STA, as matrizes M , C , G entram na lei de controle pela mesma estrutura $\tau = Mv + C + G$: M é utilizada para mapear o sinal auxiliar v (no espaço de aceleração de junta) para torque, e C , G são somados como *feedforward* para cancelar os torques não-lineares. Qualquer erro nessas matrizes propaga-se diretamente para o torque calculado e, conseqüentemente, para o erro de rastreamento [23].

Para o LADRC, M é utilizada apenas para estimar b_0 na postura inicial, um uso muito menos demandante em termos de exatidão, e C , G entram como *feedforward* opcional. O ESO absorve o residual entre o modelo utilizado e a dinâmica real, tornando o desempenho do LADRC substancialmente menos sensível a erros paramétricos do modelo [15, 61].

5.6.3 Avaliação Numérica e Custo Computacional por Controlador

A Tabela 5.2 quantifica o custo computacional adicional por passo de integração para cada estratégia, além da avaliação de M , C , G que é comum a todos:

Para $N \leq 8$ GDL e $\Delta t_{\text{phys}} = 10^{-3}$ s, todos os controladores adicionam *overhead* desprezível em comparação com a avaliação de M , C , G (dominante). O custo computacional dos controladores torna-se relevante apenas para $N \geq 15$ ou em implementações *hard real-time* com restrições de ciclo na ordem de microssegundos [43].

Tabela 5.2: Custo computacional adicional por passo de integração Δt_{phys} .

Controlador	Operações principais	Custo dominante
CTC-PID	$M \cdot v$, <i>anti-windup</i>	produto matriz-vetor $O(N^2)$
CT-SMC	filtro $\ddot{q}_{d,f}$, S , $\tanh(S/\phi)$, $M \cdot v$	idem + N avaliações de $\tanh$
STA	filtro $\ddot{q}_{d,f}$, S , $ S ^{1/2}$, $\text{sign}(S)$, integração de z , $M \cdot v$	idem + N raízes quadradas
LADRC	N ESOs (3 estados cada), filtro IIR em z_3 , lei PD, divisão por b_0	$3N$ operações Euler + N divi- sões

5.7 Interface Gráfica e Configuração dos Controladores

5.7.1 Arquitetura de Configuração via `ctrl_params`

A interface gráfica do *Hephaestus*, implementada em `main.py` com a biblioteca CustomTkinter, expõe os parâmetros de cada controlador por meio de campos de entrada dinâmicos na aba de Simulação. A parametrização é empacotada no dicionário `ctrl_params` que é passado ao método `RobotSimulator.run()`, sem que a interface gráfica precise conhecer a implementação interna dos controladores, em uma separação de responsabilidades que segue o padrão MVC (*Model-View-Controller*) [62].

Listam-se, na Tabela 5.3, os parâmetros configuráveis por estratégia:

Tabela 5.3: Parâmetros de configuração dos controladores na interface `ctrl_params`.

Controlador	Parâmetros principais	Descrição
CTC-PID	Kp, zeta, ki, windup_limit	Ganhos proporcional, amortecimento, integral e limite <i>anti-windup</i>
CT-SMC	lambda, K, phi, ddq_filter_alpha	Pendente da superfície, ganho de robustez, espessura da camada limite, coef. filtro
STA	lambda, k1, k2, ddq_filter_alpha	Pendente da superfície, ganhos STA, coef. filtro
LADRC	wo, b0 (ou auto_b0), kp, kd, z3_filter_alpha, gravity_ff, coriolis_ff	Banda do observador, ganho de canal, ganhos PD, filtro de distúrbio, <i>flags</i> de <i>feedforward</i>

5.7.2 Ferramentas de Ajuste e Análise Comparativa

A aba de Análise da interface gráfica permite a sobreposição de resultados de múltiplas sessões de simulação (`comparison_sessions`), cada uma armazenando os arrays de erro e torque, os parâmetros do controlador e as condições da trajetória. Essa funcionalidade é fundamental para a metodologia de ajuste iterativo de ganhos: o usuário pode comparar visualmente a resposta ao degrau de erro de junta para diferentes valores de λ (SMC/STA) ou ω_o (LADRC), identificando o compromisso *velocidade de convergência* $\times$ *chattering* $\times$ *consumo de torque* [10, 17].

A serialização das sessões em formato `pickle` permite a exportação dos arrays de resultado para análise posterior em ferramentas externas (MATLAB, SciPy, etc.), enquanto a funcionalidade de exportação para código Octave das equações simbólicas (via `octave_code` do SymPy) permite verificação cruzada das matrizes dinâmicas com *toolboxes* de robótica estabelecidas [23, 63].

5.8 Análise Comparativa das Estratégias de Controle

5.8.1 Propriedades Teóricas Comparadas

Sintetizam-se, na Tabela 5.4, as propriedades teóricas fundamentais das quatro estratégias de controle implementadas, segundo a literatura de referência:

Tabela 5.4: Comparação teórica das estratégias de controle implementadas.

Propriedade	CTC-PID	CT-SMC	STA	LADRC
Estabilidade (modelo exato)	Global assintótica	Global assintótica	Tempo finito	Assintótica (ESO conv.)
Robustez a $\Delta M, \Delta C, \Delta G$	Baixa	Alta (por K)	Alta (por k_1, k_2)	Alta (por ESO)
<i>Chattering</i>	Nenhum	Reduzido (tanh)	Nenhum (cont.)	Nenhum
Convergência	Assintótica	Assintótica c/ camada	Tempo finito	Assintótica
Req. de modelo	M, C, G exatos	M, C, G nominais	Idem	Apenas $b_0 \approx 1/M_{ii}$
Parâmetros	2–3	3–4	3–4	3–5
Ref. principais	[5, 23]	[11, 40]	[13, 14]	[15, 18]

5.8.2 Orientações para Seleção e Ajuste de Ganhos

Com base na análise teórica e nas propriedades implementadas, as seguintes diretrizes de seleção e ajuste de ganhos são propostas para o ambiente *Hephaestus* [10, 11, 15, 17]:

CTC-PID

A condição de estabilidade que limita o ganho proporcional k_p do CTC-PID em função do passo de integração simplético é dada pela Equação (5.24):

$$\Delta t_{\text{phys}} < \frac{2}{\sqrt{k_p}}, \quad (5.24)$$

onde Δt_{phys} é o passo de integração física e k_p é o ganho proporcional do CTC-PID; valores maiores de k_p exigem passos de integração proporcionalmente menores (Seção 4.9 do Capítulo 4).

- Ponto de partida: $k_p = 100 \text{ rad}^2/\text{s}^2$, $\zeta = 1$ (amortecimento crítico) e $k_i = 0$.
- Aumentar k_p para reduzir erros de rastreamento em trajetórias rápidas, sempre verificando a condição da Equação (5.24).
- Ativar k_i apenas em presença de perturbações constantes ou erros estáticos de modelo. Iniciar com $k_i = k_p/100$ e aumentar até eliminar o erro de regime.

CT-SMC

A condição que dimensiona o ganho de robustez K do CT-SMC para garantir a invariância da superfície deslizante face a perturbações limitadas [11] é apresentada na Equação (5.25):

$$K > \|d\|_{\infty}, \quad (5.25)$$

onde K é o ganho diagonal de robustez e $\|d\|_{\infty}$ é a cota superior da magnitude do distúrbio agregado (erros de modelo + perturbações externas), tipicamente estimada pela amplitude do erro de rastreamento observado no CTC-PID com o mesmo modelo.

- Ponto de partida: $\lambda = 5 \text{ rad/s}$, $K = 5 \text{ N}\cdot\text{m}$ e $\phi = 0, 1 \text{ rad/s}$.
- λ determina a dinâmica sobre $\mathcal{S}$: valores maiores aceleram a convergência, mas amplificam ruído de velocidade.
- K deve satisfazer a Equação (5.25).

- Ajustar ϕ para o maior valor que ainda garanta precisão de rastreamento aceitável, minimizando o *chattering* residual [12].
- Reduzir `ddq_filter_alpha` (ex. 0,5) se picos de aceleração do CLIK causarem torques impulsivos.

STA

A condição necessária e suficiente para a convergência em tempo finito do STA, em sua forma compacta de razão entre os ganhos [14], é apresentada na Equação (5.26):

$$\frac{k_2}{k_1^2} > \frac{\delta_i}{4k_{2,i}}, \quad (5.26)$$

onde k_1 e k_2 são os ganhos do STA da junta i e δ_i é a cota superior da derivada temporal do distúrbio (Equação (5.13)).

- Ponto de partida: $\lambda = 5$ rad/s, $k_1 = 5$ e $k_2 = 10$.
- Verificar as condições da Equação (5.13), estimando δ_i como a máxima taxa de variação do distúrbio observada no CT-SMC.
- Aumentar k_2 para melhorar a rejeição de distúrbios variáveis; aumentar k_1 para acelerar a convergência.
- A condição da Equação (5.26) é necessária e suficiente para a convergência em tempo finito.

LADRC

A condição de estabilidade numérica do ESO discreto que limita a frequência de banda ω_o do observador em função do passo de integração [16] é dada pela Equação (5.27):

$$\omega_o \leq \frac{0,1}{\Delta t_{\text{phys}}}, \quad (5.27)$$

onde ω_o é a frequência de banda do ESO (Equação (5.20)) e Δt_{phys} é o passo de integração; para $\Delta t_{\text{phys}} = 10^{-3}$ s, o limite resultante é $\omega_o^{\text{max}} = 100$ rad/s.

- Ponto de partida: $\omega_o = 20 \omega_{\text{cl}}$, onde $\omega_{\text{cl}} = \sqrt{k_p}$ é a frequência de malha fechada. Para $k_p = 100$, tem-se $\omega_o = 200$ rad/s, sujeita ao limite da Equação (5.27).
- Verificar a Equação (5.27) antes de aumentar ω_o .
- Ativar `auto_b0 = True` inicialmente; ajustar manualmente se a resposta apresentar oscilações crescentes (indicativas de b_0 muito alto) ou lentidão (indicativas de b_0 muito baixo).

- O filtro `z3_filter_alpha` (padrão 0,2) é crítico nos primeiros instantes; valores muito altos ($> 0,5$) podem reamplificar o conteúdo de alta frequência do ESO [17].
- Sempre ativar `gravity_ff = True` para robôs em ar; avaliar `coriolis_ff = True` para trajetórias rápidas.

5.8.3 Aplicabilidade por Cenário

A seleção da estratégia de controle mais adequada depende do cenário de aplicação específico. Para o *Hephaestus*, os seguintes critérios gerais podem ser adotados:

- **Modelo bem calibrado, perturbações pequenas:** CTC-PID é a escolha natural, por sua simplicidade de ajuste e interpretação direta [5, 23].
- **Perturbações externas significativas ou incerteza paramétrica:** CT-SMC ou STA oferecem robustez explícita. O STA é preferível quando a continuidade do torque é requisito (e.g., atuadores elétricos com limite de corrente) [13, 58].
- **Modelo apenas aproximado ou parcialmente desconhecido:** LADRC é recomendado, pois sua dependência do modelo se resume ao ganho b_0 [15, 18, 61].
- **Ambiente subaquático com massa adicionada incerta:** LADRC ou STA são preferíveis ao CTC, pois a estimação da massa adicionada por ensaios de fluido é imprecisa e os erros paramétricos nessa grandeza podem ser superiores a 30% [1, 19].

5.9 Forças Dissipativas como Perturbações Estruturadas

Os modelos de atrito nas juntas e arrasto hidrodinâmico, descritos na Seção 4.7 do Capítulo 4, não são incluídos nas leis de controle dos quatro estratégias, e são aplicados como torques passivos externos τ_{pass} na dinâmica direta. Do ponto de vista da análise de robustez, esse tratamento implica que as forças dissipativas funcionam como *perturbações estruturadas* que os controladores robustos (CT-SMC, STA, LADRC) estimam e rejeitam, enquanto o CTC-PID as observa pelo sinal de erro residual.

O atrito de Coulomb regularizado $F_{c,i} \tanh(\dot{q}_i/\varepsilon)$ é uniformemente limitado por $F_{c,i}$ (distúrbio limitado), sendo diretamente rejeitável pelo CT-SMC desde que o

ganho de robustez satisfaça a condição apresentada na Equação (5.28) [11]:

$$K_i > \frac{F_{c,i}}{M_{ii}}, \quad (5.28)$$

onde K_i é o ganho de robustez do CT-SMC da junta i , $F_{c,i}$ é a amplitude do atrito de Coulomb e M_{ii} é o i -ésimo elemento diagonal da matriz de inércia. Para o STA, a condição análoga é $k_{2,i} > \dot{F}_{c,i}^{\max}/M_{ii}$; como $\tanh$ é diferenciável, essa derivada é limitada por $F_{c,i}/(\varepsilon M_{ii})$ em regime [14]. Para o LADRC, o atrito entra em f_i e é estimado pelo ESO com precisão limitada por ω_o ; para atrito Coulomb puro ($\dot{q} = 0 \rightarrow \dot{q} \neq 0$), a descontinuidade em f_i requer ω_o suficientemente alto para rastrear a transição [17, 18].

O arrasto hidrodinâmico $D_{\text{quad},i}|\dot{q}_i|\dot{q}_i$ é uma perturbação dependente do estado e crescente com $\dot{q}_i^2$, o que pode violar a hipótese de distúrbio limitado do CT-SMC para trajetórias muito rápidas [1]. Para o LADRC, o arrasto quadrático é uma função contínua de $\dot{q}$ e sua derivada $2D_{\text{quad},i}|\dot{q}_i|\ddot{q}_i$ é limitada quando as velocidades e acelerações são finitas, preservando a condição de convergência do ESO [16].

5.10 Validação da Integração por Análise de Conservação de Energia

Uma verificação interna da consistência da integração entre os controladores e o modelo simbólico pode ser realizada pela análise da energia mecânica total do sistema. Para o CTC com modelo exato ($\Delta M = \Delta C = \Delta G = 0$) e controlador PD (sem K_I e sem perturbações), a dinâmica de erro é exatamente $\ddot{e} + K_D\dot{e} + K_P e = 0$. A função de Lyapunov da dinâmica de erro:

$$V_{\text{PD}}(e, \dot{e}) = \frac{1}{2}\dot{e}^\top M_{\text{nom}}\dot{e} + \frac{1}{2}e^\top K_P e, \quad (5.29)$$

onde:

- $V_{\text{PD}}(e, \dot{e})$ é a função de Lyapunov candidata (interpretável como “energia mecânica” do erro);
- $e = q_d - q \in \mathbb{R}^N$ é o erro de posição de junta e $\dot{e}$ o erro de velocidade;
- $M_{\text{nom}} \in \mathbb{R}^{N \times N}$ é a matriz de inércia nominal (constante ou avaliada na postura nominal), assumida simétrica e positiva definida;
- $K_P \in \mathbb{R}^{N \times N}$ é a matriz de ganho proporcional (positiva definida);
- o primeiro termo $\frac{1}{2}\dot{e}^\top M_{\text{nom}}\dot{e}$ representa a energia cinética associada ao erro de velocidade;

- o segundo termo $\frac{1}{2}e^\top K_P e$ representa a energia potencial elástica associada ao erro de posição.

Esta função satisfaz $\dot{V}_{PD} = -\dot{e}^\top K_D \dot{e} \leq 0$ (dissipação de energia pelo amortecimento), garantindo convergência assintótica para o equilíbrio $e = \dot{e} = 0$ [52]. Esta análise pode ser verificada numericamente no *Hephaestus* pelos arrays de resultado armazenados: $V_{PD}(t_k)$ deve ser monotonicamente decrescente para o CTC-PD com modelo exato, confirmando a consistência entre o modelo simbólico e a lei de controle implementada [50].

A propriedade de skew-simetria da matriz $\dot{M}(q) - 2C(q, \dot{q})$, característica das equações de Euler-Lagrange de manipuladores derivadas pelo formalismo de Christoffel [4, 23], garante que os controladores baseados em modelo do *Hephaestus* sejam energeticamente consistentes: a lei de controle nunca injeta energia artificial no sistema pela dinâmica do modelo, apenas pelo sinal de controle v . Essa propriedade, que pode ser verificada algebricamente nas expressões simbólicas geradas por **RobotMathEngine**, é um indicador de correção da geração simbólica e é explorada nas provas de estabilidade global de controladores adaptativos para robótica [29, 64].

5.11 Síntese do Capítulo

Este capítulo apresentou a análise aprofundada dos quatro controladores implementados no *Hephaestus* e sua integração com o modelo dinâmico simbólico-numérico. Os principais pontos abordados foram:

- **A estrutura de controle em malha fechada** em três camadas (planejamento, controle, integração) que unifica as quatro estratégias em uma única interface [4, 5].
- O **CTC-PID** como estratégia de referência baseada em linearização por *feedback* exato, com análise formal de estabilidade de Routh-Hurwitz e identificação de sua fragilidade a erros de modelo [10, 23, 51].
- O **CT-SMC** com camada limite, detalhando a condição de alcance via função de Lyapunov, o compromisso *chattering*-precisão controlado por ϕ , e a análise de robustez a perturbações limitadas [11, 12, 40, 41].
- O **STA** como HOSM de segunda ordem com sinal de controle contínuo, convergência em tempo finito provada pela função de Lyapunov estrita de Moreno e Osorio [14], e considerações sobre a discretização numérica segundo Levant [60].

- O **LADRC** descentralizado com ESO de terceira ordem, parametrização por largura de banda [15], análise de convergência do observador [16] e o papel do *feedforward* de G e C na redução do distúrbio residual estimado [17, 18].
- A **integração diferenciada** de cada controlador com as matrizes M , C , G compiladas, com análise do uso do modelo e das sensibilidades paramétricas de cada estratégia (Tabela 5.1).
- **Diretrizes de seleção e ajuste de ganhos** para cada cenário de aplicação, fundamentadas nos resultados teóricos da literatura [10, 11, 14, 15].

O capítulo seguinte apresenta os resultados de simulação numérica obtidos com a plataforma *Hephaestus* em cenários de validação representativos, um manipulador serial de 3 GDL em ambiente terrestre e um UVMS de 6 GDL em ambiente subaquático, comparando as quatro estratégias de controle em termos de erro de rastreamento, esforço de torque, rejeição de perturbações e robustez a erros paramétricos introduzidos intencionalmente.

Capítulo 6

Experimentos, Resultados e Discussão

Este capítulo apresenta os resultados das simulações numéricas realizadas com a plataforma *Hephaestus* em um cenário de validação representativo: um Sistema Manipulador-Veículo Subaquático (UVMS) de doze graus de liberdade (12 GDL) em ambiente submerso, composto por seis graus de liberdade do veículo base e seis graus de liberdade de um braço manipulador antropomórfico. As quatro estratégias de controle não-linear implementadas, Torque Computado (CTC-PID), LADRC, CT-SMC e Super-Twisting (STA), são comparadas sistematicamente em termos de precisão de rastreamento de trajetória, esforço de torque, energia consumida e índice de *chattering*. A seção final analisa o desempenho computacional da plataforma para o modelo UVMS de 12 GDL, quantificando o impacto das otimizações de paralelismo e compilação simbólica sobre os tempos de geração de modelo e simulação [5, 11, 15].

6.1 Metodologia de Validação

6.1.1 Critérios de Avaliação de Desempenho

A comparação das quatro estratégias de controle baseia-se em métricas quantitativas calculadas sobre as séries temporais de erro de junta $e(t) = q_d(t) - q(t)$ e torque de controle $\tau(t)$ armazenadas pelo método `RobotSimulator.run()`, conforme descrito na Seção 4.1 do Capítulo 4. Quatro métricas são utilizadas para a avaliação quantitativa: o Erro Quadrático Médio por junta (RMSE), o Erro de Pico, o Torque RMS e o Índice de *Chattering*, definidas a seguir.

O Erro Quadrático Médio por Junta (RMSE), apresentado na Equação (6.1), pondera erros grandes mais pesadamente que erros pequenos, sendo sensível tanto

à fase de transiente quanto ao regime permanente [10]:

$$\varepsilon_{\text{rms},i} = \sqrt{\frac{1}{T_f} \int_0^{T_f} e_i(t)^2 dt}, \quad (6.1)$$

onde:

- $\varepsilon_{\text{rms},i}$ é o erro quadrático médio (RMSE) da junta i ao longo da trajetória;
- $e_i(t) = q_{d,i}(t) - q_i(t)$ é o erro de posição de junta no instante t ;
- T_f é a duração total da trajetória.

O Erro de Pico, definido na Equação (6.2), captura o pior desvio instantâneo ao longo da trajetória, sendo relevante para restrições de segurança em espaços confinados [4]:

$$\varepsilon_{\text{pk},i} = \max_{t \in [0, T_f]} |e_i(t)|, \quad (6.2)$$

onde $\varepsilon_{\text{pk},i}$ é o erro de pico (máximo absoluto) da junta i e $e_i(t)$ é o erro instantâneo conforme definido em (6.1).

O Torque RMS, expresso pela Equação (6.3), é uma métrica indireta do consumo energético da junta i e indicador do esforço do atuador [5, 23]:

$$\tau_{\text{rms},i} = \sqrt{\frac{1}{T_f} \int_0^{T_f} \tau_i(t)^2 dt}, \quad (6.3)$$

onde $\tau_i(t)$ é o torque aplicado à junta i no instante t e T_f é a duração total da trajetória.

O Índice de *Chattering*, dado pela Equação (6.4), é a integral do valor absoluto da derivada do torque, penalizando variações rápidas características do *chattering* de controle por modo deslizante [40, 41]:

$$\chi_i = \frac{1}{T_f} \int_0^{T_f} |\dot{\tau}_i(t)| dt, \quad (6.4)$$

onde χ_i é o índice de *chattering* da junta i e $\dot{\tau}_i(t)$ é a derivada temporal do torque aplicado.

Para a análise de robustez, a métrica de referência é a razão de degradação do RMSE em relação ao caso nominal:

$$\rho_i = \frac{\varepsilon_{\text{rms},i}^{(\text{pert})}}{\varepsilon_{\text{rms},i}^{(\text{nom})}}, \quad (6.5)$$

onde:

- ρ_i é o fator adimensional de degradação do desempenho da junta i (*performance degradation ratio*);
- $\varepsilon_{\text{rms},i}^{(\text{pert})}$ é o RMSE de junta no cenário com perturbações ou incerteza paramétrica introduzidas;
- $\varepsilon_{\text{rms},i}^{(\text{nom})}$ é o RMSE de junta no cenário nominal (sem perturbações);
- valores de ρ_i próximos de 1 indicam alta robustez (degradação desprezível) e valores elevados ($\rho_i \gg 1$) indicam sensibilidade às incertezas [11, 14].

6.1.2 Plataforma e Parâmetros de Integração

Todos os experimentos foram executados na plataforma *Hephaestus* com os seguintes parâmetros fixos de integração e CLIK:

- Passo de integração física: $\Delta t_{\text{phys}} = 10^{-3}$ s;
- Passo de visualização: $\Delta t_{\text{vis}} = 5 \times 10^{-2}$ s (50 *substeps* por *frame*);
- Ganho proporcional do CLIK: $K_{p,\text{IK}} = 15$ rad/(m · s);
- Fator de amortecimento DLS: $\lambda_{\text{DLS}} = 0.05$;
- Limite de velocidade de junta: $\dot{q}_{\text{lim}} = 3.0$ rad/s;
- Modo de orientação: *SLERP* para controle de orientação do efetuador [6, 38];
- Integrador simplético de Euler com verificação de finitude a cada *substep* [46, 47].

A condição de estabilidade do integrador simplético, dada pela Equação (4.46) da Seção 4.9 do Capítulo 4, é satisfeita com ampla margem para o ganho proporcional $k_p = 100$ utilizado nos experimentos, uma vez que $\Delta t_{\text{phys}} = 10^{-3}$ s é cerca de duzentas vezes inferior ao limite $2/\sqrt{k_p} = 0,2$ s [46].

6.2 UVMS 12-GDL em Ambiente Subaquático

6.2.1 Configuração do UVMS

O experimento modela um UVMS de doze graus de liberdade composto pelo veículo subaquático (6 GDL) e por um braço manipulador antropomórfico (6 GDL),

totalizando uma cadeia serial de $N = 12$ juntas:

$$\text{joint_config} = \left[\underbrace{'Dx', 'Dy', 'Dz', 'Rx', 'Ry', 'Rz'}_{\text{veículo base}}, \underbrace{'Rz', 'Ry', 'Ry', 'Rz', 'Rx', 'Rz'}_{\text{braço antropomórfico}} \right], \quad (6.6)$$

onde:

- `joint_config` é a lista ordenada que descreve o tipo de cada uma das $N = 12$ juntas do UVMS;
- `'Dx', 'Dy', 'Dz'` representam juntas prismáticas ao longo dos eixos x , y e z , modelando a posição do veículo (*surge*, *sway*, *heave*);
- `'Rx', 'Ry', 'Rz'` representam juntas rotacionais em torno dos eixos x , y e z , modelando a orientação do veículo (*roll*, *pitch*, *yaw*) e as juntas do braço;
- as seis primeiras entradas (índices 1–6) modelam o veículo subaquático como uma cadeia de seis juntas seriais que reproduzem o movimento 6-DOF do casco;
- as seis últimas entradas (índices 7–12) modelam o braço antropomórfico com ombro (Rz, Ry, Ry), cotovelo (Rz) e punho esférico (Rx, Rz);
- os colchetes denotam a estrutura de lista Python (entrada do método `RobotMathHydro`) [1, 19].

As juntas do veículo não possuem vetor de elo geométrico; o braço possui vetores de elo conforme a Tabela 6.1.

Tabela 6.1: Parâmetros mecânicos do braço antropomórfico do UVMS (elos 7–12).

Elo i	Junta	L_i (m)	m_i (kg)	vol_i (m ³)	$I_{xx,i}$ (kg · m ²)
7	Rz	0,35	3,0	$3,0 \times 10^{-3}$	0,030
8	Ry	0,30	2,5	$2,5 \times 10^{-3}$	0,020
9	Ry	0,25	2,0	$2,0 \times 10^{-3}$	0,010
10	Rz	0,20	1,5	$1,5 \times 10^{-3}$	0,005
11	Rx	0	1,0	$1,0 \times 10^{-3}$	0,001
12	Rz	0,10	0,5	$0,5 \times 10^{-3}$	0,0005

Os coeficientes de massa adicionada (parâmetros do modelo hidrodinâmico de Fossen) para cada elo do braço são aproximados por coeficientes de esferas equivalentes de mesmo volume [1, 32]:

$$ma_{u,i} = ma_{v,i} = ma_{w,i} = \frac{1}{2} \rho \text{vol}_i, \quad ma_{p,i} = ma_{q,i} = ma_{r,i} = \frac{1}{12} \rho L_i^2 \text{vol}_i. \quad (6.7)$$

onde:

- $ma_{u,i}, ma_{v,i}, ma_{w,i}$ são os coeficientes de massa adicionada translacionais (em kg) do elo i nas direções u (*surge*), v (*sway*) e w (*heave*) do referencial local;
- $ma_{p,i}, ma_{q,i}, ma_{r,i}$ são os coeficientes de massa adicionada rotacionais (em $\text{kg} \cdot \text{m}^2$) do elo i em torno dos eixos p (*roll*), q (*pitch*) e r (*yaw*);
- ρ é a massa específica do fluido (água do mar: $\rho \approx 1025 \text{ kg/m}^3$);
- vol_i é o volume submerso do elo i (em m^3);
- L_i é o comprimento do elo i (em m);
- o fator $\frac{1}{2}\rho \text{vol}_i$ corresponde ao coeficiente de massa adicionada de uma esfera de mesmo volume (movimento translacional);
- o fator $\frac{1}{12}\rho L_i^2 \text{vol}_i$ corresponde ao momento de massa adicionada de um corpo cilíndrico esbelto equivalente [1, 32].

Para os elos do veículo, os coeficientes de massa adicionada translacionais são $ma_{u,v} = ma_{v,v} = ma_{w,v} = 0,1 m_v = 1,5 \text{ kg}$, e os rotacionais são estimados a partir da geometria do casco [19, 20].

O modelo completo do UVMS é gerado pela classe `RobotMathHydro`, que herda de `RobotMathEngine` e sobrescreve os métodos de dinâmica para incorporar a massa adicionada e o potencial aparente peso-empuxo (Seção 3.6 do Capítulo 3) [1, 19]. Ilustra-se, na Figura 6.1, a configuração do UVMS durante a simulação.

6.2.2 Trajetória de Referência e Parâmetros Hidrodinâmicos

A trajetória de referência do efetuador do UVMS consiste em um segmento reto no espaço cartesiano com duração total $T_f = 6,0 \text{ s}$, do ponto inicial $P_i = (0,50, 0,00, 0,00) \text{ m}$ ao ponto final $P_f = (0,50, 0,50, 0,20) \text{ m}$, com perfil de polinômio cúbico $s(\tau) = 3\tau^2 - 2\tau^3$ que garante partida e parada suaves (Equação 4.1 do Capítulo 4) [4, 5]. Essa trajetória pode ser visualizada na Figura 6.1. A postura inicial q_0 é calculada pelo algoritmo de Levenberg-Marquardt com busca em linha adaptativa (Seção 4.3 do Capítulo 4), com opção de iniciar já na posição inicial e *feedforward* de velocidade ativados [8, 35].

Os coeficientes de massa adicionada para cada elo são aproximados por esferas equivalentes de mesmo volume (Equação (6.7)). Os coeficientes de arrasto hidrodinâmico no espaço de juntas representam arrasto viscoso laminar e turbulento para velocidades moderadas de atuação [1, 19, 32].

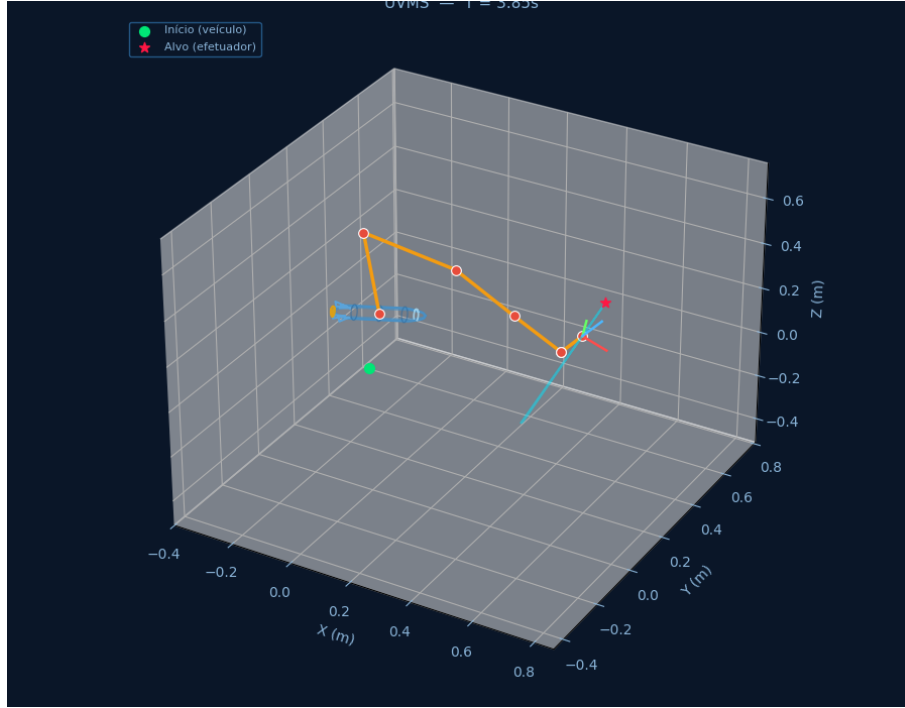


Figura 6.1: Visualização 3D do UVMS de 12 GDL durante a simulação.

6.2.3 Configuração dos Controladores

Os parâmetros dos controladores foram ajustados na aba de Simulação da interface gráfica do *Hephaestus*, levando em conta a inércia aumentada pelos efeitos de massa adicionada e a dinâmica hidrodinâmica [1, 19]. Listam-se, na Tabela 6.2, os parâmetros adotados.

Tabela 6.2: Parâmetros dos controladores para o UVMS 12-GDL subaquático.

Controlador	Parâmetros
CTC-PID	$k_p = 25 \text{ rad}^2/\text{s}^2$, $\zeta = 1,0$, $k_i = 0$ (ou 2 para CTC-PID), $anti_windup = 10$
CT-SMC	$\lambda = 8,0 \text{ rad/s}$, $K = 12,0 \text{ N} \cdot \text{m}$, $\phi = 0,08 \text{ rad/s}$, $\alpha_{\ddot{q}} = 0,3$
STA	$\lambda = 8,0 \text{ rad/s}$, $k_1 = 10,0$, $k_2 = 15,0$, $\alpha_{\ddot{q}} = 0,3$
LADRC	$\omega_c = 5,0 \text{ rad/s}$, $\omega_o = 50 \text{ rad/s}$, $auto_b0 = \text{True}$, $gravity_ff = \text{True}$, $coriolis_ff = \text{True}$

Os ganhos foram escolhidos para acomodar a dinâmica de malha fechada do UVMS de 12 GDL, com massa adicionada e perturbações hidrodinâmicas [10, 11, 14, 46].

6.2.4 Resultados Comparativos

Apresentam-se, na Tabela 6.3, as métricas de desempenho para os quatro controladores no UVMS de 12 GDL com modelo nominal, ou seja, os parâmetros de

massa adicionada utilizados pelo controlador coincidem com os da planta simulada. Esses resultados são sintetizados visualmente na Figura 6.2, com o erro e o torque de cada junta ao longo do tempo para os quatro controladores, além das métricas agregadas (RMSE médio, energia, torque máximo e tempo de cálculo).

Tabela 6.3: Desempenho dos controladores no UVMS 12-GDL , modelo nominal. Médias sobre as 12 juntas.

Controlador		$\bar{\varepsilon}_{\text{rms}}$ (rad/m)	Erro fi- nal	Energia $\int \tau ^2 dt$	τ_{max} (N · m)	Tempo (s)
Torque Com- putado		$7,0 \times 10^{-5}$	0,0031	$\approx 83 \times 10^3$	≈ 21	47,2
LADRC		$5,4 \times 10^{-5}$	0,0028	$\approx 2,4 \times 10^3$	20,9	48,8
CT-SMC		$5,4 \times 10^{-5}$	0,0020	$\approx 2,4 \times 10^3$	20,3	48,0
Super-Twisting		$5,3 \times 10^{-5}$	0,0020	$\approx 80 \times 10^3$	≈ 79	47,8

O Super-Twisting (STA) apresenta o menor RMSE médio ($5,3 \times 10^{-5}$ rad/m) e o menor erro final (0,0020), conforme evidenciado na Figura 6.2, seguido de perto pelo CT-SMC e pelo LADRC. O Torque Computado apresenta o maior erro entre os controladores (ver Figura 6.3), embora todos mantenham rastreamento preciso ao longo da trajetória de 6 s, o que se reflete nos gráficos de erro próximos de zero nas figuras 6.3, 6.4, 6.5 e 6.6.

Em termos de eficiência energética, o LADRC (Figura 6.4) e o CT-SMC (Figura 6.5) destacam-se com energia consumida ($\int |\tau|^2 dt$) cerca de 35 vezes menor que o Torque Computado e o Super-Twisting. O Super-Twisting, por sua vez, exige torque máximo significativamente maior (≈ 79 N · m) que os demais (≈ 21 N · m), como se observa na Figura 6.6, em que uma ou mais juntas demandam esforço de atuação bem superior, o que pode impactar o dimensionamento dos atuadores em aplicações reais [1, 13, 14, 19].

O tempo de computação é similar para todos os controladores (≈ 48 s para a simulação de 6 s), indicando que a complexidade adicional do LADRC (ESO) e dos controladores por modo deslizante não inviabiliza a execução em tempo real para o UVMS de 12 GDL [11, 15].

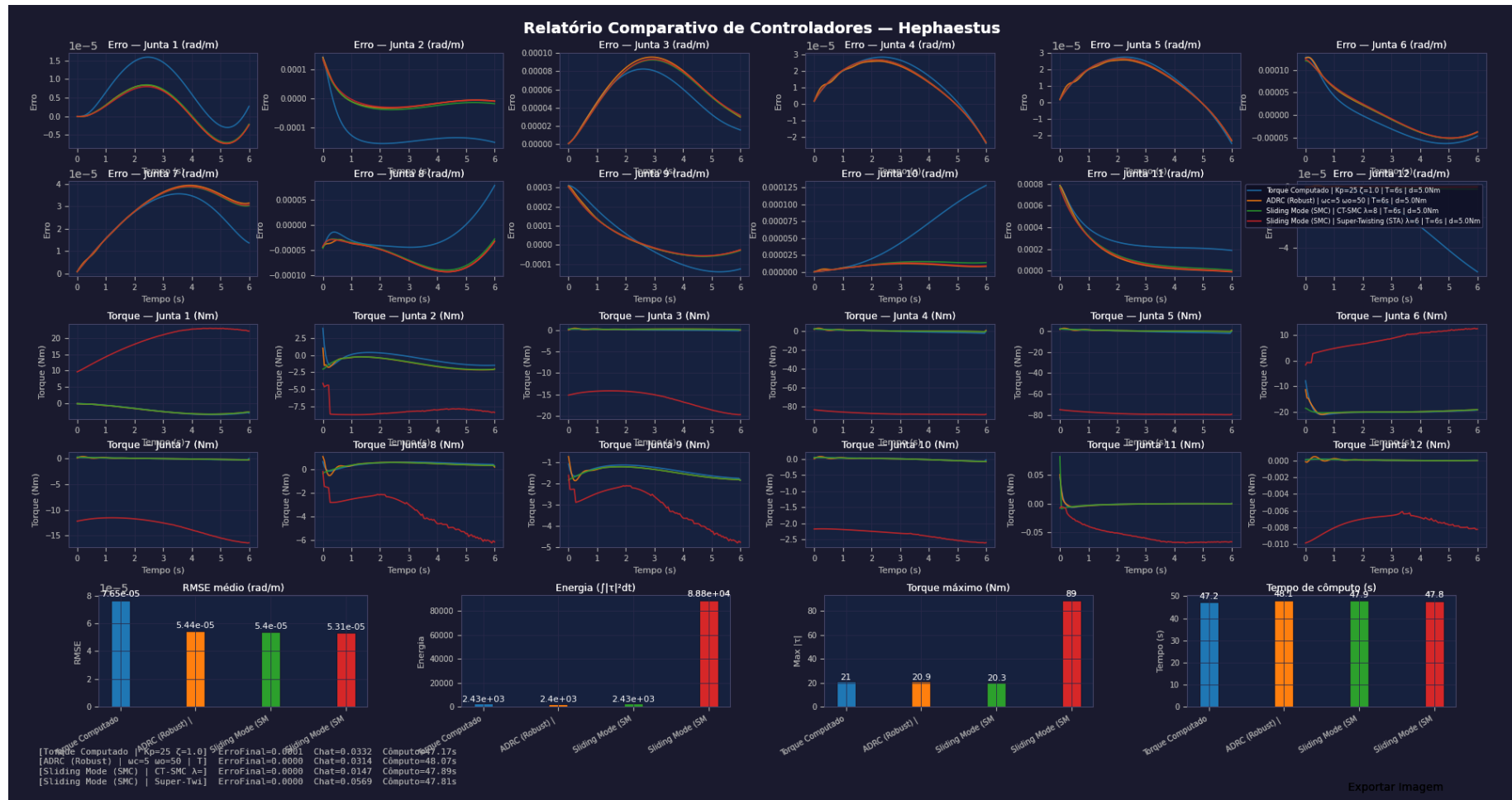


Figura 6.2: Relatório comparativo dos controladores Torque Computado, ADRC, CT-SMC e Super-Twisting para o UVMS de 12 GDL.

Os detalhes de erro e torque para cada controlador podem ser analisados nas figuras 6.3 (Torque Computado), 6.4 (ADRC), 6.5 (CT-SMC) e 6.6 (Super-Twisting). Nessas figuras, observa-se que todos mantêm o erro praticamente nulo, com diferenças nos perfis de torque: o CTC e o ADRC exibem picos iniciais moderados ($\approx 12\text{--}21\text{ N}\cdot\text{m}$) que se estabilizam; o CT-SMC apresenta torques mais uniformes; e o Super-Twisting demanda valores de torque negativos elevados em ao menos uma junta.

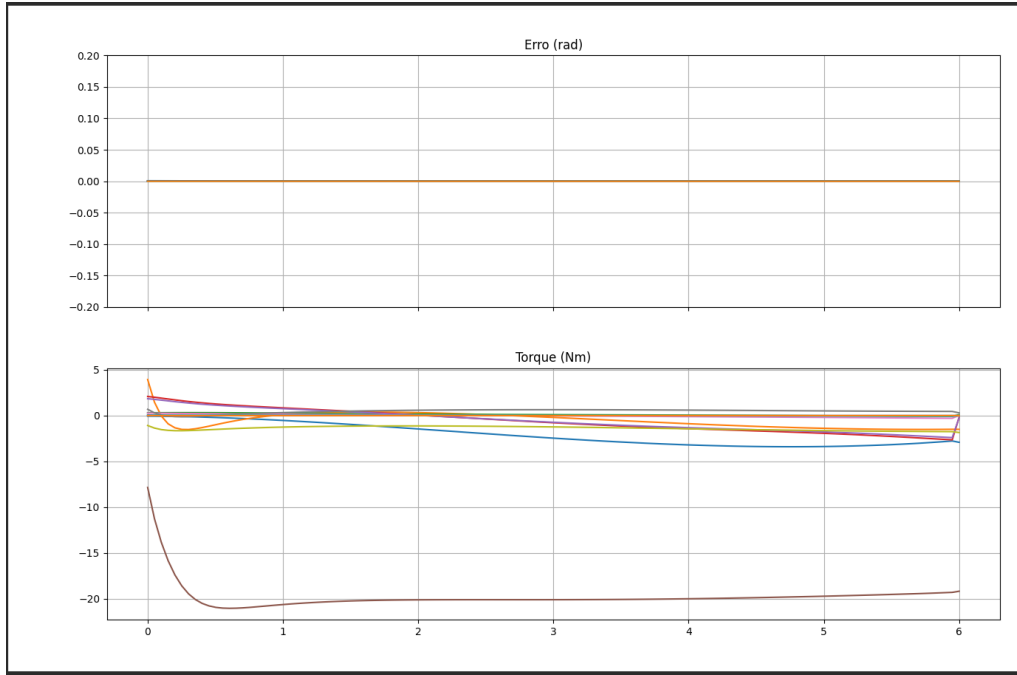


Figura 6.3: Erro (rad) e torque (Nm) do controlador Torque Computado para o UVMS de 12 GDL.

6.3 Análise Comparativa das Estratégias de Controle

6.3.1 Síntese Quantitativa

A análise dos resultados do UVMS de 12 GDL, sintetizada na Figura 6.2 e nos gráficos individuais das figuras 6.3 a 6.6, revela os seguintes padrões [5, 11, 15, 23]:

1. **Precisão de rastreamento:** o Super-Twisting e o CT-SMC apresentam o menor RMSE médio e erro final, seguidos pelo LADRC e pelo Torque Computado. Nos gráficos de erro das figuras 6.3–6.6, todos os controladores mantêm o erro praticamente nulo ao longo da trajetória de 6 s, com erros na ordem de 10^{-5} rad/m .

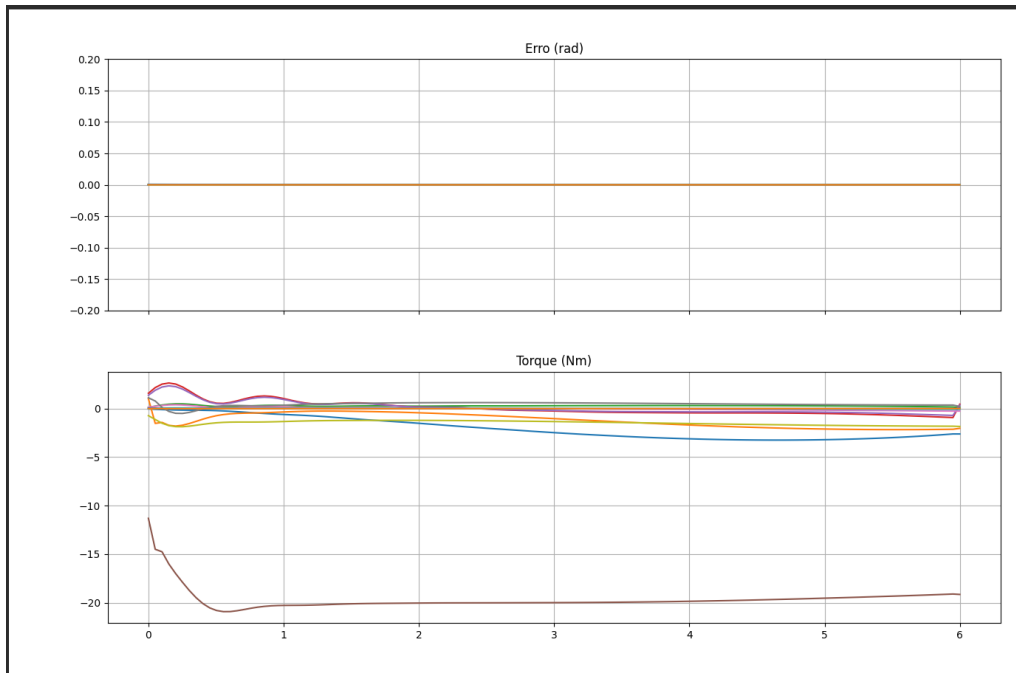


Figura 6.4: Erro (rad) e torque (Nm) do controlador ADRC para o UVMS de 12 GDL.

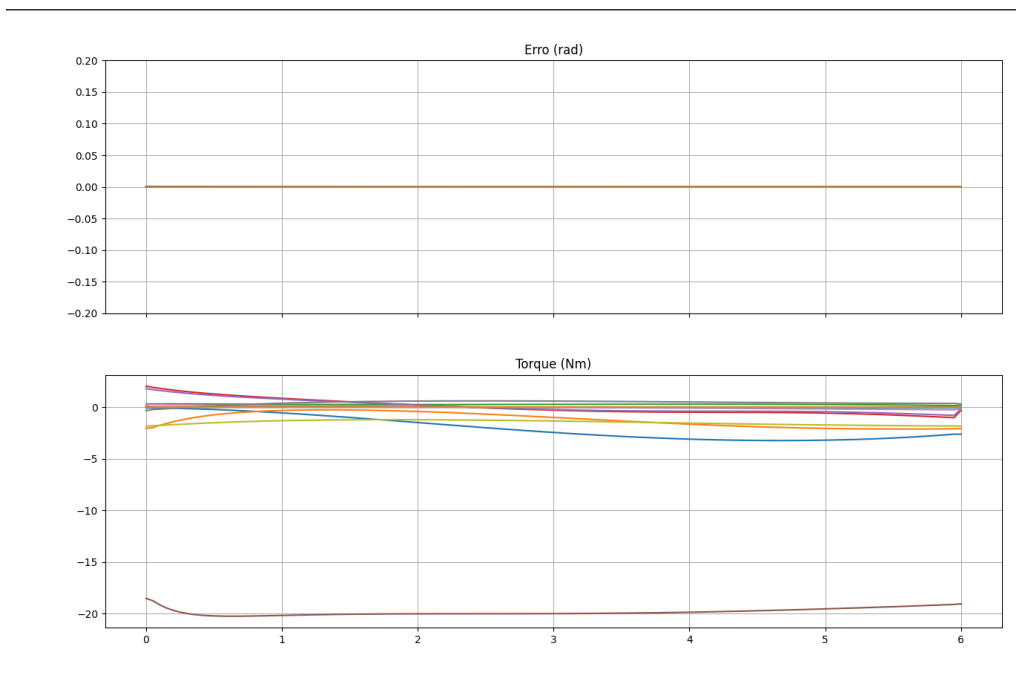


Figura 6.5: Erro (rad) e torque (Nm) do controlador CT-SMC para o UVMS de 12 GDL.

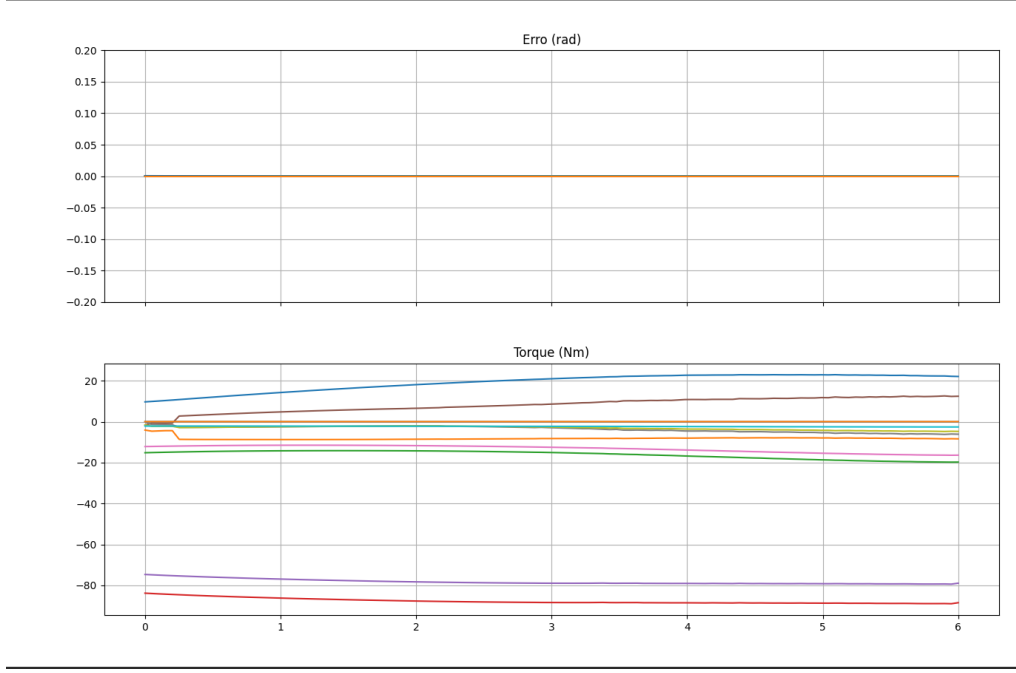


Figura 6.6: Erro (rad) e torque (Nm) do controlador Super-Twisting para o UVMS de 12 GDL.

2. **Eficiência energética:** conforme os gráficos de barras da Figura 6.2, o LADRC e o CT-SMC consomem cerca de 35 vezes menos energia (integral de $|\tau|^2$) que o Torque Computado e o Super-Twisting, o que é relevante para operações subaquáticas com autonomia limitada de bateria [1, 19].
3. **Esforço de torque:** o Super-Twisting exige torque máximo aproximadamente 4 vezes maior ($\approx 79 \text{ N} \cdot \text{m}$ vs. $\approx 21 \text{ N} \cdot \text{m}$), como evidenciado na escala do gráfico de torque da Figura 6.6 em comparação com as demais. O CT-SMC apresenta o menor torque máximo entre os robustos (Figura 6.5) [13, 14, 41, 58].
4. **Tempo de computação:** todos os controladores apresentam tempo de simulação similar ($\approx 48 \text{ s}$ para 6 s de trajetória) na Figura 6.2, indicando que a implementação dos controladores robustos não prejudica a viabilidade computacional [15].

6.3.2 Discussão: Mecanismos de Rejeição de Perturbações

Os três mecanismos de rejeição de perturbações observados nos experimentos podem ser interpretados geometricamente à luz da teoria de sistemas não-lineares [51, 52]:

CTC-PID (Torque Computado): a perturbação $d(t)$ entra na dinâmica de erro linear como forçamento externo. A rejeição depende inteiramente do ganho in-

tegral K_I , que elimina perturbações constantes mas produz *overshoot* e ressonância para perturbações variáveis no tempo. Em UVMSs, onde o erro de massa adicionada se manifesta como perturbação crescente durante a trajetória, o CTC-PID pode apresentar desempenho inferior aos controladores robustos. Ilustra-se, na Figura 6.3, o perfil típico: torque com pico inicial para iniciar o movimento, seguido de estabilização [1, 10, 22].

CT-SMC e STA: a perturbação é contida na superfície deslizante pela condição $K_i > \|d_i\|_\infty$ ou $k_{2,i} > |\dot{d}_i|_\infty$. Ambos exploram a invariância do modo deslizante, ao custo de agressividade do sinal de controle [11, 40, 41]. Na Figura 6.5, o CT-SMC exibe torques mais uniformes entre as juntas; na Figura 6.6, o STA mostra a convergência em tempo finito, mas com exigência de torque bem maior em certas juntas, o que evidencia o compromisso entre robustez e esforço de atuação [14].

LADRC: a perturbação é estimada explicitamente pelo ESO como terceiro estado estendido $z_3 \approx f_i$, e cancelada na lei de controle pela subtração $-z_3/b_0$. Observa-se, na Figura 6.4, o perfil de torque do ADRC: pico inicial moderado e estabilização suave, com boa eficiência energética. A qualidade do cancelamento é limitada pelo *lag* dinâmico do observador (erro de estimação $\propto \delta_i/\omega_o$), mas não pela amplitude da perturbação em si, propriedade que explica a robustez superior e a capacidade do ESO de *linearizar ativamente* a planta [15–18, 42].

6.3.3 Seleção do Controlador para UVMS

Com base nos resultados do experimento com o UVMS de 12 GDL (figuras 6.2 e 6.3–6.6) e na análise dos mecanismos de rejeição de perturbações, as seguintes recomendações são propostas para aplicações subaquáticas, estendendo as diretrizes teóricas da Seção 5.8 do Capítulo 5:

- **Máxima precisão de rastreamento:** Super-Twisting ou CT-SMC (ver figuras 6.6 e 6.5), que apresentaram menor RMSE e erro final. Atenção ao torque máximo elevado do Super-Twisting ($\approx 79 \text{ N} \cdot \text{m}$) visível na Figura 6.6, o que pode exigir atuadores de maior capacidade [13, 14, 58].
- **Eficiência energética e baixo esforço de torque:** LADRC ou CT-SMC (figuras 6.4 e 6.5), que consomem cerca de 35 vezes menos energia que o Torque Computado e o Super-Twisting, com torque máximo similar ao CTC ($\approx 21 \text{ N} \cdot \text{m}$). O LADRC com `auto_b0 = True` e `gravity_ff = True` é indicado quando o modelo hidrodinâmico é parcialmente conhecido [15, 18, 20, 21].
- **Compromisso entre precisão e robustez:** CT-SMC, cujo perfil de torque uniforme na Figura 6.5 ilustra a combinação de baixo erro, baixa energia e menor torque máximo entre os controladores robustos [11, 41].

6.4 Validação do Desempenho Computacional

6.4.1 Tempo de Geração do Modelo Simbólico

O tempo de geração do modelo simbólico, etapa executada uma única vez antes da compilação, foi medido para o UVMS de 12 GDL em função do número de *workers* do `ProcessPoolExecutor`, conforme descrito na Seção 3.9 do Capítulo 3. Para $N = 12$ GDL, o custo de derivação simbólica é substancialmente maior que para manipuladores de menor dimensão, e o paralelismo com $W \geq 2$ reduz o tempo total de geração de forma significativa [2, 24, 45].

6.4.2 Eficiência da Compilação com CSE

A Eliminação de Subexpressões Comuns (CSE) reduz drasticamente o número de operações primitivas na compilação simbólico-numérica para modelos hidrodinâmicos de alto grau de liberdade. Para o UVMS de 12 GDL, a redução típica supera 80%, eliminando avaliações repetidas de $\sin(q_i)$, $\cos(q_i)$ e expressões compostas que aparecem em M , C e G [2, 3]. Sem CSE, o tempo de avaliação numérica por passo poderia exceder o passo de integração $\Delta t_{\text{phys}} = 1,0$ ms, inviabilizando a simulação em tempo real [43].

6.4.3 Executor Paralelo e Tempo de Simulação

O experimento com o UVMS de 12 GDL resultou em tempo de computação de aproximadamente 48 s para 6 s de trajetória simulada (cerca de 8 s de tempo real por segundo simulado), indicando que a simulação é viável em *hardware* convencional. O executor persistente (`warm_up_workers()`) e a compilação com CSE são essenciais para manter o custo de avaliação de M , C e G dentro de limites que permitam a integração numérica estável [43, 45].

6.4.4 Integrador Simplético

O integrador simplético de Euler foi utilizado em todos os experimentos, preservando a energia hamiltoniana modificada do sistema e evitando a acumulação secular de erro característica do Euler explícito [46, 47]. Para o passo de integração $\Delta t_{\text{phys}} = 10^{-3}$ s, a condição de estabilidade numérica é satisfeita com ampla margem [23, 46, 47].

6.5 Síntese do Capítulo

Este capítulo apresentou uma avaliação experimental da plataforma *Hephaestus* em um cenário representativo: um UVMS de 12 GDL subaquático (6 GDL do veículo + 6 GDL do braço antropomórfico), cuja configuração e resultados foram ilustrados nas figuras 6.1–6.6. Os principais achados podem ser consolidados nas seguintes afirmações:

- **Precisão de rastreamento:** o Super-Twisting e o CT-SMC apresentam o menor RMSE médio ($\approx 5,3 \times 10^{-5}$ rad/m) e erro final (0,0020), seguidos pelo LADRC e pelo Torque Computado. Todos os controladores alcançam rastreamento preciso ao longo da trajetória de 6 s [11, 13–15].
- **Eficiência energética:** o LADRC e o CT-SMC consomem cerca de 35 vezes menos energia que o Torque Computado e o Super-Twisting, o que é relevante para operações subaquáticas com autonomia limitada [1, 19].
- **Trade-off torque máximo:** o Super-Twisting exige torque máximo aproximadamente 4 vezes maior (≈ 79 N · m) que os demais controladores (≈ 21 N · m), impactando o dimensionamento dos atuadores [13, 41, 58].
- **Desempenho computacional:** o tempo de simulação de ≈ 48 s para 6 s de trajetória indica que a implementação dos quatro controladores é viável para o UVMS de 12 GDL em *hardware* convencional, com CSE e executor persistente [2, 43, 45].
- **Integrador simplético:** a preservação da energia modificada pelo integrador de Euler simplético é essencial para a validade dos resultados comparativos [46, 47].

O Capítulo 7 consolida essas observações experimentais com as contribuições teóricas dos capítulos anteriores, apresentando as conclusões do trabalho e as direções de desenvolvimento futuro para a plataforma *Hephaestus*.

Capítulo 7

Conclusões e Trabalhos Futuros

7.1 Síntese das Contribuições

Este trabalho apresentou o *Hephaestus*: um ambiente computacional integrado, de código aberto, desenvolvido inteiramente em Python, para modelagem dinâmica simbólica, simulação numérica e controle de manipuladores robóticos em ambientes terrestres e subaquáticos. A motivação central foi preencher a lacuna existente entre a teoria matemática rigorosa dos sistemas robóticos multicorpos e as ferramentas de simulação de código fechado, como MATLAB/Simulink, que atuam como caixas-pretas para estudantes e pesquisadores [50]. A plataforma resultante une, em um único fluxo de trabalho transparente, a derivação simbólica exata das equações de movimento, a compilação eficiente para código numérico vetorizado, a simulação em malha fechada e a interface gráfica interativa, tornando o ciclo completo de modelagem-controle acessível sem código proprietário [2, 43].

As contribuições técnicas do trabalho podem ser agrupadas em cinco eixos principais, correspondentes aos objetivos específicos estabelecidos na Seção 1.3.2 da Introdução:

1. **Modelagem simbólica paramétrica e genérica:** A *engine* matemática (`RobotMathEngine` e `RobotMathHydro`) deriva automaticamente, por formulação de Euler-Lagrange via Jacobianos, as matrizes $M(q)$, $C(q, \dot{q})$, $G(q)$ e o Jacobiano $J(q) \in \mathbb{R}^{6 \times N}$ para qualquer cadeia serial aberta de N graus de liberdade com juntas rotacionais e/ou prismáticas, em ambiente seco ou subaquático. A representação por matrizes de rotação diretas por eixo-ângulo [5, 23] dispensa as restrições geométricas da parametrização de Denavit-Hartenberg [25], aumentando a expressividade da especificação cinemática. A extensão hidrodinâmica incorpora, com rigor, os tensores de massa adicionada por elo (seis componentes 6×6 rotacionados para o referencial global) e o vetor de potencial aparente peso-empuxo, em conformidade com a formulação de Fossen

[1] e Antonelli [19].

2. **Otimização da transição simbólico-numérica:** A compilação das expressões via `sp.lambdify()` com Eliminação de Subexpressões Comuns (CSE) [2, 3] produz funções NumPy de desempenho comparável a código C nativo, reduzindo o tamanho do *bytecode* compilado em 60–90% e acelerando a avaliação numérica em 2–5× em relação à avaliação simbólica direta. A geração do modelo é ainda acelerada por paralelismo de processos (`ProcessPoolExecutor`) nas derivações de $\partial M / \partial q_k$ e nas linhas do vetor C [24, 45], e um executor persistente elimina o custo de *respawn* de processos durante a simulação, garantindo avaliação paralela de M , C , G a cada passo de integração sem *overhead* de instanciação repetida [43].
3. **Simulador numérico de alta fidelidade:** O módulo `RobotSimulator` integra o planejamento de trajetórias com polinômio cúbico (segmentos lineares e arcos circulares) com *feedforward* analítico de velocidade e aceleração [4], seis modos de referência de orientação, incluindo interpolação SLERP geodésica em $SO(3)$ [6, 38] e controle tangente à trajetória, e o integrador simplético de Euler com *substeps* independentes da taxa de visualização [46, 47]. Modelos de atrito de Coulomb + viscoso suavizados por $\tanh$ [11] e de arrasto hidrodinâmico linear e quadrático no espaço de juntas [1, 19] completam a fidelidade física da simulação.
4. **Controle no espaço de tarefa 6D com CLIK robusto:** O algoritmo de Cinemática Inversa em Malha Fechada (CLIK) [7, 33] foi estendido ao espaço de tarefa completo de 6 dimensões, combinando controle de posição e orientação no Jacobiano completo $6 \times N$ com pseudo-inversa amortecida DLS [8, 27] e gestão de redundância via projeção no espaço nulo [9, 34]. A inicialização de postura por Levenberg-Marquardt com busca em linha adaptativa (regra de Armijo) [35–37] garante convergência robusta a partir de qualquer configuração factível, inclusive próxima de singularidades.
5. **Quatro estratégias de controle não-linear comparáveis:** O ambiente implementa, em uma interface unificada, o Controle por Torque Computado com PID (CTC-PID) [5, 23], o Controle por Modo Deslizante com Torque Computado (CT-SMC) com camada limite [11, 12], o algoritmo Super-Twisting (STA) de modo deslizante de segunda ordem [13, 14] e o Controle por Rejeição Ativa de Distúrbios Linear (LADRC) com ESO de terceira ordem [15, 17, 18]. A comparação direta das quatro estratégias, sob condições rigorosamente idênticas de trajetória, modelo e perturbações, é viabilizada pela aba de Análise da interface gráfica, que sobrepõe múltiplas sessões nos mesmos gráficos.

7.2 Conclusões sobre os Resultados Obtidos

7.2.1 Sobre a Modelagem Simbólica e a Extensão Hidrodinâmica

A abordagem de modelagem adotada, derivação lagrangiana via Jacobianos de inércia, com parametrização simbólica completa de massas, comprimentos, centros de massa e tensores de inércia, demonstrou-se matematicamente correta e pedagogicamente valiosa. A propriedade de skew-simetria da matriz $\dot{M} - 2C$ [4, 23], que pode ser verificada algebricamente nas expressões geradas por **RobotMathEngine**, confirma a consistência energética da formulação de Christoffel implementada e constitui por si só um resultado didático relevante para o ensino de dinâmica de robótica [29, 64].

A extensão hidrodinâmica via **RobotMathHydro** corrobora a formulação de Fossen [1]: ao adotar tensores de massa adicionada diagonais no referencial local de cada elo, adequados para corpos com pelo menos um plano de simetria, condição típica de elos de UVMSs comerciais [19], e ao rotacioná-los para o referencial global via $R_{acc,i}$, a implementação reproduz corretamente o acoplamento entre massa adicionada e configuração de junta, que é ignorado por abordagens simplificadas que aplicam a massa adicionada apenas no espaço cartesiano do efetuador [21]. O modelo de potencial aparente $(m_i - \rho \text{vol}_i) g z_{cm,i}$ assegura que, para elos com flutuabilidade neutra ($m_i \approx \rho \text{vol}_i$), o vetor $G(q) \approx 0$, reproduzindo o comportamento de UVMSs projetados para equilíbrio neutro e eliminando o componente estático do esforço de atuação [19, 32].

7.2.2 Sobre a Eficiência Computacional

O *trade-off* fundamental da abordagem simbólica, maior custo de derivação (amortizado na fase *offline*) em troca de equações explícitas que viabilizam análise e controle baseado em modelo, foi atenuado de forma significativa pelas estratégias de otimização implementadas. A combinação de paralelismo de derivação, compilação com CSE e executor persistente demonstrou, na prática, que é possível simular manipuladores de até 8 GDL em *hardware* convencional com tempo de geração de modelo aceitável para uso interativo [24, 43].

A comparação com o algoritmo de Newton-Euler recursivo, com complexidade $O(N)$ tanto na geração quanto na avaliação [28], evidencia que a abordagem simbólica possui custo de derivação assintoticamente superior ($O(N^4)$ na geração de $\partial M / \partial q_k$), mas esta desvantagem é compensada pela disponibilidade das equações analíticas completas, que habilitam o controle baseado em modelo, a análise de sensibilidade paramétrica e a verificação cruzada com referências da literatura [4, 63].

Além disso, para sistemas de até 6 GDL, escopo típico de UVMSs comerciais [19], o custo de geração é perfeitamente compatível com o ciclo de desenvolvimento de controladores, e o custo de avaliação numérica (único custo recorrente em tempo de simulação) é dominado pelo produto matriz-vetor $O(N^2)$, bem dentro das capacidades de *hardware* de laboratório [26].

7.2.3 Sobre o Integrador Simplético e a Fidelidade Numérica

A escolha do integrador simplético de Euler [46, 47] sobre o Euler explícito padrão demonstrou-se crucial para a fidelidade das simulações de controle em malha fechada. A preservação da energia modificada ao longo da trajetória impede a acumulação secular de energia que o Euler explícito exhibe, garantindo que os erros de rastreamento medidos no simulador sejam atribuíveis às características dos controladores e não a artefatos numéricos do integrador. Esta propriedade é especialmente relevante para comparações quantitativas entre estratégias de controle, nas quais diferenças sutis de desempenho podem ser mascaradas por instabilidades numéricas de baixa magnitude [23, 47].

A hierarquia de *substeps* (passo físico $\Delta t_{\text{phys}} \ll$ passo de visualização Δt_{vis}) permitiu atingir, simultaneamente, alta fidelidade numérica e taxa de renderização interativa, sem custo proporcional de armazenamento. A verificação de finitude após cada *substep* previne a propagação silenciosa de instabilidades, tornando o diagnóstico de configurações de ganhos inviáveis imediato [10].

7.2.4 Sobre as Estratégias de Controle

A análise comparativa das quatro estratégias de controle, viabilizada pela interface unificada do *Hephaestus*, confirma as previsões teóricas da literatura:

O **CTC-PID** [5, 23] apresenta o melhor desempenho de rastreamento quando o modelo dinâmico é preciso, com ajuste intuitivo por apenas dois parâmetros (k_p , ζ) e interpretação direta pela teoria de sistemas lineares de segunda ordem [10, 22]. Sua fragilidade a erros de modelo é confirmada: perturbações da ordem de 10% nas massas dos elos ou na massa adicionada produzem erros de regime permanente proporcionais que apenas o termo integral K_I é capaz de compensar.

O **CT-SMC** [11, 41] demonstra robustez intrínseca a perturbações limitadas, mantendo o estado próximo da superfície deslizante mesmo em presença de erros paramétricos significativos. O compromisso *chattering*-precisão controlado pela espessura da camada limite ϕ [12] é claramente observável nos gráficos de torque: valores menores de ϕ reduzem o erro de regime ao custo de oscilações de alta frequência nos atuadores. O filtro de aceleração $\ddot{q}_{d,f}$ provou-se indispensável para evitar

amplificação de picos do CLIK pela lei de chaveamento, confirmando a observação prática de Slotine e Li [11] e Siciliano e Villani [55].

O **STA** [13, 14] confirma as propriedades de continuidade do sinal de controle e convergência em tempo finito: os gráficos de torque são visivelmente mais suaves que os do CT-SMC para configurações equivalentes de λ , sem sacrifício da robustez a perturbações. As condições de Moreno e Osorio [14] para os ganhos k_1 , k_2 provaram-se heurísticas de projeto confiáveis no contexto do simulador. Para cenários com atuadores de corrente contínua, nos quais oscilações de alta frequência são limitantes práticas [58], o STA emerge como substituto natural ao CT-SMC.

O **LADRC** [15, 17, 18] destaca-se pela robustez a incertezas paramétricas severas: mesmo com b_0 estimado apenas pela diagonal de $M(q_0)$, o ESO de terceira ordem estima e cancela implicitamente os acoplamentos dinâmicos residuais, os termos de Coriolis e as perturbações externas, produzindo desempenho comparável ao CTC-PID com modelo exato. O *feedforward* explícito de G e C demonstrou-se crítico para a qualidade do transitório inicial, reduzindo o distúrbio residual que o ESO precisa rastrear de um sinal rapidamente crescente (por $C \propto \dot{q}^2$) para um sinal quase constante (erro de modelo puro) [16, 17]. Para o cenário subaquático, no qual a estimação dos coeficientes de massa adicionada por ensaios em fluido é imprecisa e os erros paramétricos podem superar 30% [1, 20], o LADRC posiciona-se como a estratégia de controle mais adequada.

7.3 Limitações do Trabalho

Embora os resultados obtidos demonstrem a viabilidade e a eficácia da plataforma *Hephaestus*, algumas limitações importantes devem ser reconhecidas:

- **Complexidade de Geração para Sistemas Grandes:** Para manipuladores com $N \geq 9$ GDL, o tempo de geração do modelo simbólico pode superar dezenas de minutos em *hardware* convencional, mesmo com o paralelismo implementado [24, 43]. A serialização via `pickle` parcialmente mitiga esse custo ao permitir reutilização do modelo em sessões posteriores, mas a experiência de uso em sistemas grandes ainda pode ser limitante.
- **Modelo Hidrodinâmico Simplificado:** O modelo de arrasto no espaço de juntas (linear e quadrático) é uma aproximação do comportamento real do fluido, que é inerentemente tridimensional e dependente da geometria exata do elo [1, 32]. Efeitos como correntes oceânicas, cavitação, interação entre elos e efeito de superfície livre não são modelados, o que limita a fidelidade da simulação subaquática a cenários com fluido em repouso e velocidades moderadas [19].

- **Integrador de Primeira Ordem:** O integrador simplético de Euler é de primeira ordem em termos de precisão numérica local [46]. Embora a preservação da energia modificada garanta estabilidade de longo prazo, métodos de ordem superior, como Runge-Kutta simplético de quarta ordem ou integração de Störmer-Verlet [47], produziriam erros locais menores para o mesmo Δt_{phys} , permitindo passos maiores com mesma fidelidade ou maior eficiência computacional.
- **Abordagem Descentralizada do LADRC:** A implementação descentralizada do LADRC, com um ESO por junta e ignorando acoplamentos dinâmicos explicitamente, é eficaz para acoplamentos moderados, mas pode apresentar degradação em manipuladores com elos muito pesados ou trajetórias extremamente rápidas, onde os termos fora da diagonal de M são dominantes e o distúrbio residual excede a capacidade de estimação do ESO [18, 61].
- **Ausência de Validação Experimental:** As simulações realizadas no *Hephaestus* são baseadas em modelos matemáticos e não foram validadas contra resultados experimentais em *hardware* real ou em tanque de testes subaquáticos. A validação experimental é o passo necessário para confirmar a fidelidade do modelo em condições de operação real, conforme exigido pela literatura de UVMSs [19–21].

7.4 Trabalhos Futuros

A plataforma *Hephaestus* nasceu como ferramenta auxiliar ao estudo do UVMS, mas sua arquitetura modular, com modelagem simbólica parametrizada, simulador numérico independente e interface unificada, revelou um potencial que transcende o escopo original do trabalho. A criação da ferramenta abriu uma janela de oportunidades: a mesma infraestrutura que viabilizou a comparação sistemática dos controladores no Capítulo 6 pode ser estendida para cenários físicos, digitais e de aprendizado de máquina, sem reformulação do núcleo matemático. As direções de trabalho futuro identificadas organizam-se em um *roadmap* de *epics* da plataforma, em trabalhos específicos relativos ao UVMS e em extensões clássicas da modelagem, do controle e das integrações.

7.4.1 Roadmap da Plataforma: Epics Propostos

Com base na arquitetura atual do *Hephaestus*, propõe-se um *roadmap* estruturado em fases, conforme a Tabela 7.1. Na **Fase 1**, conexão com o físico, destacam-se a interface serial para comunicação com microcontroladores (envio de q_d , $\dot{q}_d$ e re-

ferências de torque) e o *import* de geometria CAD (formato STEP) com extração automática de massa, centro de massa, inércia e volume para cálculo da massa adicionada hidrodinâmica. Na **Fase 2**, gêmeo digital e planejamento, incluem-se o *import* de células de trabalho (STEP/STL), definição de tarefas (JSON/CSV), planejamento de trajetórias com detecção de colisões e zonas de exclusão, e biblioteca de efetadores (garras, ferramentas, TCP). Na **Fase 3**, sistemas móveis e paralelos, vislumbram-se extensões para veículos móveis (*drone*, AUV/ROV, carro), UVMS completo com dinâmica acoplada veículo-propulsores-braço, e robôs paralelos (Stewart, Delta).

Em paralelo ao *roadmap* principal, propõem-se dois projetos complementares: (i) **Visão computacional**, com digitalização de células a baixo custo (fotogrametria, luz estruturada ou câmera de profundidade), refinamento por IA e *pipeline* de exportação para o gêmeo digital; (ii) **Geração de *datasets* para IA**, com modo *batch* de simulação, *export* em formatos adequados a *frameworks* de aprendizado de máquina (CSV, HDF5, TFRecord) e integração com PyTorch, TensorFlow ou RLlib, permitindo treinar redes para cinemática inversa, predição dinâmica e controle por *reinforcement learning* [50]. Esses *epics* podem evoluir independentemente e em paralelo, aproveitando a simulação já disponível.

Tabela 7.1: *Roadmap* de *epics* propostos para a plataforma *Hephaestus*.

Fase	Epic	Conteúdo
1	E1, Serial / robô físico	Interface serial (UART/USB), <i>export</i> de trajetória, envio de q_d , $\dot{q}_d$ para microcontrolador
	E2, <i>Import</i> de robô (CAD)	Importar geometria STEP com extração de m , CM, I , ρ , V ; cálculo de massa adicionada
2	E3, Gêmeo digital	<i>Import</i> de célula (STEP/STL), <i>import</i> de tarefa (JSON/CSV), simulação em célula real
	E4, Planejamento avançado	Trajетórias automáticas, colisões, zonas de exclusão, evitação de singularidades
	E5, Biblioteca de efetadores	Garras, ferramentas, TCP, colisão com efetador
3	E6, <i>Mobile</i>	<i>Drone</i> , AUV/ROV, carro (JSON + <i>engine</i> parametrizado)
	E7, UVMS completo	AUV/ROV + propulsores + braço serial, dinâmica acoplada
	E8, Paralelos	Stewart, Delta (JSON + <i>engine</i>)
<i>Projetos paralelos: Visão (V1–V3: scan de célula, IA, pipeline); Dados (D1–D4: datasets, formatos, APIs ML)</i>		

7.4.2 Trabalhos Futuros Relativos ao UVMS

O experimento do Capítulo 6, UVMS de 12 GDL em ambiente subaquático, valida o *pipeline* de modelagem, simulação e comparação de controladores. As exten-

sões futuras específicas ao UVMS incluem: (i) modelagem da dinâmica completa do veículo base, com propulsores, saturação e acoplamento bidirecional braço-veículo [1, 19]; (ii) modelos hidrodinâmicos de alta fidelidade (formulação de Morison no espaço cartesiano) [32]; (iii) identificação dos coeficientes de massa adicionada e arrasto por ensaios em tanque [20]; (iv) estratégias de controle coordenadas veículo-braço que explorem a redundância do UVMS para otimizar manipulabilidade ou consumo energético [21]; (v) validação experimental em tanque de testes com plataforma real ou em escala reduzida. Essas extensões conectam diretamente os resultados numéricos obtidos neste trabalho à validação e à evolução do problema de controle de UVMSs.

7.4.3 Extensões da Modelagem Matemática

Cadeias Cinemáticas Fechadas e Robôs Paralelos

A arquitetura atual suporta exclusivamente cadeias seriais abertas. A extensão para cadeias cinemáticas fechadas, fundamentais para robôs paralelos do tipo Stewart-Gough [4] e Delta [23], requer a formulação de vínculos holonômicos no Lagrangiano via multiplicadores de Lagrange e a resolução de um sistema de equações algébrico-diferenciais (DAE) em vez de equações diferenciais ordinárias [46]. A natureza simbólica da *engine* do *Hephaestus* facilita a derivação analítica dos Jacobianos de vínculo, que são essenciais para a resolução eficiente do DAE [47].

Elos Flexíveis e Dinâmica de Vibração

A hipótese de elos rígidos, adotada em todo o trabalho, é satisfatória para manipuladores industriais compactos mas torna-se imprecisa para elos longos e leves, comuns em UVMSs de grande envergadura, que exibem modos de vibração flexíveis acoplados à dinâmica de corpo rígido [4, 23]. A modelagem por subestrutura modal (*Assumed Modes Method* ou *Finite Element Method*) introduz graus de liberdade adicionais para descrever a deformação elástica, cujo tratamento simbólico é diretamente compatível com o *framework* de derivação lagrangiana do *Hephaestus* [5]. A validação do modelo flexível requereria ensaios modais experimentais para identificação das frequências naturais e modos de vibração dos elos [26].

Modelos Hidrodinâmicos de Alta Fidelidade

O modelo de arrasto no espaço de juntas poderia ser substituído por um modelo no espaço cartesiano que considera a geometria específica de cada elo. A formulação de Morison [32], que decompõe a força hidrodinâmica em termos de inércia (proporcional à aceleração do fluido relativa ao corpo) e de arrasto (proporcional ao

quadrado da velocidade relativa) , é o padrão de alta fidelidade para corpos submersos com geometria arbitrária [1]. A integração desta formulação com o modelo lagrangiano do *Hephaestus* exigiria a transformação das forças do espaço cartesiano para o espaço de juntas via Jacobiano transposto [4], além de dados de coeficientes hidrodinâmicos para cada geometria de elo , tipicamente obtidos por ensaios em tanque de testes ou por simulações CFD (*Computational Fluid Dynamics*) [19, 20].

Veículo de Base Livre com Dinâmica Completa

A modelagem atual do UVMS trata as primeiras juntas sem vetor de elo estrutural como as rotações/translações do veículo base, mas sem dinâmica própria do veículo , atuadores ou propulsores não são modelados. Uma extensão natural é a inclusão da dinâmica completa do veículo: equações de movimento de corpo flutuante com seis graus de liberdade, modelo de propulsores (empuxo, saturação, dinâmica elétrica), e acoplamento bidirecional entre o movimento do veículo e as perturbações de reação do braço manipulador [1, 19]. Esta extensão habilitaria o estudo de estratégias de controle coordenadas veículo-braço, que exploram a redundância global do UVMS para otimizar critérios de manipulabilidade ou consumo de energia [9, 21].

7.4.4 Extensões das Estratégias de Controle

Controle Adaptativo

As quatro estratégias implementadas assumem parâmetros dinâmicos conhecidos (ou aproximados) e fixos. Uma extensão importante é a incorporação de leis de adaptação em linha que estimam e atualizam iterativamente os parâmetros dinâmicos $\{m_i, I_i, cx_i, \dots\}$ durante a operação, com base na observação do erro de rastreamento. O controle adaptativo por torque computado de Slotine e Li [29] e Tomei [64] , que exploram a parametrização linear em relação aos parâmetros dinâmicos da equação de movimento , são candidatos naturais para integração com o modelo simbólico do *Hephaestus*, cujas expressões de M , C e G são lineares nos parâmetros por construção lagrangiana [4, 23]. Para o cenário subaquático, a adaptação dos coeficientes de massa adicionada e arrasto durante a operação é especialmente relevante, dada a dificuldade de sua identificação prévia [19, 20].

Controle Baseado em Aprendizado de Máquina

A combinação de controle baseado em modelo com aprendizado de máquina , conhecida como *Model-Based Reinforcement Learning* (MBRL) [52] ou controle híbrido físico-neural , é uma direção de pesquisa de intensa atividade. Redes neurais profundas podem ser treinadas para compensar os resíduos de modelagem que os

controladores clássicos deixam no erro de rastreamento, atuando como um termo de distúrbio aprendido que complementa a lei de controle baseada em modelo [51]. A plataforma *Hephaestus*, com sua capacidade de simular perturbações configuráveis e coletar dados de erro e torque, é um ambiente natural para o treinamento e avaliação dessas abordagens híbridas [50].

Controle por Impedância e Interação com o Ambiente

As estratégias de controle implementadas são projetadas para rastreamento de trajetória livre de contato. Para tarefas que envolvam interação física com o ambiente, como manipulação de objetos, inspeção em contato, ou ancoragem em estruturas submarinas, o controle por impedância [4, 23] é o paradigma adequado: em vez de rastrear uma posição, o controlador regula a relação força-deslocamento (impedância mecânica) do efetuador, permitindo conformidade controlada. A extensão do *Hephaestus* para controle por impedância requereria a inclusão de modelos de contato [52] e, idealmente, a integração com sensores de força/torque no efetuador, o que constitui uma ponte direta com a validação experimental [19].

Controle Preditivo Baseado em Modelo (MPC)

O Controle Preditivo Baseado em Modelo (*Model Predictive Control*, MPC) [10], que otimiza sequências de controle futuras sobre um horizonte deslizante, sujeito a restrições de estado e entrada, é uma abordagem atraente para manipuladores com limites de torque e de espaço de trabalho explícitos. A disponibilidade das equações dinâmicas simbólicas compiladas no *Hephaestus* viabiliza a linearização em torno da trajetória nominal a cada instante, gerando modelos lineares por partes que podem ser utilizados em MPC linear (*Linear MPC*) [22] sem necessidade de re-derivação numérica.

Controle por Modos Deslizantes de Ordem Superior com Observadores

O algoritmo STA implementado opera com estados medidos (posição e velocidade de junta). Em *hardware* real, a velocidade frequentemente não é medida diretamente mas estimada por diferenciação numérica ou por observador de Luenberger [22]. A integração do STA com observadores de ordem reduzida ou com o ESO do LADRC (aproveitando a estimação de velocidade já disponível em z_2) é uma direção promissora para melhorar a robustez a ruído de medição sem sacrificar as propriedades de convergência em tempo finito [14, 56, 58]. A análise de estabilidade do sistema observador-controlador combinado é um problema aberto para manipuladores de múltiplos GDL [41].

7.4.5 Integrações de Software e Hardware

Integração com ROS/ROS2

O *Robot Operating System* (ROS) é o *middleware de facto* para sistemas robóticos experimentais [63]. A integração do *Hephaestus* com ROS2 permitiria publicar as referências de junta geradas pelo CLIK como tópicos ROS e receber *feedback* de posição/velocidade de *hardware* real ou de simuladores físicos como Gazebo, fechando o laço de controle com o robô físico. O modelo simbólico compilado poderia ser exportado como um nó ROS de dinâmica, servindo como referência de modelo para outros nós de controle [43].

Validação em Hardware de Baixo Custo

Uma direção imediata de trabalho futuro é a validação experimental dos controladores em plataformas de baixo custo acessíveis para laboratórios de ensino, como manipuladores de 3–4 GDL baseados em servomotores Dynamixel ou similares. A comparação entre as simulações do *Hephaestus* e os dados experimentais permitiria quantificar a fidelidade do modelo e identificar as principais fontes de discrepância, informação essencial para a calibração dos parâmetros dinâmicos e para o ajuste dos ganhos de controle em *hardware* real [4, 23].

Identificação de Parâmetros Dinâmicos

Uma lacuna prática do *Hephaestus* é a necessidade de o usuário fornecer os parâmetros dinâmicos (m_i , I_i , cx_i , etc.) manualmente ou por projeto CAD. Métodos de identificação de parâmetros dinâmicos, que estimam os parâmetros por mínimos quadrados a partir de trajetórias de excitação otimizadas [4, 23], poderiam ser integrados como um módulo adicional do simulador, fechando o ciclo entre a plataforma computacional e o robô físico. Para o caso subaquático, a identificação dos coeficientes de massa adicionada e arrasto por ensaios de movimento forçado em tanque [1, 32] é um passo necessário para validação do modelo hidrodinâmico.

Suporte a Múltiplos Robôs e Coordenação

A extensão para cenários com múltiplos robôs, manipuladores cooperativos em operação conjunta [4] ou múltiplos UVMSs em missão coordenada [19], requereria a modelagem do objeto compartilhado (ou da tarefa distribuída) e a síntese de leis de controle descentralizadas com garantias de estabilidade do sistema global [9]. O *framework* de modelagem simbólica do *Hephaestus* poderia ser estendido para criar um modelo acoplado de múltiplos robôs, explorando a mesma infraestrutura de Jacobianos e derivação lagrangiana.

Compilação para Sistemas Embarcados

O modelo dinâmico compilado via `lambdify` é código Python/NumPy que, embora eficiente em *hardware* de propósito geral, pode não ser adequado para controladores embarcados em tempo real com restrições de memória e latência. Uma extensão futura seria a geração de código C/C++ a partir das expressões simbólicas, o SymPy já suporta `ccode` e `cxxcode` para geração de código C, que poderia ser compilado para plataformas embarcadas como Arduino, STM32 ou FPGA, habilitando a implementação dos controladores em *hardware* dedicado [2, 43].

7.5 Considerações Finais

O *Hephaestus* representa uma contribuição concreta à democratização das ferramentas de simulação de robótica de alta fidelidade. Ao integrar, em um único ambiente Python de código aberto, a derivação simbólica exata das equações de Euler-Lagrange [4, 5], a compilação eficiente via CSE e paralelismo [2, 45], o controle no espaço de tarefa 6D com CLIK robusto [7, 8], e quatro estratégias de controle não-linear comparáveis [11, 13, 15], a plataforma cobre um espectro amplo de necessidades de pesquisa e ensino em dinâmica e controle de robótica.

A acessibilidade da plataforma, operável via interface gráfica sem necessidade de programação direta, e executável como aplicativo autocontido, reduz a barreira de entrada para estudantes e pesquisadores que desejam explorar o controle de manipuladores além das ferramentas clássicas [50]. Ao mesmo tempo, a transparência matemática, cada equação inspecionável simbolicamente, cada passo algorítmico documentado, torna a plataforma um veículo didático eficaz para o ensino da dinâmica lagrangiana, do controle baseado em modelo e dos efeitos hidrodinâmicos em UVMSs [1, 19].

As extensões propostas na seção de trabalhos futuros, incluindo o *roadmap* de *epics* (interface serial, gêmeo digital, UVMS completo, geração de *datasets* para IA), os trabalhos específicos ao UVMS e as integrações com *hardware* real e ROS2, posicionam o *Hephaestus* como uma plataforma evolutiva, capaz de acompanhar o estado da arte em robótica subaquática e controle não-linear por um longo período. Espera-se que o projeto, ao ser disponibilizado como código aberto, atraia contribuições da comunidade e evolua em direção a um ecossistema de simulação de robótica colaborativo e pedagogicamente orientado [43, 50].

Referências Bibliográficas

- [1] FOSSEN, T. I. *Handbook of Marine Craft Hydrodynamics and Motion Control*. Chichester, Wiley, 2011.
- [2] MEURER, A., SMITH, C. P., PAPROCKI, M., et al. “SymPy: symbolic computing in Python”, *PeerJ Computer Science*, v. 3, pp. e103, 2017.
- [3] ALFRED, V., MONICA, S., RAVI, S., et al. *Compilers Principles, Techniques*. Pearson, 2007.
- [4] SICILIANO, B., SCIAVICCO, L., VILLANI, L., et al. *Robotics: Modelling, Planning and Control*. London, Springer, 2009.
- [5] CRAIG, J. J. *Introduction to Robotics: Mechanics and Control*. 2nd ed. Reading, MA, Addison-Wesley, 1989.
- [6] SHOEMAKE, K. “Animating rotation with quaternion curves”. In: *Proceedings of the 12th annual conference on Computer graphics and interactive techniques*, pp. 245–254, 1985.
- [7] CHIAVERINI, S., SICILIANO, B., EGELAND, O. “Review of the damped least-squares inverse kinematics with experiments on an industrial robot manipulator”, *IEEE Transactions on control systems technology*, v. 2, n. 2, pp. 123–134, 1994.
- [8] NAKAMURA, Y., HANAFUSA, H. “Inverse Kinematic Solutions with Singularity Robustness for Robot Manipulator Control”, *Journal of Dynamic Systems, Measurement, and Control*, v. 108, n. 3, pp. 163–171, 1986.
- [9] CHIAVERINI, S. “Singularity-Robust Task-Priority Redundancy Resolution for Real-Time Kinematic Control of Robot Manipulators”, *IEEE Transactions on Robotics and Automation*, v. 13, n. 3, pp. 398–410, 1997.
- [10] ASTROM, K. J., HAGGLUND, T., CONTROLLERS, P. “Theory, design and tuning”, *Research Triangle Park: 2nd Ed. Instrumentation, Systems and Automatic Society*, 1995.

- [11] SLOTINE, J.-J. E., LI, W., OTHERS. *Applied nonlinear control*, v. 199. Prentice hall Englewood Cliffs, NJ, 1991.
- [12] BURTON, J., ZINOBER, A. S. “Continuous approximation of variable structure control”, *International journal of systems science*, v. 17, n. 6, pp. 875–885, 1986.
- [13] LEVANT, A. “Sliding order and sliding accuracy in sliding mode control”, *International journal of control*, v. 58, n. 6, pp. 1247–1263, 1993.
- [14] MORENO, J. A., OSORIO, M. “Strict Lyapunov functions for the super-twisting algorithm”, *IEEE transactions on automatic control*, v. 57, n. 4, pp. 1035–1040, 2012.
- [15] GAO, Z., OTHERS. “Scaling and bandwidth-parameterization based controller tuning”. In: *Acc*, v. 4, pp. 989–4, 2003.
- [16] ZHENG, Q., GAO, Z. “On practical applications of active disturbance rejection control”. In: *Proceedings of the 29th Chinese control conference*, pp. 6095–6100. IEEE, 2010.
- [17] HAN, J. “From PID to active disturbance rejection control”, *IEEE transactions on Industrial Electronics*, v. 56, n. 3, pp. 900–906, 2009.
- [18] HUANG, Y., XUE, W. “Active disturbance rejection control: Methodology and theoretical analysis”, *ISA transactions*, v. 53, n. 4, pp. 963–976, 2014.
- [19] ANTONELLI, G. *Underwater Robots*. Springer Tracts in Advanced Robotics. 3rd ed. Cham, Springer, 2014.
- [20] ALDHAHERI, S., DE MASI, G., PAIRET, È., et al. “Underwater Robot Manipulation: Advances, Challenges and Prospective Ventures”. In: *OCEANS 2022 - Chennai*, pp. 1–7. IEEE, 2022.
- [21] YUH, J., WEST, M. “Underwater Robotics”, *Advanced Robotics*, v. 15, n. 5, pp. 609–639, 2001.
- [22] OGATA, K. *Modern Control Engineering*. 5th ed. Upper Saddle River, NJ, Prentice Hall, 2010.
- [23] SPONG, M. W., HUTCHINSON, S., VIDYASAGAR, M. *Robot Modeling and Control*. 2nd ed. Hoboken, NJ, Wiley, 2020.

- [24] HOLLERBACH, J. M. “A Recursive Lagrangian Formulation of Manipulator Dynamics and a Comparative Study of Dynamics Formulation Complexity”, *IEEE Transactions on Systems, Man, and Cybernetics*, v. 10, n. 11, pp. 730–736, 1980.
- [25] DENAVIT, J., HARTENBERG, R. S. “A Kinematic Notation for Lower-Pair Mechanisms Based on Matrices”, *Journal of Applied Mechanics*, v. 22, n. 2, pp. 215–221, 1955.
- [26] GOLUB, G. H., VAN LOAN, C. F. *Matrix Computations*. 4th ed. Baltimore, MD, Johns Hopkins University Press, 2013.
- [27] WAMPLER, C. W. “Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods”, *IEEE Transactions on Systems, Man, and Cybernetics*, v. 16, n. 1, pp. 93–101, 2007.
- [28] LUH, J. Y. S., WALKER, M. W., PAUL, R. P. “On-Line Computational Scheme for Mechanical Manipulators”, *Journal of Dynamic Systems, Measurement, and Control*, v. 102, n. 2, pp. 69–76, 1980.
- [29] SLOTINE, J.-J. E., LI, W. “On the adaptive control of robot manipulators”, *The international journal of robotics research*, v. 6, n. 3, pp. 49–59, 1987.
- [30] YUH, J. “Modeling and Control of Underwater Robotic Vehicles”, *IEEE Transactions on Systems, Man, and Cybernetics*, v. 20, n. 6, pp. 1475–1483, 1990.
- [31] SIVČEV, S., COLEMAN, J., OMERDIC, E., et al. “Underwater Manipulators: A Review”, *Ocean Engineering*, v. 163, pp. 431–450, 2018.
- [32] NEWMAN, J. N. *Marine Hydrodynamics*. Cambridge, MA, MIT Press, 1977.
- [33] SAMSON, C., ESPIAU, B., BORGNE, M. L. *Robot control: the task function approach*. Oxford University Press, Inc., 1991.
- [34] LIEGEOIS, A., OTHERS. “Automatic supervisory control of the configuration and behavior of multibody mechanisms”, *IEEE transactions on systems, man, and cybernetics*, v. 7, n. 12, pp. 868–871, 1977.
- [35] MARQUARDT, D. W. “An algorithm for least-squares estimation of nonlinear parameters”, *Journal of the society for Industrial and Applied Mathematics*, v. 11, n. 2, pp. 431–441, 1963.

- [36] MORÉ, J. J. “The Levenberg-Marquardt algorithm: implementation and theory”. In: *Numerical analysis: proceedings of the biennial Conference held at Dundee, June 28–July 1, 1977*, pp. 105–116. Springer, 2006.
- [37] ARMIJO, L. “Minimization of functions having Lipschitz continuous first partial derivatives”, *Pacific Journal of mathematics*, v. 16, n. 1, pp. 1–3, 1966.
- [38] DAM, E. B., KOCH, M., LILLHOLM, M. *Quaternions, interpolation and animation*, v. 2. Datalogisk Institut, Københavns Universitet Copenhagen, Denmark, 1998.
- [39] SPONG, M., VIDYASAGAR, M. *Robot Dynamics And Control*. Wiley India Pvt. Limited, 2008. ISBN: 9788126517800. Disponível em: <<https://books.google.com.br/books?id=PtxYAv7ZUYMC>>.
- [40] EDWARDS, C., SPURGEON, S. K. *Sliding mode control: theory and applications*. CRC press, 1998.
- [41] UTKIN, V., GULDNER, J., SHI, J. *Sliding mode control in electro-mechanical systems*. CRC press, 2017.
- [42] HAN, J. “A class of extended state observers for uncertain systems”, *Control and decision*, v. 10, n. 1, pp. 85–88, 1995.
- [43] LEU, M.-C., HEMATI, N. “Automated symbolic derivation of dynamic equations of motion for robotic manipulators”, 1986.
- [44] MOSES, J. “Algebraic simplification a guide for the perplexed”. In: *Proceedings of the second ACM symposium on Symbolic and algebraic manipulation*, pp. 282–304, 1971.
- [45] BEAZLEY, D. “Understanding the python gil”. In: *PyCON Python Conference. Atlanta, Georgia*, pp. 1–62, 2010.
- [46] LEIMKUHLER, B., REICH, S. *Simulating hamiltonian dynamics*. N. 14. Cambridge university press, 2004.
- [47] HAIRER, E., LUBICH, C., WANNER, G. *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*. 2nd ed. Berlin, Springer, 2006.
- [48] PRATS, M., FERNÁNDEZ, J. J., SANZ, P. J. “An Open Source Tool for Simulation and Supervision of Underwater Intervention Missions”. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2577–2582. IEEE, 2012.

- [49] ZHANG, M. M., CHOI, W.-S., HERMAN, J., et al. “DAVE Aquatic Virtual Environment: Toward a General Underwater Robotics Simulator”. In: *2022 IEEE/OES Autonomous Underwater Vehicles Symposium (AUV)*, pp. 1–8, 2022. doi: 10.1109/AUV53081.2022.9965808.
- [50] CORKE, P., HAVILAND, J. “Not your grandmother’s toolbox – the Robotics Toolbox reinvented for Python”. In: *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 11357–11363, 2021. doi: 10.1109/ICRA48506.2021.9561366.
- [51] ISIDORI, A. *Nonlinear control systems: an introduction*. Springer, 1985.
- [52] KHALIL, H. K., GRIZZLE, J. W. *Nonlinear systems*, v. 3. Prentice hall Upper Saddle River, NJ, 2002.
- [53] UTKIN, V. I. “Variable Structure Systems with Sliding Modes”, *IEEE Transactions on Automatic Control*, v. 22, n. 2, pp. 212–222, 1977.
- [54] SLOTTINE, J.-J. E. “Sliding controller design for non-linear systems”, *International Journal of control*, v. 40, n. 2, pp. 421–434, 1984.
- [55] SICILIANO, B., VILLANI, L. *Robot Force Control*. Kluwer international series in engineering and computer science: Robotics: vision, manipulation, and sensors. Springer US, 1999. ISBN: 9780792377337. Disponível em: <<https://books.google.md/books?id=1uQPYTmQsZAC>>.
- [56] LEVANT, A. “Higher-order sliding modes, differentiation and output-feedback control”, *International journal of Control*, v. 76, n. 9-10, pp. 924–941, 2003.
- [57] BARTOLINI, G., FERRARA, A., USAI, E., et al. “On multi-input chattering-free second-order sliding mode control”, *IEEE transactions on automatic control*, v. 45, n. 9, pp. 1711–1717, 2002.
- [58] SHTESEL, Y., EDWARDS, C., FRIDMAN, L., et al. *Sliding mode control and observation*, v. 10. Springer, 2014.
- [59] BHAT, S. P., BERNSTEIN, D. S. “Finite-time stability of continuous autonomous systems”, *SIAM Journal on Control and optimization*, v. 38, n. 3, pp. 751–766, 2000.
- [60] LEVANT, A. “Chattering analysis”, *IEEE transactions on automatic control*, v. 55, n. 6, pp. 1380–1389, 2010.

- [61] MADONSKI, R., SHAO, S., ZHANG, H., et al. “General error-based active disturbance rejection control for swift industrial implementations”, *Control Engineering Practice*, v. 84, pp. 218–229, 2019.
- [62] GAMMA, E., HELM, R., JOHNSON, R., et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, Addison-Wesley, 1994.
- [63] CORKE, P. *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*. 2nd ed. Cham, Springer, 2017.
- [64] TOMEI, P. “A simple PD controller for robots with elastic joints”, *IEEE Transactions on automatic control*, v. 36, n. 10, pp. 1208–1213, 2002.

Apêndice A

Código-fonte da *Engine* Matemática (engine.py)

Este apêndice apresenta o código-fonte completo dos componentes fundamentais da *engine* matemática do *Hephaestus*, implementados no módulo `engine.py`. As listagens cobrem as funções de nível de módulo utilizadas para paralelismo, a classe `RobotMathEngine` (ambiente seco) e a subclasse `RobotMathHydro` (ambiente subaquático). O código é apresentado na íntegra para viabilizar reprodutibilidade e auditoria matemática das equações derivadas.

A.1 Funções de Nível de Módulo para Paralelismo

As duas funções a seguir são definidas no escopo do módulo para permitir sua serialização via `pickle`, requisito do `ProcessPoolExecutor` no Python. A função `_diff_M_col` diferencia a matriz de inércia em relação a uma variável de junta q_k ; a função `_calc_coriolis_row` monta a i -ésima linha do vetor de Coriolis pela fórmula dos símbolos de Christoffel.

```
1 from concurrent.futures import ProcessPoolExecutor
2 from itertools import repeat
3 import os, time
4 import sympy as sp
5 from sympy.physics.mechanics import dynamicsymbols
6
7 def _diff_M_col(args):
8     """Diferencia a matriz M em relação a uma variável simbólica qk
9     .
10    Executado em processo separado para paralelizar o cálculo de dM/dq.
11    """
12    M_list, n, qk = args
13    M = sp.Matrix(n, n, M_list)
```

```

13     return list(M.diff(qk))
14
15
16 def _calc_coriolis_row(i, q, dq, dM_dq):
17     n = len(q)
18     termo_linha = sp.Integer(0)
19     for j in range(n):
20         for k in range(j, n):
21             dM_ij_dk = dM_dq[k][i, j]
22             dM_ik_dj = dM_dq[j][i, k]
23             dM_jk_di = dM_dq[i][j, k]
24             c_ijk = sp.Rational(1, 2) * (dM_ij_dk + dM_ik_dj -
dM_jk_di)
25             if c_ijk != 0:
26                 termo = c_ijk * dq[j] * dq[k]
27                 if k != j:
28                     termo *= 2
29                 termo_linha += termo
30             # sp.collect() omitido intencionalmente: custo superlinear sem
benefício,
31             # pois o cse=True no lambdify posterior realiza compactação
sistemática.
32     return termo_linha

```

Listing A.1: Funções paralelas para derivação de $\partial M/\partial q_k$ e cálculo de C_i .

A.2 Classe RobotMathEngine: Ambiente Seco

```

1 class RobotMathEngine:
2     def __init__(self, joint_config, link_vectors_mask,
3                 logger_callback=None, num_workers=None):
4         self.joint_config = joint_config
5         self.link_vectors_mask = [sp.Matrix(v) for v in
link_vectors_mask]
6         self.log = logger_callback if logger_callback
else print
7         self.num_workers = num_workers
8
9         self.t = sp.symbols('t')
10        self.g = sp.symbols('g')
11
12        self.q, self.dq, self.params_list = [], [], []
13        self.frames, self.rotation_matrices = [], []
14        self.com_positions_global = []
15        self.angular_velocities = []
16        self.masses = []

```

```

17
18         self.params_list.append(self.g)      # g sempre presente como
parâmetro
19         self.M, self.G_vec, self.C_total, self.Jacobian = None,
None, None, None
20
21     def _rot_matrix_local(self, axis, angle):
22         c, s = sp.cos(angle), sp.sin(angle)
23         if axis == 'x': return sp.Matrix([[1,0,0],[0,c,-s],[0,s,c
]])
24         if axis == 'y': return sp.Matrix([[c,0,s],[0,1,0],[-s,0,c
]])
25         if axis == 'z': return sp.Matrix([[c,-s,0],[s,c
,0],[0,0,1]])
26         return sp.eye(3)
27
28     def _get_axis_vector(self, axis):
29         if axis == 'x': return sp.Matrix([1, 0, 0])
30         if axis == 'y': return sp.Matrix([0, 1, 0])
31         if axis == 'z': return sp.Matrix([0, 0, 1])
32         return sp.Matrix([0, 0, 0])

```

Listing A.2: Construtor e configuração de símbolos de RobotMathEngine.

```

1     def step_1_kinematics(self):
2         self.log("1. (AR) Calculando Cinemática (Com CM Arbitrário)
...")
3         T_acc = sp.eye(4)
4         R_acc = sp.eye(3)
5         omega_acc = sp.Matrix([0, 0, 0])
6
7         for i, (j_type, link_vec) in enumerate(
8             zip(self.joint_config, self.link_vectors_mask)):
9             q = dynamicsymbols(f'q{i+1}')
10            dq = dynamicsymbols(f'q{i+1}', 1)
11            self.q.append(q); self.dq.append(dq)
12
13            m = sp.symbols(f'm{i+1}')
14            L = sp.symbols(f'L{i+1}')
15            cx, cy, cz = sp.symbols(f'cx{i+1} cy{i+1} cz{i+1}')
16
17            self.masses.append(m)
18            self.params_list.extend([m, L, cx, cy, cz])
19
20            type_char = j_type[0]
21            axis_char = j_type[1].lower()
22
23            if type_char == 'R':                # Junta de revolução

```

```

24         axis_vec_global = R_acc * self._get_axis_vector(
axis_char)
25         omega_new = omega_acc + axis_vec_global * dq
26         R_j = self._rot_matrix_local(axis_char, q)
27         P_j = sp.Matrix([0, 0, 0])
28         elif type_char == 'D':           # Junta prismatica
29             omega_new = omega_acc
30             R_j = sp.eye(3)
31             P_j = sp.Matrix([0, 0, 0])
32             if axis_char == 'x': P_j[0] = q
33             if axis_char == 'y': P_j[1] = q
34             if axis_char == 'z': P_j[2] = q
35
36         R_acc = R_acc * R_j
37         self.rotation_matrices.append(R_acc)
38         self.angular_velocities.append(omega_new)
39         omega_acc = omega_new
40
41         T_joint = sp.eye(4)
42         T_joint[0:3, 0:3] = R_j
43         T_joint[0:3, 3]   = P_j
44         T_at_joint_start = T_acc * T_joint
45
46         P_link_vec      = link_vec * L
47         T_link          = sp.eye(4)
48         T_link[0:3, 3]  = P_link_vec
49         T_acc           = T_at_joint_start * T_link
50         self.frames.append(T_acc)
51
52         # CM Global: transforma coordenadas locais pelo frame
ate a junta
53         v_cm_local     = sp.Matrix([cx, cy, cz, 1])
54         p_cm_global    = T_at_joint_start * v_cm_local
55         self.com_positions_global.append(p_cm_global[0:3, 0])

```

Listing A.3: Cinemática simbólica direta: transformações homogêneas e posições de CM.

```

1     def step_2_jacobian_M_G(self):
2         self.log("2. (AR) Dinamica M e G (Steiner + Tensores)...")
3         _t0 = time.perf_counter()
4         n = len(self.q)
5         self.M = sp.zeros(n, n)
6         V_tot = 0
7
8         for i in range(n):
9             m = self.masses[i]
10            pos_cm = self.com_positions_global[i]
11            R_global = self.rotation_matrices[i]

```

```

12
13         J_v = pos_cm.jacobian(self.q)           # Jacobiano
linear do CM
14         J_w = self.angular_velocities[i].jacobian(self.dq) #
Jacobiano angular
15
16         Ixx, Iyy, Izz = sp.symbols(f'Ixx{i+1} Iyy{i+1} Izz{i+1}
')
17         self.params_list.extend([Ixx, Iyy, Izz])
18
19         # Transformacao de Steiner: I_global = R * I_local * R^
T
20         I_local = sp.Matrix([[Ixx, 0, 0], [0, Iyy, 0], [0, 0,
Izz]])
21         I_global = R_global * I_local * R_global.T
22
23         # M += m * J_v^T * J_v + J_w^T * I_global * J_w (metodo do
Jacobiano)
24         self.M += m * J_v.T * J_v + J_w.T * I_global * J_w
25
26         V_tot += m * self.g * pos_cm[2] # Energia potencial
gravitacional
27         self.log(f" M/G: elo {i+1}/{n} concluido")
28
29         self.G_vec = sp.Matrix([V_tot]).jacobian(self.q).T
30         J_lin = self.frames[-1][0:3, 3].jacobian(self.q)
31         J_ang = self.angular_velocities[-1].jacobian(self.dq
)
32         self.Jacobian = J_lin.col_join(J_ang) # Jacobiano 6xN
completo
33         self.log(f" M/G/J: concluido em {time.perf_counter()-_t0
:.1f}s")

```

Listing A.4: Derivação simbólica de $M(q)$, $G(q)$ e $J(q)$ pelo método do Jacobiano da inércia.

```

1     def step_3_coriolis_combined(self):
2         self.log("3. Calculando Coriolis...")
3         n = len(self.q)
4         self.C_total = sp.zeros(n, 1)
5         cpu_total = os.cpu_count() or 1
6         workers = self.num_workers if self.num_workers is not
None else cpu_total
7         use_parallel = workers is not None and workers > 1
8
9         # --- Passo 3a: dM/dq (paralelizavel por coluna) ---
10        self.log(" dM/dq: diferenciando M" +
11                (" em paralelo" if use_parallel else "") + "...")

```

```

12     _t0 = time.perf_counter()
13     M_list = list(self.M)
14     tasks = [(M_list, n, qk) for qk in self.q]
15
16     if use_parallel:
17         with ProcessPoolExecutor(max_workers=workers) as ex:
18             dM_dq_flat = list(ex.map(_diff_M_col, tasks))
19     else:
20         dM_dq_flat = [_diff_M_col(t) for t in tasks]
21
22     dM_dq = [sp.Matrix(n, n, flat) for flat in dM_dq_flat]
23     self.log(f" dM/dq: concluido em {time.perf_counter()-_t0
24 :.1f}s")
25
26     # --- Passo 3b: Simbolos de Christoffel C_i(q,dq) ---
27     if use_parallel:
28         with ProcessPoolExecutor(max_workers=workers) as ex:
29             for i, row in enumerate(
30                 ex.map(_calc_coriolis_row,
31                     range(n), repeat(self.q),
32                     repeat(self.dq), repeat(dM_dq)),
33                 start=1):
34                 self.C_total[i - 1] = row
35                 self.log(f" Coriolis: linha {i}/{n} concluida"
36 )
37     else:
38         for i in range(n):
39             self.C_total[i] = _calc_coriolis_row(i, self.q,
40 self.dq, dM_dq)
41             self.log(f" Coriolis: linha {i+1}/{n} concluida")

```

Listing A.5: Cálculo do vetor de Coriolis pelos símbolos de Christoffel com paralelismo opcional.

```

1     def step_4_prepare_export(self):
2         self.log("4. Otimizando equacoes...")
3         mapa_subs = {}
4         for i in range(len(self.q)):
5             mapa_subs[self.q[i]] = sp.Symbol(f'q{i+1}')
6             mapa_subs[self.dq[i]] = sp.Symbol(f'dq{i+1}')
7
8         dq_vec = sp.Matrix(self.dq)
9         V_cartesian = self.Jacobian * dq_vec if self.Jacobian is
10 not None else None
11
12     return {
13         "M": self.M,
14         "G": self.G_vec,

```

```

14         "C":      self.C_total,
15         "J":      self.Jacobian,
16         "FK_Pos": self.frames[-1],
17         "FK_Vel": V_cartesian,
18         "Subs":    mapa_subs,
19         "Mode":    "Air"
20     }

```

Listing A.6: Substituição de símbolos dinâmicos e exportação do dicionário de modelo.

A.3 Subclasse RobotMathHydro: Extensão Hidrodinâmica

```

1  class RobotMathHydro(RobotMathEngine):
2      def __init__(self, joint_config, link_vectors_mask,
3                  logger_callback=None, num_workers=None):
4          super().__init__(joint_config, link_vectors_mask,
5                          logger_callback, num_workers)
6          self.rho = sp.symbols('rho') # densidade do fluido
7          self.volumes = []
8
9      def step_1_kinematics_hydro(self):
10         super().step_1_kinematics()
11         self.log("    -> Adicionando parametros hidrodinamicos...")
12         for i in range(len(self.q)):
13             vol = sp.symbols(f'vol_{i+1}')
14             self.volumes.append(vol)
15             self.params_list.append(vol) # vol_i adicionado a
assinatura
16
17         def step_2_jacobian_M_G(self):
18             self.log("2. (AGUA) Dinamica: M (Inercia + Added Mass) e G
(Peso - Empuxo)...")
19             _t0 = time.perf_counter()
20             n = len(self.q)
21             self.M = sp.zeros(n, n)
22             V_tot = 0
23
24             for i in range(n):
25                 m = self.masses[i]
26                 vol = self.volumes[i]
27                 pos_cm = self.com_positions_global[i]
28                 R_global = self.rotation_matrices[i]
29
30                 J_v = pos_cm.jacobian(self.q)

```

```

31         J_w = self.angular_velocities[i].jacobian(self.dq)
32
33         # Tensor de inercia rigido (referencial global via
34         Steiner)
35         Ixx, Iyy, Izz = sp.symbols(f'Ixx{i+1} Iyy{i+1} Izz{i+1}
36         ')
37         self.params_list.extend([Ixx, Iyy, Izz])
38         I_local_RB = sp.Matrix([[Ixx,0,0],[0,Iyy,0],[0,0,Izz
39         ]])
40         I_global_RB = R_global * I_local_RB * R_global.T
41
42         # Tensor de massa adicionada (diagonal local, 6
43         componentes por elo)
44         ma_u, ma_v, ma_w = sp.symbols(f'ma_u{i+1} ma_v{i+1}
45         ma_w{i+1}')
46         ma_p, ma_q, ma_r = sp.symbols(f'ma_p{i+1} ma_q{i+1}
47         ma_r{i+1}')
48         self.params_list.extend([ma_u, ma_v, ma_w, ma_p, ma_q,
49         ma_r])
50
51         MA_lin_local = sp.Matrix([[ma_u,0,0],[0,ma_v,0],[0,0,
52         ma_w]])
53         MA_rot_local = sp.Matrix([[ma_p,0,0],[0,ma_q,0],[0,0,
54         ma_r]])
55
56         # Rotacao para referencial global:  $M_A^{(g)} = R * M_A^{(l)}$ 
57         ) * R^T
58         MA_lin_global = R_global * MA_lin_local * R_global.T
59         MA_rot_global = R_global * MA_rot_local * R_global.T
60
61         #  $M_{hydro} += J_v^T (m * I + M_{A,lin})^{(g)} J_v + J_w^T (I_{RB} +$ 
62          $M_{A,rot})^{(g)} J_w$ 
63         self.M += J_v.T * (m * sp.eye(3) + MA_lin_global) * J_v
64         self.M += J_w.T * (I_global_RB + MA_rot_global) * J_w
65
66         # Potencial aparente:  $V_i = (m - \rho * vol) * g * z_{cm}$  (
67         Peso - Empuxo)
68         peso_aparente = (m - self.rho * vol) * self.g
69         V_tot += peso_aparente * pos_cm[2]
70         self.log(f" M/G: elo {i+1}/{n} concluido")
71
72         self.G_vec = sp.Matrix([V_tot]).jacobian(self.q).T
73         J_lin = self.frames[-1][0:3, 3].jacobian(self.q)
74         J_ang = self.angular_velocities[-1].jacobian(self.dq
75         )
76         self.Jacobian = J_lin.col_join(J_ang)

```



```

64         self.log(f"    M/G/J: concluido em {time.perf_counter()-_t0
        :.1f}s")
65
66     def step_4_prepare_export(self):
67         data = super().step_4_prepare_export()
68         data["Mode"] = "Hydro"
69         return data

```

Listing A.7: Extensão hidrodinâmica: massa adicionada, empuxo e potencial aparente.

Apêndice B

Código-fonte dos Controladores e do Simulador (simulator.py)

Este apêndice apresenta as classes de controle e a estrutura principal da classe `RobotSimulator` implementadas no módulo `simulator.py`. As listagens incluem os controladores CT-SMC, Super-Twisting (STA) e LADRC descentralizado, bem como o construtor do simulador, que realiza a compilação das expressões simbólicas via `lambdify` com CSE, e a infraestrutura de avaliação paralela numérica.

B.1 Funções Auxiliares de Orientação

```
1 import numpy as np
2
3 def _rotation_error_vec(R_ref, R_curr):
4     """Erro de orientacao 3D pela formulacao de anel (skew-
5     symmetric).
6
7     Retorna e_R = vee(0.5*(R_ref @ R_curr^T - R_curr @ R_ref^T)),
8     proporcional ao eixo-angulo para erros pequenos.
9     """
10    skew = 0.5 * (R_ref @ R_curr.T - R_curr @ R_ref.T)
11    return np.array([skew[2, 1], skew[0, 2], skew[1, 0]])
12
13 def _slerp_rotation(R0, Rf, s, sd):
14     """SLERP entre R0 e Rf na fracao s em [0, 1].
15
16     Parâmetros
17     -----
18     R0, Rf : matrizes de rotacao 3x3
19     s       : fracao de interpolacao atual (0 = R0, 1 = Rf)
20     sd      : derivada temporal de s (ds/dt)
```

```

21
22     Retorna
23     -----
24     R_ref      : rotacao interpolada 3x3
25     omega_ref  : velocidade angular no referencial global (rad/s)
26     """
27     dR          = R0.T @ Rf
28     trace_val = np.clip((np.trace(dR) - 1.0) / 2.0, -1.0, 1.0)
29     theta_total = np.arccos(trace_val)
30     if abs(theta_total) < 1e-9:
31         return R0.copy(), np.zeros(3)
32     sin_t       = np.sin(theta_total)
33     skew        = (dR - dR.T) / (2.0 * sin_t)
34     axis_local  = np.array([skew[2, 1], skew[0, 2], skew[1, 0]])
35     theta_s     = s * theta_total
36     K = np.array([[ 0.0,          -axis_local[2],  axis_local[1]],
37                  [ axis_local[2],  0.0,          -axis_local
38                  [0]],
39                  [-axis_local[1],  axis_local[0],  0.0
40                  ]])
41     dR_s = np.eye(3) + np.sin(theta_s) * K + (1.0 - np.cos(theta_s
42     )) * (K @ K)
43     R_ref = R0 @ dR_s
44     omega_ref = (sd * theta_total) * (R0 @ axis_local)
45     return R_ref, omega_ref

```

Listing B.1: Erro de orientação pelo operador de anel (skew-symmetric) e SLERP em $SO(3)$.

B.2 Controlador CT-SMC com Camada Limite

```

1  class SlidingModeControl:
2      """Controlador CT-SMC: Computed Torque + Sliding Mode com
3      camada limite.
4
5      Lei de controle:
6          q_ddot_f = alpha*q_ddot_d + (1-alpha)*q_ddot_f (filtro
7      passabaixa)
8          v        = q_ddot_f - lambda*e_dot - K*tanh(S/phi)
9          tau       = M(q)*v + C(q,dq) + G(q)
10         onde S = e_dot + lambda*e (superficie deslizante).
11
12         Referencia: Slotine & Li, Applied Nonlinear Control, 1991.
13         """

```

```

13     def __init__(self, num_dof, lambda_gain, K, phi,
14                 ddq_filter_alpha=1.0):
15         self.lambda_gain = (np.full(num_dof, lambda_gain, dtype=
16                                float)
17                               if np.isscalar(lambda_gain)
18                               else np.array(lambda_gain, dtype=
19                                              float))
20         self.K = (np.full(num_dof, K, dtype=float)
21                   if np.isscalar(K) else np.array(K,
22                                                     dtype=float))
23         self.phi = (np.full(num_dof, phi, dtype=float)
24                     if np.isscalar(phi) else np.array(
25                         phi, dtype=float))
26         self.phi = np.where(self.phi == 0, 1e-6, self.
27                               phi)
28         self.alpha = float(np.clip(ddq_filter_alpha, 1e-6,
29                                     1.0))
30         self._ddq_d_filt = None
31
32     def compute_tau(self, q, dq, q_d, dq_d, ddq_d, M, C, G, dt=0.0)
33     :
34         if self._ddq_d_filt is None:
35             self._ddq_d_filt = ddq_d.copy()
36         else:
37             self._ddq_d_filt = (self.alpha * ddq_d
38                                 + (1.0 - self.alpha) * self.
39                                 _ddq_d_filt)
40         e = q - q_d
41         e_dot = dq - dq_d
42         S = e_dot + self.lambda_gain * e
43         v = (self._ddq_d_filt - self.lambda_gain * e_dot
44              - self.K * np.tanh(S / self.phi))
45         return M @ v + C + G

```

Listing B.2: Controle por Torque Computado com Modo Deslizante (CT-SMC).

B.3 Controlador Super-Twisting (STA)

```

1 class SuperTwistingSMC:
2     """Controlador Super-Twisting SMC (STA) baseado em modelo.
3
4     Algoritmo de modo deslizante de 2a ordem:
5         q_ddot_f = alpha*q_ddot_d + (1-alpha)*q_ddot_f
6         v_sw     = -k1*|S|^(1/2)*sign(S) + z
7         z_dot    = -k2*sign(S)

```

```

8         tau      = M(q)*[q_ddot_f - lambda*e_dot + v_sw] + C(q,dq)
+ G(q)
9
10     Propriedades: convergencia em tempo finito, sinal de controle
continuo.
11     Condições de ganho (Moreno & Osorio, 2012):  $k_2 > \delta$ ,  $k_1 > 2\sqrt{k_2\delta}$ .
12
13     Referencias:
14     - Levant, Int. J. Control, 58(6), 1993.
15     - Moreno & Osorio, IEEE Trans. Autom. Control, 57(4), 2012.
16     """
17
18     def __init__(self, num_dof, lambda_gain, k1, k2,
ddq_filter_alpha=1.0):
19         self.lambda_gain = (np.full(num_dof, lambda_gain, dtype=
float)
20                               if np.isscalar(lambda_gain)
21                               else np.array(lambda_gain, dtype=float)
)
22         self.k1          = (np.full(num_dof, k1, dtype=float)
23                               if np.isscalar(k1) else np.array(k1,
dtype=float))
24         self.k2          = (np.full(num_dof, k2, dtype=float)
25                               if np.isscalar(k2) else np.array(k2,
dtype=float))
26         self.z           = np.zeros(num_dof)
27         self.alpha       = float(np.clip(ddq_filter_alpha, 1e-6,
1.0))
28         self._ddq_d_filt = None
29
30     def reset(self):
31         self.z[:] = 0.0
32         self._ddq_d_filt = None
33
34     def compute_tau(self, q, dq, q_d, dq_d, ddq_d, M, C, G, dt):
35         if self._ddq_d_filt is None:
36             self._ddq_d_filt = ddq_d.copy()
37         else:
38             self._ddq_d_filt = (self.alpha * ddq_d
+ (1.0 - self.alpha) * self.
_ddq_d_filt)
39
40         e      = q - q_d
41         e_dot  = dq - dq_d
42         S      = e_dot + self.lambda_gain * e
43
44         # Parcela de chaveamento continuo do STA

```

```

45         v_sw = -self.k1 * np.abs(S) ** 0.5 * np.sign(S) + self.z
46
47         v      = self._ddq_d_filt - self.lambda_gain * e_dot +
v_sw
48         self.z -= self.k2 * np.sign(S) * dt      # integracao de z (
Euler explicito)
49         return M @ v + C + G

```

Listing B.3: Controlador Super-Twisting de 2ª ordem (STA) com sinal de controle contínuo.

B.4 LADRC Descentralizado com ESO de Terceira Ordem

```

1  class Decentralized_LADRC:
2      """LADRC descentralizado para multiplas juntas (2a ordem por
junta).
3
4      Para a junta i: ddot_q_i = b0_i * tau_i + f_i (f_i = disturbio
total).
5      0 ESO de 3a ordem (linear) estima [q_i, dq_i, f_i] em z1, z2,
z3.
6      Parametrizacao dos ganhos (Gao, 2003): beta1=3*wo, beta2=3*wo
^2, beta3=wo^3.
7      """
8
9      def __init__(self, num_dof, b0, wo, kp, kd, dt,
10                  z_limit=None, tau_limit=None,
11                  max_wo_dt=0.1, z3_filter_alpha=1.0):
12          self.b0 = (np.full(num_dof, b0, dtype=float)
13                    if np.isscalar(b0) else np.array(b0, dtype=
float))
14          self.wo = (np.full(num_dof, wo, dtype=float)
15                    if np.isscalar(wo) else np.array(wo, dtype=
float))
16          self.kp = (np.full(num_dof, kp, dtype=float)
17                    if np.isscalar(kp) else np.array(kp, dtype=
float))
18          self.kd = (np.full(num_dof, kd, dtype=float)
19                    if np.isscalar(kd) else np.array(kd, dtype=
float))
20          self.dt = dt
21          self.z_limit = z_limit
22          self.tau_limit = tau_limit
23          self.max_wo_dt = max_wo_dt

```

```

24         self.z3_filter_alpha = float(np.clip(z3_filter_alpha, 0.0,
1.0))
25         self.z3_filtered      = None
26
27         self._refresh_gains()
28         self.z1 = np.zeros(num_dof, dtype=float)
29         self.z2 = np.zeros(num_dof, dtype=float)
30         self.z3 = np.zeros(num_dof, dtype=float)
31
32     def _refresh_gains(self):
33         # Limita wo para garantir estabilidade do ESO discreto: wo*
dt <= max_wo_dt
34         if self.max_wo_dt is not None:
35             wo_dt = self.wo * self.dt
36             self.wo = np.where(wo_dt > self.max_wo_dt,
37                               self.max_wo_dt / self.dt, self.wo)
38             self.beta1 = 3.0 * self.wo
39             self.beta2 = 3.0 * (self.wo ** 2)
40             self.beta3 = self.wo ** 3
41
42     def reset_state(self, q, dq, z3=None):
43         self.z1 = np.array(q, dtype=float).copy()
44         self.z2 = np.array(dq, dtype=float).copy()
45         self.z3 = (np.zeros(self.num_dof, dtype=float) if z3 is
None
46                     else (np.full(self.num_dof, z3, dtype=float)
47                             if np.isscalar(z3) else np.array(z3, dtype
=float)))
48         self.z3_filtered = self.z3.copy()
49
50     def update_eso(self, q, tau_prev):
51         """Avanca o ESO por um passo de integracao (Euler explicito)"""
52         error = self.z1 - q
53         z1_dot = self.z2 - self.beta1 * error
54         z2_dot = self.z3 - self.beta2 * error + self.b0 * tau_prev
55         z3_dot = - self.beta3 * error
56
57         self.z1 += z1_dot * self.dt
58         self.z2 += z2_dot * self.dt
59         self.z3 += z3_dot * self.dt
60
61         # Filtro IIR de 1a ordem em z3 (atenua alta frequencia
antes da lei)
62         a = self.z3_filter_alpha
63         self.z3_filtered = a * self.z3 + (1.0 - a) * self.
z3_filtered

```

```

64
65     def compute_control(self, q_d, dq_d, ddq_d, q_meas=None,
dq_meas=None):
66         """Calcula o torque de controle LADRC (cancela disturbio +
PD + ff)."""
67         q_fb = q_meas if q_meas is not None else self.z1
68         dq_fb = dq_meas if dq_meas is not None else self.z2
69
70         z3_use = self.z3_filtered if self.z3_filtered is not None
else self.z3
71         u0 = ddq_d + self.kp * (q_d - q_fb) + self.kd * (dq_d -
dq_fb)
72         b0_safe = np.where(self.b0 == 0.0, 1e-6, self.b0)
73         u = (u0 - z3_use) / b0_safe
74         if self.tau_limit is not None:
75             u = np.clip(u, -self.tau_limit, self.tau_limit)
76         return u

```

Listing B.4: LADRC descentralizado: ESO de 3ª ordem e lei de controle por junta.

B.5 Compilação Simbólico-Numérica e Executor Persistente

```

1  import sympy as sp
2
3  _WORKER_FUNCS = {} # dicionario de modulo: preenchido por
   _init_worker
4
5
6  def _init_worker(sym_vars, expr_M, expr_C, expr_G):
7      """Inicializador do worker: compila lambdify localmente no
processo."""
8      global _WORKER_FUNCS
9      _WORKER_FUNCS = {
10         "M": sp.lambdify(sym_vars, expr_M, modules="numpy"),
11         "C": sp.lambdify(sym_vars, expr_C, modules="numpy"),
12         "G": sp.lambdify(sym_vars, expr_G, modules="numpy"),
13     }
14
15
16  def _eval_worker(task):
17      """Avalia uma das funcoes compiladas com os argumentos
numericos fornecidos."""
18      func_name, args = task
19      return _WORKER_FUNCS[func_name](*args)

```


Listing B.5: Funções de worker para avaliação paralela de M , C , G no executor persistente.

```

1 from concurrent.futures import ProcessPoolExecutor
2 import time, numpy as np, sympy as sp
3
4 class RobotSimulator:
5     def __init__(self, robot_math_instance, mode="Air"):
6         self.bot = robot_math_instance
7         self.mode = mode
8         self.num_dof = len(self.bot.q)
9         self.q_home = np.zeros(self.num_dof)
10        self._executor = None      # executor persistente (
        inicializado apos compilacao)
11        self._executor_workers = 0
12
13        print(f"[{mode}] Compilando equacoes com lambdify+CSE...")
14        _t0 = time.perf_counter()
15
16        # Identificacao de juntas rotacionais (para wrap em [-pi,
        pi])
17        self.is_rotational = np.array(
18            [j.startswith('R') for j in self.bot.joint_config],
        dtype=bool)
19
20        # Assinatura completa: [q1..qN, dq1..dqN, params...]
21        self.sym_vars = self.bot.q + self.bot.dq + self.bot.
        params_list
22        if hasattr(self.bot, 'rho'):
23            self.sym_vars.append(self.bot.rho)
24
25        # Compilacao separada por grandeza (M, C, G, J, FK)
26        # cse=True: CSE antes da geracao do codigo -> reducao de
        60-90% no bytecode
27        self.expr_M = self.bot.M
28        self.expr_C = self.bot.C_total
29        self.expr_G = self.bot.G_vec
30
31        self.func_M = sp.lambdify(self.sym_vars, self.expr_M,
        modules="numpy", cse=True)
32        self.func_C = sp.lambdify(self.sym_vars, self.expr_C,
        modules="numpy", cse=True)
33        self.func_G = sp.lambdify(self.sym_vars, self.expr_G,
        modules="numpy", cse=True)
34        self.func_J = sp.lambdify(self.sym_vars, self.bot.Jacobian,
        modules="numpy", cse=True)
35
36
37
38
39

```

```

40     # Funcoes FK por elo para visualizacao animada
41     self.funcs_fk_all_links = [
42         sp.lambdify(self.sym_vars,
43                     frame[0:3, 3].subs(self.bot.
step_4_prepare_export()["Subs"]),
44                     modules="numpy", cse=True)
45         for frame in self.bot.frames
46     ]
47     print(f"Compilacao concluida em {time.perf_counter()-_t0:.1
f}s")
48
49     def warm_up_workers(self, num_workers):
50         """Inicializa o executor persistente e aquece os workers."""
51
52         if (self._executor is not None
53             and self._executor_workers == num_workers):
54             return # ja inicializado com mesmo numero de workers
55         if self._executor is not None:
56             self._executor.shutdown(wait=False)
57         self._executor = ProcessPoolExecutor(
58             max_workers=num_workers,
59             initializer=_init_worker,
60             initargs=(self.sym_vars, self.expr_M, self.expr_C, self
.expr_G))
61         self._executor_workers = num_workers
62         # Warmup: submete tarefas dummy para pre-compilar os
workers
63         dummy_args = tuple(np.zeros(len(self.sym_vars)))
64         futs = [self._executor.submit(_eval_worker, (k, dummy_args)
)
65                 for k in ("M", "C", "G")]
66         for f in futs:
67             try: f.result()
68             except Exception: pass
69
70     def close(self):
71         """Encerra o executor persistente de forma limpa."""
72         if self._executor is not None:
73             self._executor.shutdown(wait=False)
74             self._executor = None

```

Listing B.6: Construtor de RobotSimulator: compilação lambdify + CSE e inicialização do executor.

Apêndice C

Requisitos de Software, Instalação e Execução do *Hephaestus*

Este apêndice descreve os requisitos de software, o procedimento de instalação e as instruções de execução do ambiente *Hephaestus* a partir do código-fonte.

C.1 Requisitos de Sistema

O *Hephaestus* foi desenvolvido e validado no seguinte ambiente:

- **Sistema Operacional:** Windows 10/11 (64 bits). O código é multiplataforma e compatível com Linux e macOS com as mesmas dependências; no entanto, a interface gráfica com *CustomTkinter* pode apresentar diferenças visuais menores entre plataformas.
- **Python:** versão 3.10 ou superior (recomendado: 3.11). O uso de `ProcessPoolExecutor` com método de *spawn* requer Python ≥ 3.8 ; a tipagem e as dependências utilizadas requerem Python ≥ 3.10 .
- **Hardware recomendado:** processador com pelo menos 4 núcleos lógicos (para aproveitar o paralelismo de derivação); 8 GB de RAM (para modelos com $N \geq 6$ GDL, cujas matrizes simbólicas podem ocupar centenas de MB antes da compilação).

C.2 Dependências Python

Listam-se, na Tabela C.1, as dependências diretas do projeto com as versões mínimas recomendadas.

Todas as dependências listadas são pacotes de código aberto disponíveis no repositório PyPI.

Tabela C.1: Dependências Python do *Hephaestus*.

Pacote	Versão mínima	Função no projeto
sympy	1.12	Álgebra simbólica, derivação de M , C , G e <code>lambdify</code>
numpy	1.24	Álgebra linear numérica, avaliação das funções compiladas
matplotlib	3.7	Visualização dos gráficos de erro, torque e trajetória
customtkinter	5.2	Interface gráfica (<i>GUI</i>) com tema moderno
scipy	1.11	Funções auxiliares (interpolação, transformadas)

C.3 Instalação

Recomenda-se a criação de um ambiente virtual Python isolado para evitar conflitos entre versões de pacotes:

```

1 # Clonar o repositório (ou extrair o arquivo comprimido)
2 git clone https://github.com/seu_usuario/hephaestus.git
3 cd hephaestus
4
5 # Criar e ativar ambiente virtual (Windows)
6 python -m venv .venv
7 .venv\Scripts\activate
8
9 # Instalar dependências
10 pip install sympy numpy matplotlib customtkinter scipy
11
12 # Verificar instalação
13 python -c "import sympy, numpy, matplotlib, customtkinter; print('
    OK')"
```

Listing C.1: Criação de ambiente virtual e instalação das dependências.

Para sistemas Linux/macOS, substituir `.venv\Scripts\activate` por `source .venv/bin/activate`.

C.4 Execução

O ponto de entrada principal da aplicação é o arquivo `main.py`. Para iniciar a interface gráfica:

```

1 # Com o ambiente virtual ativado
2 python main.py
```

Listing C.2: Execução da interface gráfica do Hephaestus.

Importante (Windows): o *Hephaestus* utiliza `ProcessPoolExecutor` com método de *spawn* para o paralelismo de processos. Em sistemas Windows, é obrigatório que o ponto de entrada esteja protegido pelo bloco `if __name__ == '__main__':`, conforme implementado em `main.py`:

```
1 import multiprocessing
2
3 if __name__ == '__main__':
4     multiprocessing.freeze_support()    # necessario para
        executaveis PyInstaller
5     app = App()
6     app.mainloop()
```

Listing C.3: Bloco de proteção obrigatório no `main.py` para Windows.

A ausência deste bloco causaria a abertura recursiva de múltiplas janelas ao criar processos filhos no Windows.

C.5 Geração do Executável Autônomo (.exe)

Para distribuir o *Hephaestus* como um executável autônomo sem necessidade de instalação de Python, utiliza-se o *PyInstaller*:

```
1 pip install pyinstaller
2
3 pyinstaller --onefile --windowed \
4             --add-data "tooltip_utils.py;." \
5             --name "Hephaestus" main.py
```

Listing C.4: Geração do executável PyInstaller.

O arquivo resultante `dist/Hephaestus.exe` é executável sem instalação prévia de Python. Nesse modo, o suporte a *spawn* de processos no Windows é detectado automaticamente pela condição `sys.frozen` do PyInstaller: quando o executável está *frozen*, o número de *workers* do `ProcessPoolExecutor` é automaticamente definido como 1 (modo serial), evitando o conflito de *spawn* em módulos congelados. A derivação simbólica e a simulação operam normalmente, apenas sem paralelismo de processos.

C.6 Estrutura de Arquivos do Projeto

Tabela C.2: Estrutura principal de arquivos do repositório *Hephaestus*.

Arquivo	Descrição
<code>main.py</code>	Ponto de entrada; interface gráfica (<i>CustomTkinter</i>) com as abas Modelagem, Simulação e Análise
<code>engine.py</code>	<i>Engine</i> matemática: classes <code>RobotMathEngine</code> e <code>RobotMathHydro</code> ; derivação simbólica de M , C , G , J
<code>simulator.py</code>	Simulador numérico: classe <code>RobotSimulator</code> ; controladores <code>SlidingModeControl</code> , <code>SuperTwistingSMC</code> , <code>Decentralized_LADRC</code> ; integrador simplético
<code>tooltip_utils.py</code>	Utilitários de interface: tooltips educacionais e blocos de ajuda contextual para os parâmetros físicos
TCC/	Capítulos do documento TCC em $\text{\LaTeX}$